DELL Technologies

Deployment and Configuration

Using VMware vSphere Storage APIs for Array Integration with Dell PowerMax

Increasing operational efficiency with VMware and Dell PowerMax

Abstract

This paper discusses how VMware's vSphere Storage APIs for Array Integration, also known as VAAI, offloads various virtual machine operations from the host to the Dell PowerMax.

April 2026

Revisions

Date	Description
March 2023	Initial release of new template
November 2023	PowerMaxOS 10.1.0.0, vSphere 8.0U2
March 2024	Explain how to add multiple claim rules for XCOPY when using multiple arrays
September 2024	PowerMaxOS 10.2.0.0, vSphere 8.0U3
October 2025	PowerMaxOS 10.3.0.0, vSphere 9.0
April 2026	PowerMaxOS 10.4.0.0

Acknowledgments

Author: Drew Tonnesen, Dell Engineering

The information in this publication is provided “as is.” Dell Inc. makes no representations or warranties of any kind with respect to the information in this publication and specifically disclaims implied warranties of merchantability or fitness for a particular purpose.

Use, copying, and distribution of any software described in this publication requires an applicable software license.

This document may contain certain words that are not consistent with Dell's current language guidelines. Dell plans to update the document over subsequent future releases to revise these words accordingly.

This document may contain language from third party content that is not under Dell's control and is not consistent with Dell's current guidelines for Dell's own content. When such third-party content is updated by the relevant third parties, this document will be revised accordingly.

Copyright © April 2026 Rev 1.40 Dell Inc. or its subsidiaries. All Rights Reserved. Dell Technologies, Dell, EMC, Dell EMC and other trademarks are trademarks of Dell Inc. or its subsidiaries. Other trademarks may be trademarks of their respective owners. [4/3/2026] [Deployment and Configuration] [h8115]

Table of contents

Revisions....2
Acknowledgments....2
Table of contents....4
Executive summary....6
1 VAAI primitives....8
1.1 Hardware-accelerated Full Copy or XCOPY....8
1.2 Hardware-accelerated Block Zero....9
1.3 Hardware-assisted locking....11
1.3.1 ATS heartbeat....13
1.4 UNMAP....15
1.4.1 Datastore....15
1.4.2 Guest OS....15
1.5 VAAI and HYPERMAX OS/PowerMaxOS version support....16
1.6 Compression/Deduplication/Data Reduction....17
1.7 Host IO Limits....17
1.8 Viewing VAAI support....18
1.9 VAAI Statistics....21
1.9.1 UNMAP Example....22
1.9.2 XCOPY Example....23
1.10 Enabling the Storage APIs for Array Integration....25
1.10.1 Full Copy, Block Zero, and Hardware-assisted locking....25
2 NVMe over Fabrics (NVMeoF)....27
2.1 VAAI....27
3 eNAS and PowerMax File....28
3.1 Extended stats....29
3.2 Nested clones....29
4 Full Copy Detail....31
4.1 Default copy size....31
4.2 XCOPY claim rules....32
4.2.1 Custom vendor rule....33
4.2.2 Virtual Storage Integrator....36
4.2.3 Claim rule results....38
4.3 SRDF and XCOPY....41
4.4 XCOPY with multiple arrays on an ESXi host....44

4.5 Caveats for using hardware-accelerated Full Copy 44
4.5.1 SRDF/Metro specific....45
4.5.2 Compression/Deduplication (Data Reduction) specific 46
5 Block Zero Detail 47
5.1.1 Eagerzeroedthick disks 47
5.1.2 Zeroedthick virtual disks 49
5.2 Caveats for using hardware-accelerated Block Zero 49
6 UNMAP Detail 50
6.1 Automated UNMAP 50
6.1.1 Reclamation priority....52
6.1.2 Changing reclamation values in vSphere....56
6.1.3 Reclamation hosts in 8.0U3....56
6.1.4 Automatic UNMAP support for SESparse 56
6.2 Recommendations....56
6.3 Manual UNMAP after VM deletion....57
6.4 UNMAP internals 60
6.4.1 Recommendation....60
6.5 Caveats for using UNMAP....61
7 Monitoring VAAI operations with ESXTOP 63
7.1 Monitoring VAAI with NFS 66
8 Performance....68
8.1 VDI....68
9 Conclusion....69
A Technical support and resources 70
A.1 Dell....70
A.2 VMware....70

Executive summary

VMware vSphere Storage APIs is a family of APIs used by third-party hardware, software, and storage providers, such as Dell, to develop components that enhance several vSphere features and solutions. This paper focuses on one component of those APIs known as VMware's Storage APIs for Array Integration (VAAI - also formerly known as vStorage APIs). Dell has four main API storage integrations as part of VAAI. These integrations are available with the HYPERMAX OS on the VMAX3/VMAX All Flash, and the PowerMaxOS on the PowerMax when running with VMware vSphere 8.0 and higher.[1] VAAI, by default, provides the ability to offload specific storage operations to the Dell PowerMax to increase both overall system performance and efficiency. The four main VMware's Storage APIs supported by PowerMax are:

- **Full Copy** - This feature delivers hardware-accelerated copying of data by performing all duplication and migration operations on the array. Customers can achieve faster data movement via VMware Storage vMotion®, and virtual machine creation and deployment from templates and virtual machine cloning. The term Full Copy and the XCOPY SCSI command refer to the same thing.
- **Block Zero** - This feature delivers hardware-accelerated zero initialization, reducing common input/output tasks such as creating new virtual machines. This feature is especially beneficial when creating fault-tolerant (FT)-enabled virtual machines or when performing routine application-level Block Zeroing.
- **Hardware-assisted locking** - This feature delivers improved locking controls on Virtual Machine File System (VMFS), allowing far more virtual machines per datastore and shortened simultaneous block virtual machine boot times. This improves performance of common tasks such as virtual machine migration, powering many virtual machines on or off and creating a virtual machine from a template.
- **UNMAP** - This feature enables the reclamation of blocks of thin-provisioned LUNs by informing the array that specific blocks are obsolete. VMware offers both Guest OS UNMAP and automated UNMAP at the VMFS level.

In addition to the four main APIs, VAAI includes an API for Thin Provisioning. Thin Provisioning is enabled by default. Thin Provisioning encompasses a couple different capabilities. First, the API allows ESXi to tell the array when there is reclaimable space on a thin provisioned LUN. This is essential for UNMAP. Second, the API supports out of space errors. This means the array can notify ESXi when it is running out of space. The out of space errors capability is also known as "Stun & Resume" because when a VM runs out of space on the array, rather than crashing, it will enter a "stunned" state until the user increases space (or powers off the VM) on the array allowing the VM to "resume."

To determine whether a device supports the Thin Provisioning API, query it with esxcli:

```
esxcli storage core device list
```

One of the columns has the name "Thin Provisioning Status." When the device supports the API, this column will return "yes," when not supported the status will be either "no" or "unknown." In the screens below in Figure 1, there are two PowerMax devices, one that supports the API and one which does not.

[1] vSphere 7 reached End of Service on October 2, 2025.

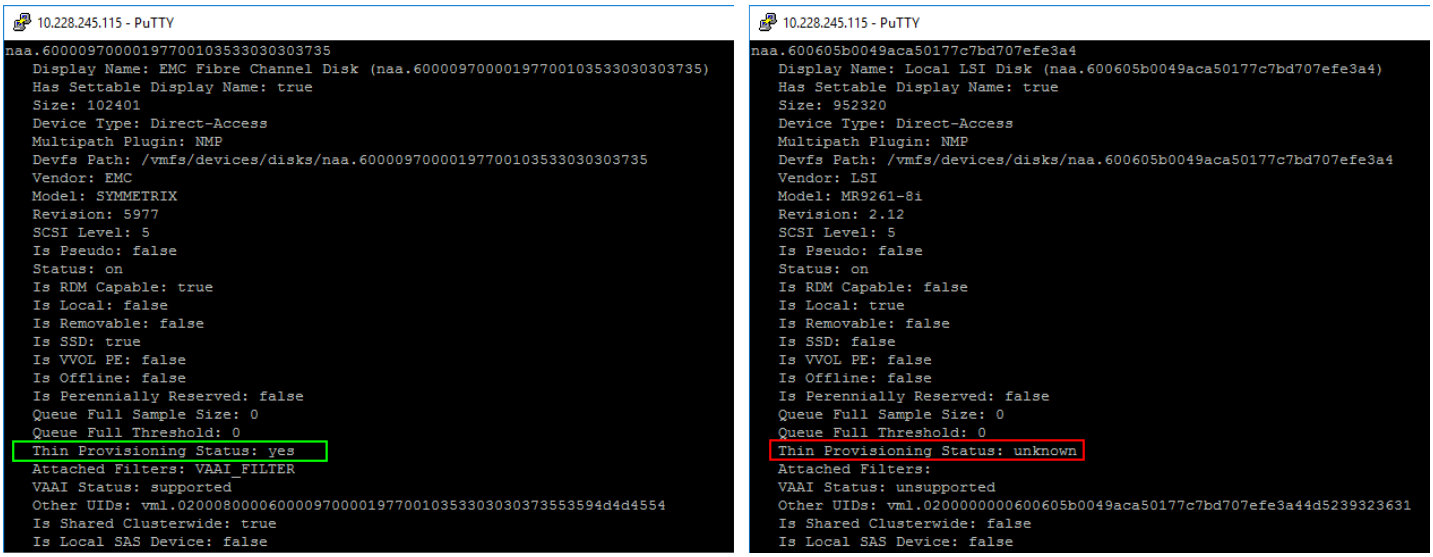


Figure 1. Thin Provisioning Status

When running the supported versions of HYPERMAX OS/PowerMaxOS and vSphere, VAAI functionality is enabled by default on any PowerMax array.

This paper will discuss the operation of and best use cases for utilizing VAAI features with the PowerMax. In doing so, it will demonstrate how to maximize efficiency and increase the effectiveness of a VMware vSphere environment deployed on a PowerMax.

This paper covers through the PowerMaxOS 10.4.0.0 release.

1 VAAI primitives

Storage APIs for Array Integration provide an interface that allows VMware storage partners to offload specific data management operations to the storage array. Offloading these operations minimizes host CPU, network, and I/O overhead, improving overall efficiency and performance. The PowerMax fully supports that API that enables four primary offload primitives:

- Full Copy – hardware-accelerated data copy operation
- Block Zero – hardware-accelerated block initialization
- Hardware-Assisted Locking – array-based coordination for block access control
- UNMAP – efficient reclamation of unused storage blocks

Note: The primitives will not cause failures in the vSphere environment if conditions prevent their use. In such cases, VMware simply reverts back to the default software behavior. For example, if the Full Copy primitive is enabled and a user attempts to clone a VM to a local datastore which does not support hardware acceleration, VMware will revert to the default software copy. The primitives may also be enabled or disabled even in the middle of operations that are utilizing them.

1.1 Hardware-accelerated Full Copy or XCOPY

The Full Copy primitive reduces the time it takes to deploy or migrate a virtual machine because the process runs on the storage array and not on the ESXi server. This minimizes overall traffic on the ESXi server. In addition to deploying new virtual machines from a template or through cloning, Storage vMotion utilizes Full Copy. When you migrate a virtual machine between datastores on the same array the hot copy users XCOPY.

Full Copy is an asynchronous process on the PowerMax. When using the PowerMax, the host simply passes information to the array about what data to copy or move and where to place it. The progress of this transfer of information is what the vSphere Client reports. When the vCenter reports that the clone or Storage vMotion task is complete, it means the PowerMax has received all the information it needs to perform the task. The array, using the same mechanism as the Dell TimeFinder local replication software, completes the transfer of data either synchronously or asynchronously, depending on the state of the device. The end-user is completely unaware of this since the virtual machine is available during the Storage vMotion, or in the case of a clone immediately after the vCenter task is complete.

To ensure that the Full Copy process does not over-utilize array resources, thereby impacting critical operations, the array has an upper ceiling of operations; however, reaching these limits is very unlikely. If it does occur, vSphere seamlessly switches to software copy in mid-operation. VMware vSphere will retry the XCOPY command after a brief time, at which point if resources are available, the array copy will resume. Note that a clone or Storage vMotion can switch back and forth between software and hardware copies without any user intervention, and in fact without the end-user knowing. If the user wishes to monitor such activity, the section Monitoring VAAI operations with ESXTOP in this paper provides detail.

Not only does Full Copy save time, but it also saves server CPU cycles, memory, IP and SAN network bandwidth, and storage front-end controller I/O. This is due to the fact that the host is relieved of the normal function it would serve of having to read the data from the array and then write the data back down to the array. That activity requires CPU cycles and memory as well as significant network bandwidth. Figure 2 provides a graphical representation of Full Copy.

Note: Dell uses the terms Full Copy and XCOPY interchangeably in this paper.

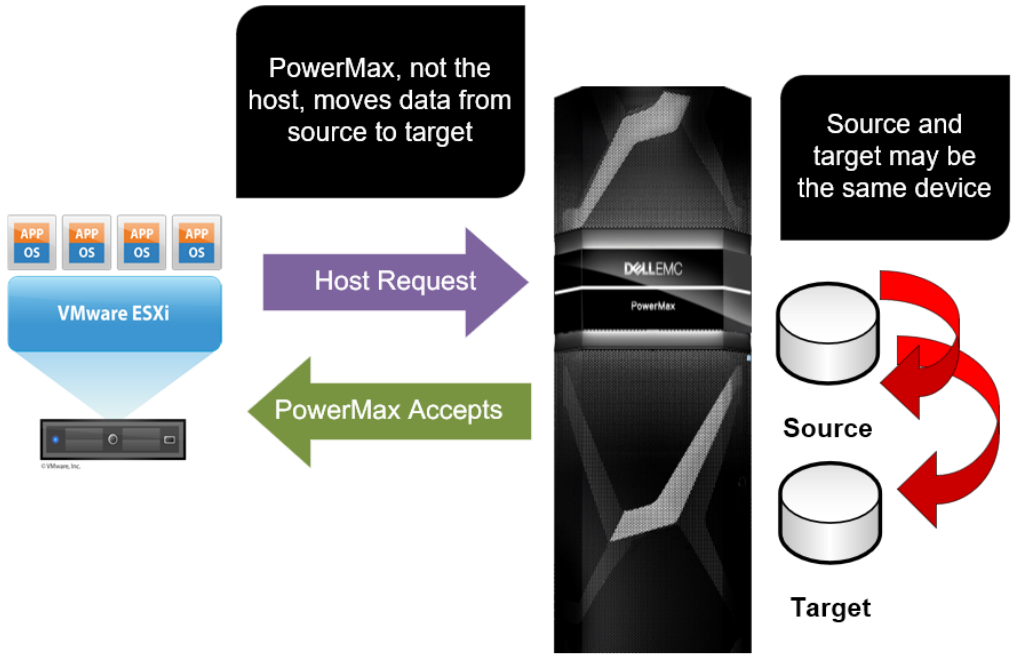


Figure 2. Hardware-accelerated Full Copy

1.2 Hardware-accelerated Block Zero

Using software to zero out a virtual disk is a slow and inefficient process. By passing off this task to the array, the user avoids the typical back and forth from host to array. Instead the PowerMax accomplishes the zeroing.

A typical use case for Block Zeroing is when creating virtual disks that are eagerzeroedthick (EZT) in format. Without the Block Zeroing primitive, the ESXi server must complete all the zero writes of the entire disk before it reports back that the disk zeroing is complete. For large disks this is time-consuming. When employing the Block Zeroing primitive as in Figure 3, (also referred to as "write same" since it uses the WRITE SAME SCSI operation (0x93) to achieve its task) the disk array returns the cursor to the requesting service as though the process of writing the zeros has been completed. It then finishes the job of zeroing out those blocks asynchronously on the array. Software zeroing, on the other hand, is a synchronous process.

The PowerMax does not actually write zeroes or reserve space. This is because the PowerMax is a single pool of storage, since new extents can be allocated from any thin pool on the array. Therefore, the only way that a thin device can run out of space is if the entire SRP runs out of space. It is unnecessary, therefore, to reserve space on the PowerMax when using Block Zero. The array must process the WRITE SAME commands regardless. Note, however, that because of the architecture of the PowerMax there is no performance impact even though the space is not pre-allocated.

VMware also initiates Block Zero with "zeroedthick" or "thin" virtual disks when a guest operating system writes to a previously unallocated portion of the disk. When a new write comes through, ESXi issues write same to the block(s) and then executes the write. The array uses the Block Zero operation only once per block upon a first write while subsequent writes to the same block do not require a Block Zero operation. An initial format of a VMFS volume to reserve space for VMFS metadata uses Block Zero. VMware only formats and reserves a small portion of the device used by VMFS.

This primitive has a significant impact when deploying "eagerzeroedthick" virtual disks or writing to unallocated sectors of a “zeroedthick” or “thin” virtual disk. When enabled, the PowerMax will allocate the relevant tracks (if not already allocated) but mark the tracks as never written. Block Zero operations therefore result in allocated, but not written to, tracks. Tracks such as these will always return zeros to a host read and in the case of a TimeFinder or SRDF, it skips them.

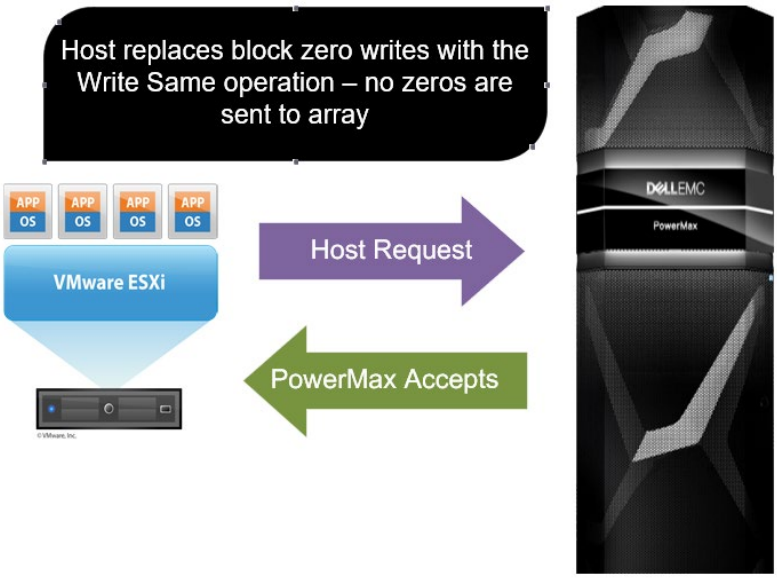


Figure 3. Hardware-accelerated Block Zero

Without Block Zero, VMware will send zeros and the PowerMax will allocate them. If the storage group where the datastore resides has data reduction enabled, however, the PowerMax will remove the zeros and deallocate the tracks when it is destaged from cache. The only condition where this does not hold true is if the device writes to Storage Class Memory (SCM) as the PowerMax will not use data reduction on that media. In such circumstances, or if data reduction is not in use, the storage can be deallocated by using the volume reclaim functionality in Unisphere for PowerMax, the steps are in Figure 4.

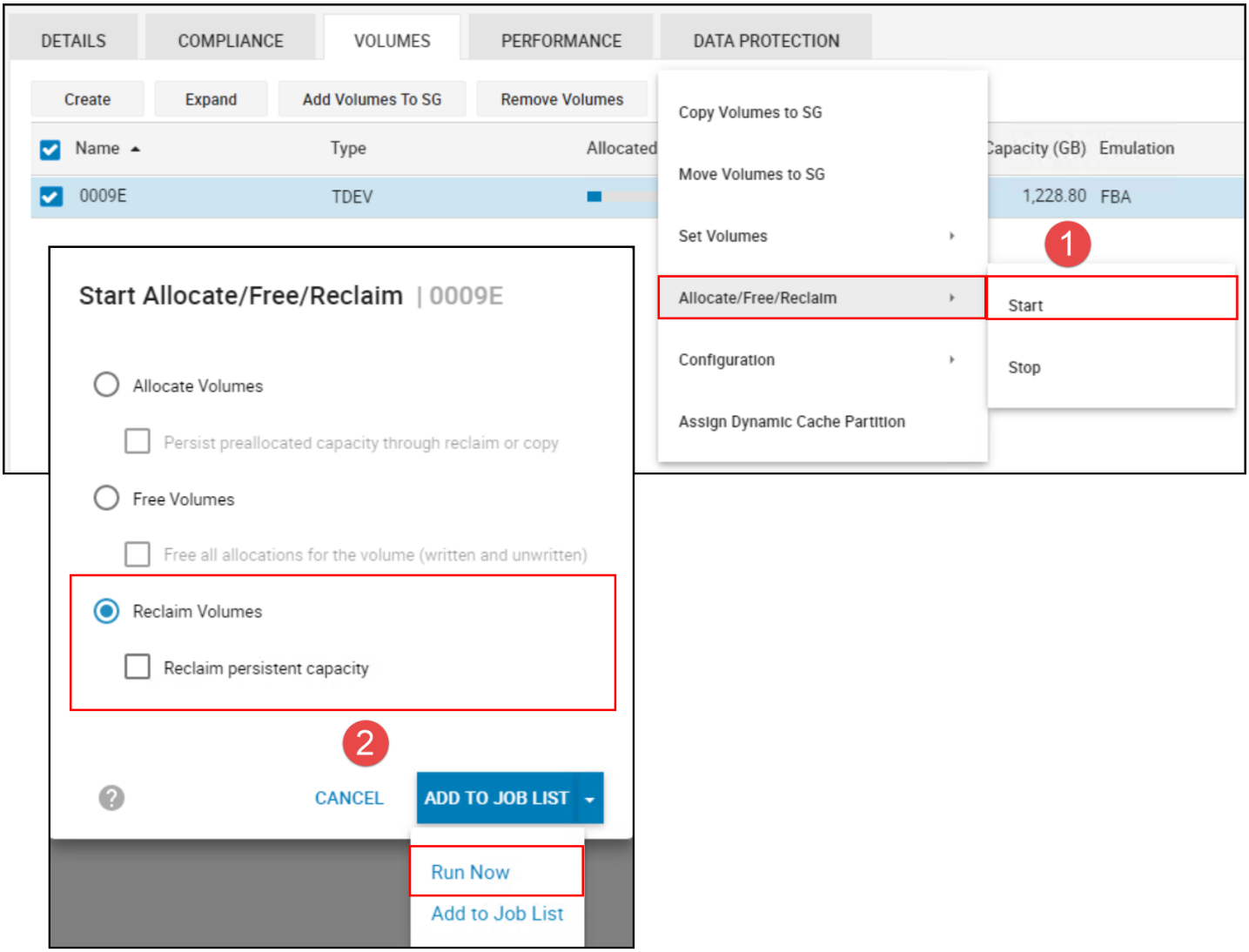


Figure 4. Reclaim zeroed out space

Note: Storage reclamation is a background process on the array so as to avoid host IO impact. When freeing enormous amounts of data, it will take time to see the results in the SRP.

1.3 Hardware-assisted locking

As a clustered shared-storage manager, VMware's Virtual Machine File System (VMFS) needs to coordinate the access of multiple ESXi server hosts to parts of that shared storage. VMFS allocates part of the storage available to it for data describing virtual machines and their configurations, as well as the virtual disks that they access. Within a cluster of ESXi servers you can run the virtual machines on any of the ESXi instances. You can also move them between instances for load balancing and high availability.

VMware implements locking structures within the VMFS datastores that it uses to prevent any virtual machine from running on more than one ESXi host at a time. VMware uses this locking metadata update operation when changing a virtual machine's state. Activities that generate virtual machine locks include power management, configuration changes, vMotion or DRS. VMware built the initial implementation of mutual exclusion for updates to these locking structures using SCSI

RESERVE and RELEASE commands. This protocol claims sole access to an entire logical unit for the reserving host until it issues a release. Under the protection of a SCSI RESERVE, a server node could update metadata records on the device without the risk of interference from any other host attempting to update the same records. This approach, present in Figure 5, has significant impact on overall cluster performance since VMware prevents access to the device by other hosts while SCSI RESERVE is in effect. As ESXi clusters have grown in size, as well as in their frequency of modifications to the virtual machines they are running, the performance degradation from the use of SCSI RESERVE and RELEASE commands has become unacceptable.

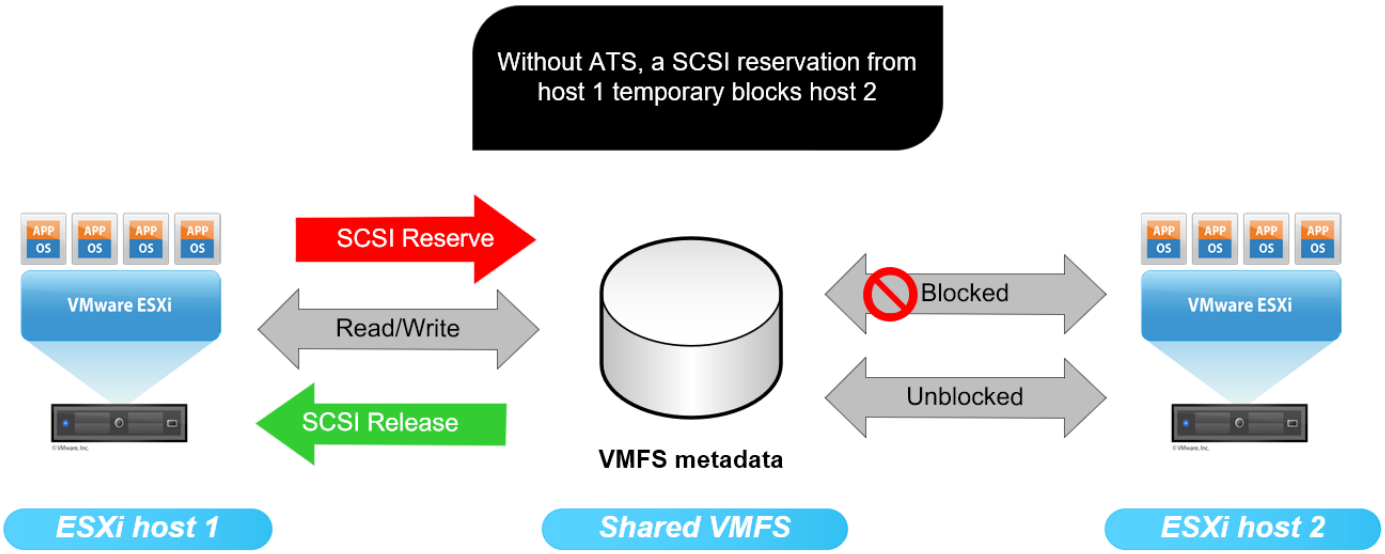


Figure 5. Traditional VMFS locking before hardware-assisted locking (ATS)

This led to the development of the third primitive for VAAI, hardware-assisted locking or ATS. This primitive provides a more granular means of protecting the VMFS metadata than SCSI reservations. Hardware-assisted locking leverages a storage array atomic test and set capability to enable a fine-grained block-level locking mechanism as shown in Figure 6. Firstly, hardware-assisted locking replaces the sequence of RESERVE, READ, WRITE, and RELEASE SCSI commands. It does this with a single SCSI request for an atomic read-modify-write operation conditional on the presumed availability of the target lock. Secondly, this new request only requires exclusivity on the targeted locked block, rather than on the entire VMFS volume containing the lock.

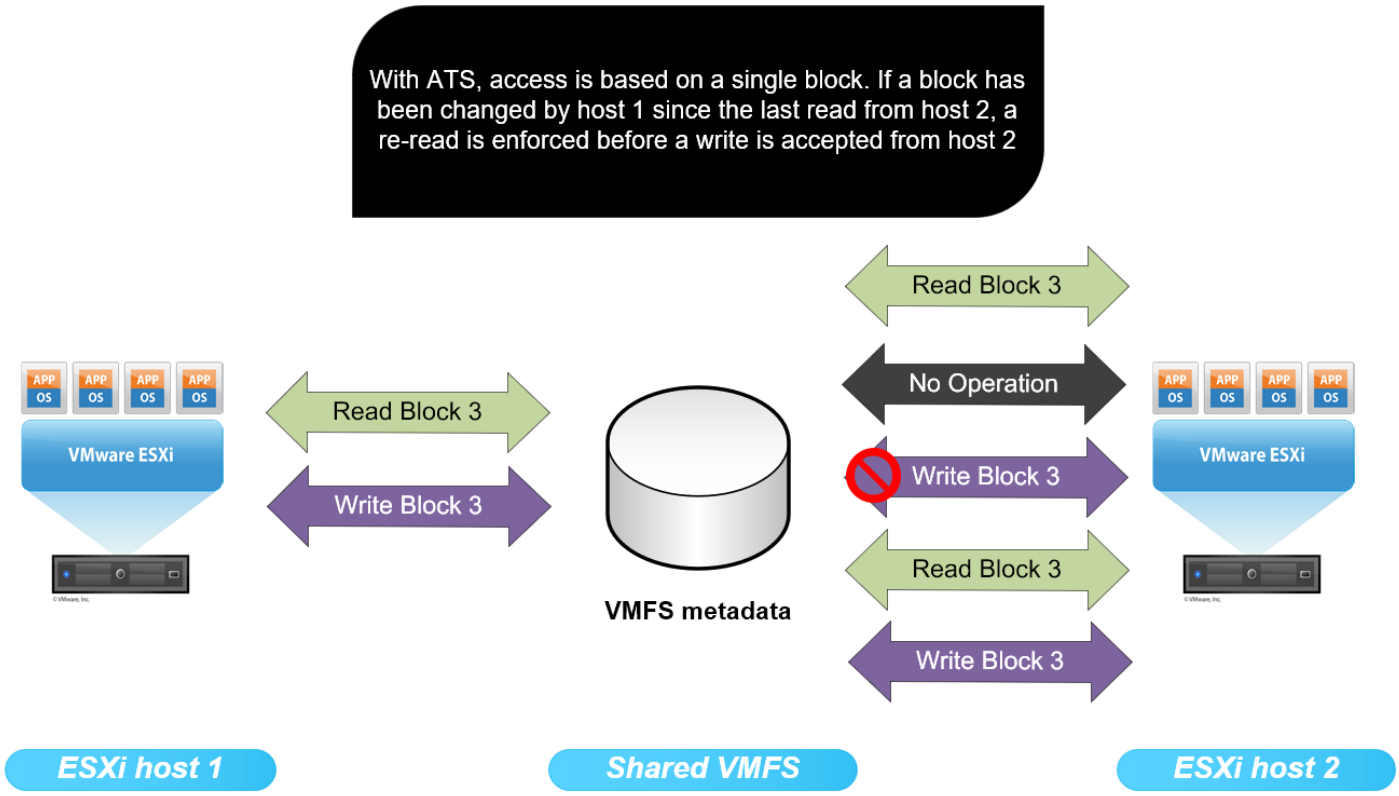


Figure 6. VMFS locking with hardware-assisted locking (ATS)

Although the non-hardware assisted SCSI reservation locking mechanism does not often result in performance degradation, the use of hardware-assisted locking provides a much more efficient means to avoid retries for getting a lock when many ESXi servers are sharing the same datastore. Hardware-assisted locking enables the offloading of the lock mechanism to the array and then the array does the locking at a very granular level. This permits significant scalability in a VMware cluster sharing a datastore without compromising the integrity of the VMFS shared storage-pool metadata.

1.3.1 ATS heartbeat

One of the types of locks that VMware performs is the heartbeat. The heartbeat synchronizes operations between multiple hosts as well as check on the vitality of the host. In the initial implementation, VMware used SCSI reads and writes for the heartbeat, even on storage arrays that supported VAAI (ATS). Soon after, however, VMware defaulted to using ATS on supported arrays, offloading the heartbeat management. The change in behavior means there is a significant increase in the ATS traffic to the storage array. Under certain load conditions, therefore, ATS may fail with a false ATS miscompare or similar ATS error.[2] Upon failure ESXi will then re-verify its access to the datastore which will lead to error messages in the vSphere Client similar to: ***Lost access to datastore***. VMware writes the ATS miscompare messages to the /var/log/vmkernel.log file, along with other ATS failures.

[2] VMware KB article 2113956. Although the KB refers to an IBM storage system, the error can occur on any storage platform.

An ESXi host will heartbeat every three seconds. After five failed attempts it will set the datastore inaccessible.

1.3.1.1 Disabling ATS heartbeat

In the unlikely event that ATS errors are persistently occurring in the environment, VMware and Dell recommend disabling ATS for the heartbeat mechanism. Disabling the function means ESXi will revert to the non-ATS functionality, or SCSI reads and writes. VMware indicates that the performance implications of reverting to previous functionality are not significant, thus, it is safe to do so.

Like most VAAI functions, disabling the ATS heartbeat is an online task. To view the current setting (Int Value) for the ATS heartbeat, execute the command in Figure 7:

```
esxcli system settings advanced list -o /VMFS3/UseATSForHBOnVMFS5
```

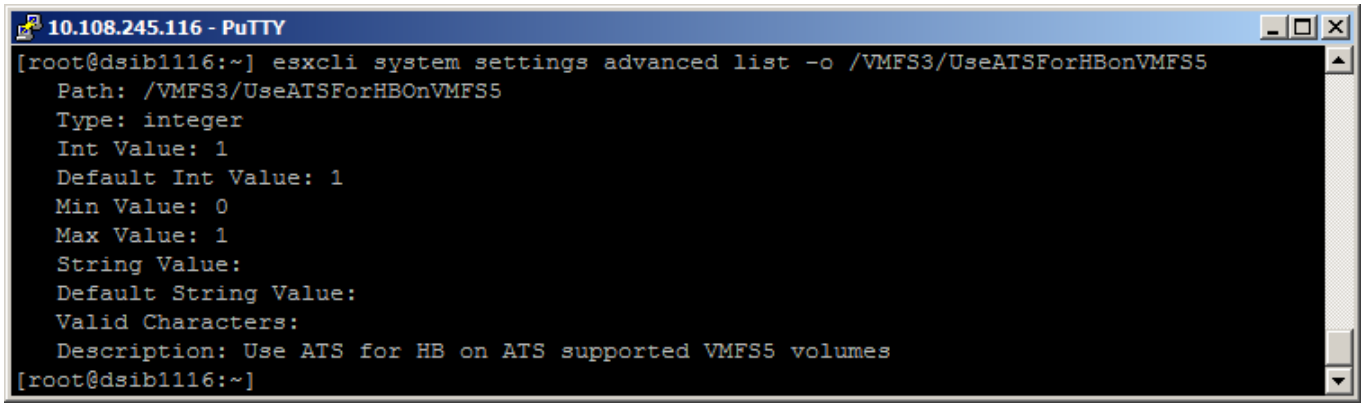


Figure 7. ATS heartbeat setting

As noted in Figure 7, the default value is on (1). In order to disable it, run the following command on each host that accesses the datastore in question. Note it will not return a response:

```
esxcli system settings advanced set -i 0 -o /VMFS3/UseATSForHBOnVMFS5
```

The output is present in Figure 8.

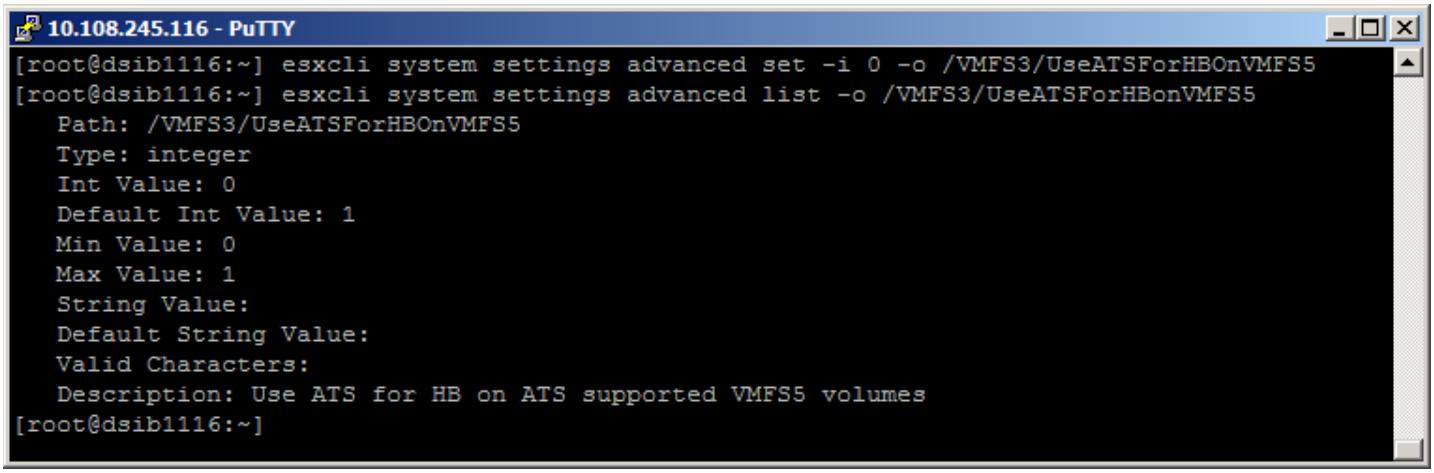


Figure 8. Disabling ATS heartbeat setting

Note: Disabling the ATS heartbeat does not impact any other locking functions that VMware manages by the VAAI ATS primitive. Its sole impact is on the creation or updating of the VMFS heartbeat. The ATS heartbeat is a subset of functionality of ATS. Dell does not recommend disabling the ATS primitive (*VMFS3.HardwareAcceleratedLocking)* itself.

1.4 UNMAP

The purpose behind UNMAP is the reclamation of space in the thin pool on the array. VMware utilizes UNMAP in two different scenarios. The first is to free space in a VMFS datastore – manually or automatically. The second is to free space in a thin vmdk that is part of a Guest OS in a VM. This is known as Guest OS, or in-guest UNMAP. It supports both VMFS and vVol datastores.

1.4.1 Datastore

The lifecycle of a virtual machine invariably strands space in the VMFS and underlying device. The array is unaware of this obsolete data and thus cannot remove it. Prior to the integration of this new Storage API in vSphere and in the PowerMax, reclamation of space required zeroing out the unused data in the pool and then running a zero reclaim process on the array. With UNMAP support, vSphere tells the PowerMax what blocks can be unmapped or reclaimed. If those blocks span a complete extent or extents, they are deallocated and returned to the thin pool for reuse. If the range covers only some tracks in an extent, the PowerMax marks those tracks as not written by host; however, the extent cannot be deallocated. This is still beneficial, however, since the array will not retrieve those tracks from disk during a read request. Instead, the PowerMax array sees the track as not written and immediately returns all zeros.

VMware will reuse existing space in a datastore even if it has not unmapped the data; however, testing shows that VMware will use new space in the datastore before reusing existing. For example, a user creates a new VM in a 1 TB datastore which allocates 400 GB of actual space on the array. This datastore is not using automated UNMAP. The user then deletes the VM and creates a new one of the same size. The space allocation on the array now shows 800 GB because VMware used new space before old. Once all the new space is exhausted, VMware will reuse the old space occupied by that original 400 GB VM, but now the entire device is allocated. Automated UNMAP avoids this condition. In addition, by reclaiming the space in the thin pool on the PowerMax it becomes available to any device in the pool, not just the one backing the datastore. The use of the SCSI command UNMAP, whether issued manually or automatically, is the only way you can fully reclaim the space in the datastore.

Note: Automatic UNMAP does not necessarily remove all unused extents in a datastore for a variety of reasons. If it is critical that you reclaim all possible unmappable extents, no matter how small, Dell recommends running an occasional manual UNMAP.

1.4.2 Guest OS

VMware supports Guest OS or in-guest UNMAP in both Windows and Linux. Follow these prerequisites to enable Guest OS UNMAP:

- Thin virtual disk (vmdk)
- VM hardware version 13+
- If using Linux, the OS and file system must support the UNMAP command.

- The advanced parameter **EnableBlockDelete** must be set to 1 if using VMFS 5 (vVols do not require the parameter). It is off by default.

```
esxcli system settings advanced set --int-value 1 --option
/VMFS3/EnableBlockDelete
```

While Windows automatically issues UNMAP, Linux does not do so by default. Though `sg_unmap` and `fstrim` can be issued manually to free space, a simple parameter is available on file systems to reclaim space automatically. When mounting the file system that supports UNMAP (e.g., ext4), use the `discard` option and all deleted files will cause UNMAP to be issued. Here is an example.

```
mount /dev/sdb1 /u02 –o discard
```

1.5 VAAI and HYPERMAX OS/PowerMaxOS version support

Dell's implements VAAI within the HYPERMAX OS and PowerMaxOS software. As with any software, bugs are an inevitable byproduct. While Dell makes every effort to evaluate each use case for VAAI, it is impossible to mimic every customer environment and every type of use of the VAAI primitives. To limit any detrimental impact customers might experience with VAAI, Dell provides the following table, Table 1. It includes all current releases, and any recommend E Packs, beginning with the recommended base level of 5977 to the current PowerMaxOS 10.4.0.0 release. Using this table will ensure that a customer's array software release is at the proper level for VAAI interaction. It is essential to review both the Important Notes column and any footnotes.

Table 1. HYPERMAX OS/PowerMaxOS and VAAI support

HYPERMAX OS/PowerMaxOS code level	E Pack required	VAAI primitives supported	Important Notes
PowerMaxOS 10.4.0.0 (6079)	No	ATS, Block Zero, Full Copy, UNMAP	NVMeoF does not support the Full Copy (XCOPY) primitive. Supports use of XCOPY with vmkfstools.
PowerMaxOS 10.3.0.0 (6079)	Yes, Q126	ATS, Block Zero, Full Copy, UNMAP	NVMeoF does not support the Full Copy (XCOPY) primitive. E Pack enables XCOPY for vmkfstools.
PowerMaxOS 10.2.0.0 (6079)	No	ATS, Block Zero, Full Copy, UNMAP	NVMeoF does not support the Full Copy (XCOPY) primitive.
PowerMaxOS 10.1.0.0 (6079)	No	ATS, Block Zero, Full Copy, UNMAP	NVMeoF does not support the Full Copy (XCOPY) primitive.
5978.720.720	Yes, Q126	ATS, Block Zero, Full Copy, UNMAP	NVMeoF does not support Full Copy (XCOPY) or Block Zero (WRITE SAME) primitives. E Pack enables XCOPY for vmkfstools.
5978.669.669	No	ATS, Block Zero, Full Copy, UNMAP	NVMeoF does not support Full Copy (XCOPY) or Block Zero (WRITE SAME) primitives.

5978.479.479	No	ATS, Block Zero, Full Copy, UNMAP	
5978.444.444	No	ATS, Block Zero, Full Copy, UNMAP	First release to support VAAI metrics.
5978.221.221	No	ATS, Block Zero, Full Copy, UNMAP	
5978.144.144	No	ATS, Block Zero, Full Copy, UNMAP	This is the first PowerMaxOS release for the new PowerMax platform.
5977.1131.1131	No	ATS, Block Zero, Full Copy, UNMAP	Recommended SR on top of 5977.1125.1125.

Note: Be aware that this table only displays information related to using vSphere with VAAI and HYPERMAX OS/PowerMaxOS. It is not making any statements about general vSphere support with HYPERMAX OS/PowerMaxOS. For support information consult E-Lab Navigator. It may also not contain every release available.

1.6 Compression/Deduplication/Data Reduction

Starting with HYPERMAX OS release 5977 2016 Q3 SR, Dell offers compression on VMAX All Flash arrays. Starting with PowerMaxOS 5978 on PowerMax, Dell supports both compression and deduplication, referred to as Data Reduction. Compression and deduplication are not available on VMAX3 arrays and deduplication is not available on VMAX All Flash arrays. The VAAI primitives work with compression and deduplication without user intervention.

1.7 Host IO Limits

Dell provides support for the setting of Host I/O Limits. This feature allows users to place limits on the front-end bandwidth and IOPS consumed by applications on PowerMax systems.

Limits are set at the storage group level. As users build masking views with these storage groups, the array distributes the limits for maximum front-end IOPS or MBs/s across the directors within the associated masking view. The array monitors the bandwidth and IOPS to ensure that they do not exceed the user-specified maximum, throttling when necessary.

VAAI interacts with Host I/O Limits in that the commands issued by VMware to the array count toward either IOPS or MBs/s. From an IOPS perspective, the array counts these commands as individual IOs and registers them as regular metrics based on front-end activity. From a bandwidth perspective, UNMAP and Full Copy the array counts the commands as individual IOs of modest size since the code does not know the actual extent sizes. But the address ranges that VMware passes with the Block Zero commands count against the Host I/O MBs/s limit. This activity will show up in the Host IO Limit MBs/sec metric, while not showing up in the regular Host MBs/sec metric.

In the following example, we set a 50 MB/s Host I/O Limit on a storage group with a single 500 GB device. We then created a 250 GB eagerzeroedthick disk on the device. Figure 9 shows that the Write Same commands accrue against the limit (yellow line), but the array is not registering it as regular IO (blue line). Note the time it takes to create the vmdk - more than 1 hour. The normal creation time is about 30 seconds. This example exaggerates by using a small size for the limit in order to demonstrate how the VAAI and Host I/O Limits interact.

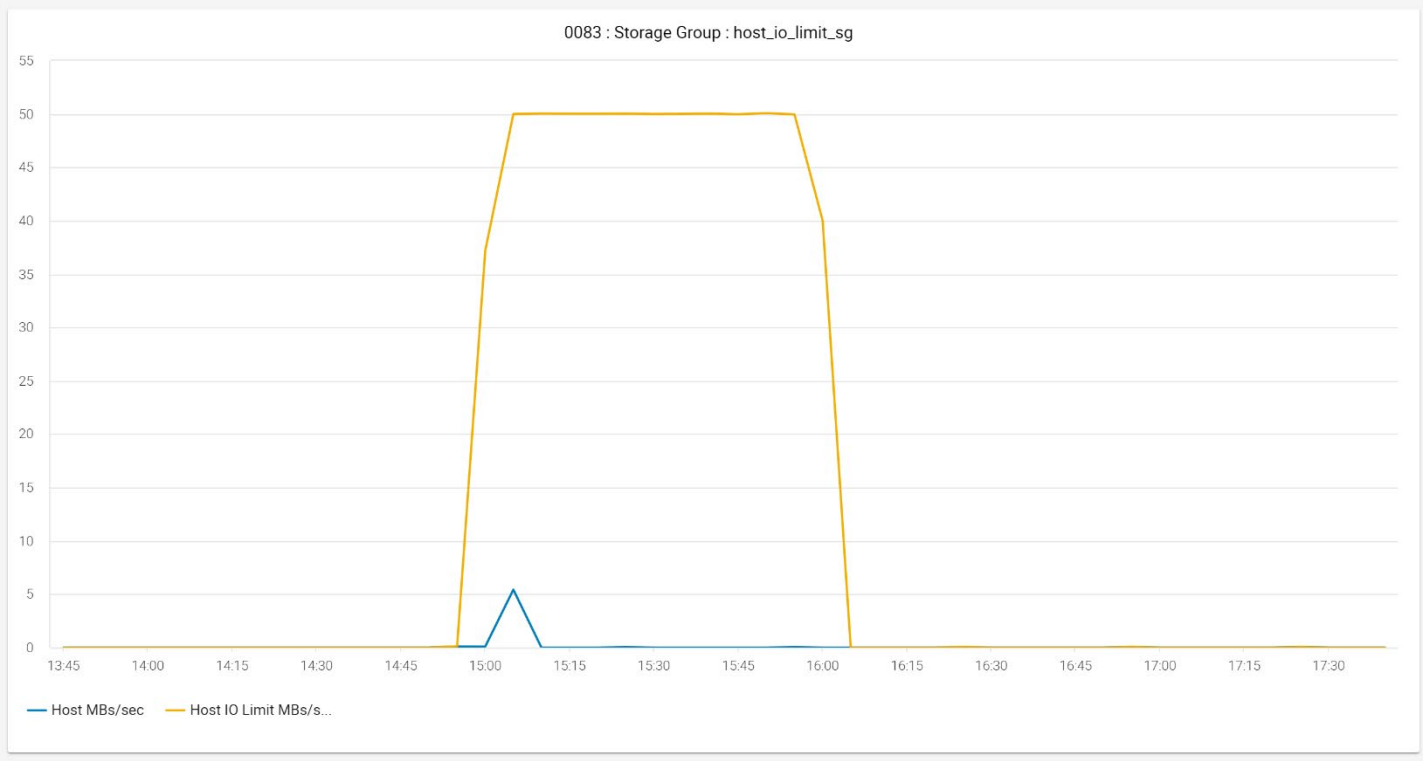


Figure 9. Host IO Limits with VAAI

1.8 Viewing VAAI support

Through the CLI or within the vSphere Client, you can see whether a particular device or datastore supports hardware acceleration or VAAI. From the CLI, using the Network Addressing Authority (NAA) identifier, you can see the status for each VAAI primitive in Figure 10. Note there is an additional row for cross-array XCOPY which the PowerMax does not support.

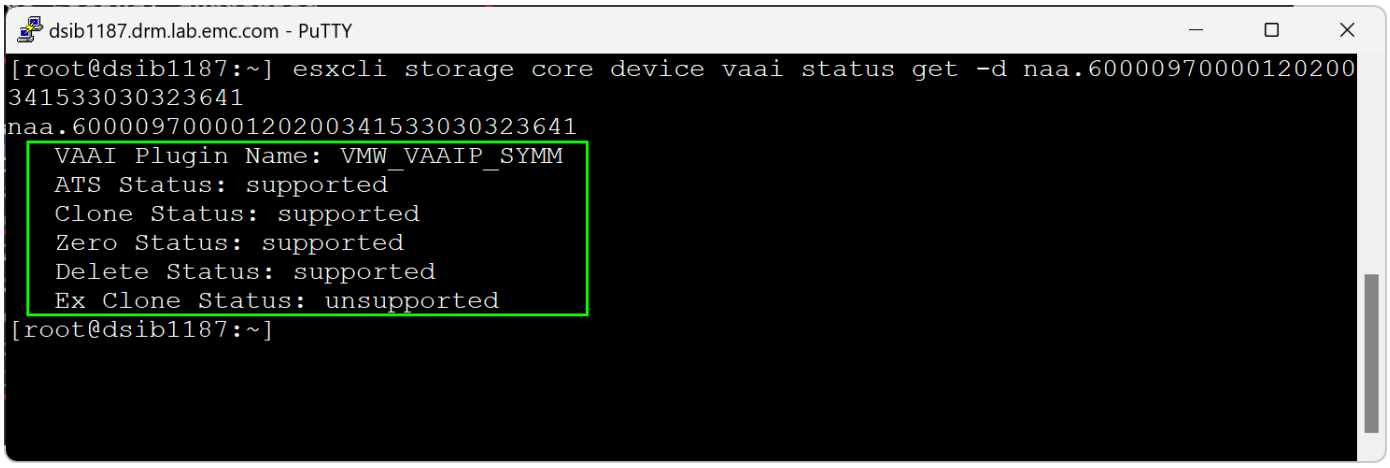


Figure 10. Viewing VAAI primitive support in vSphere

It is important to remember that querying the device provides the status from the array's perspective and not vSphere's. This means that if you disable a primitive within ESXi, the query will still show the primitives as supported by the array. As it is possible to disable primitives on the array itself, however, the query could return an unsupported value for one of those primitives. Figure 11 shows the status when you disable UNMAP and XCOPY at the array level. Note that ESXi reports only Delete Status as unsupported and not Clone Status.

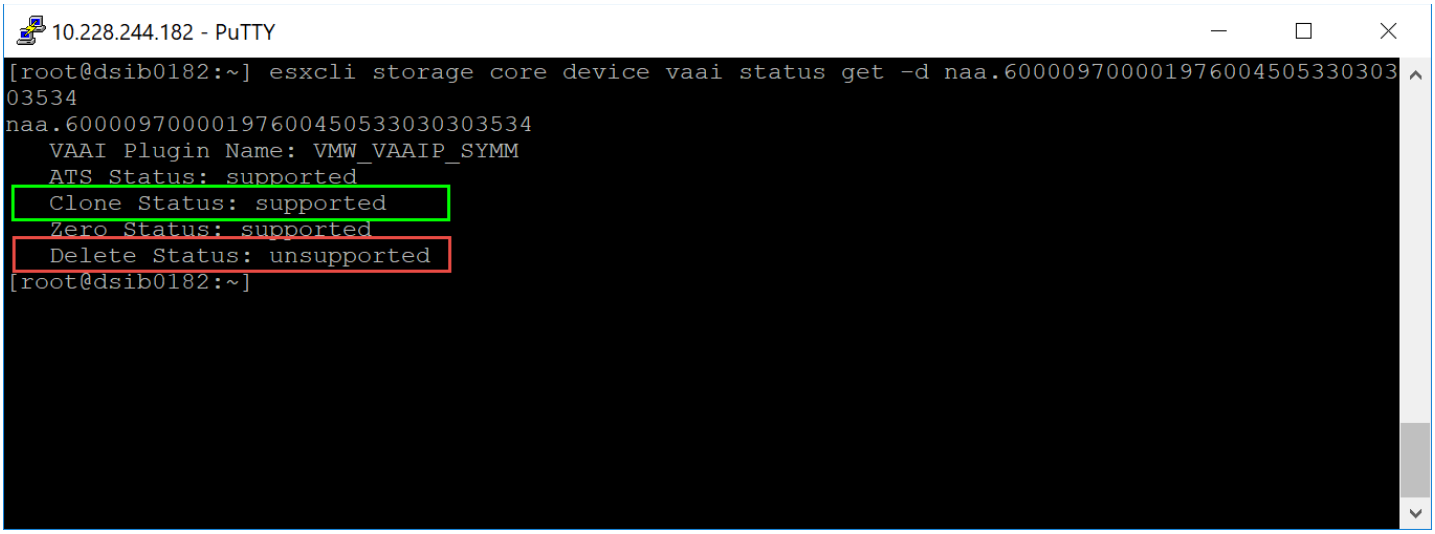


Figure 11. UNMAP support disabled on the array

If using eNAS or PowerMax File and thus NFS (see section eNAS and VAAI), to view VAAI support run the vmkfstools command shown in Figure 12.

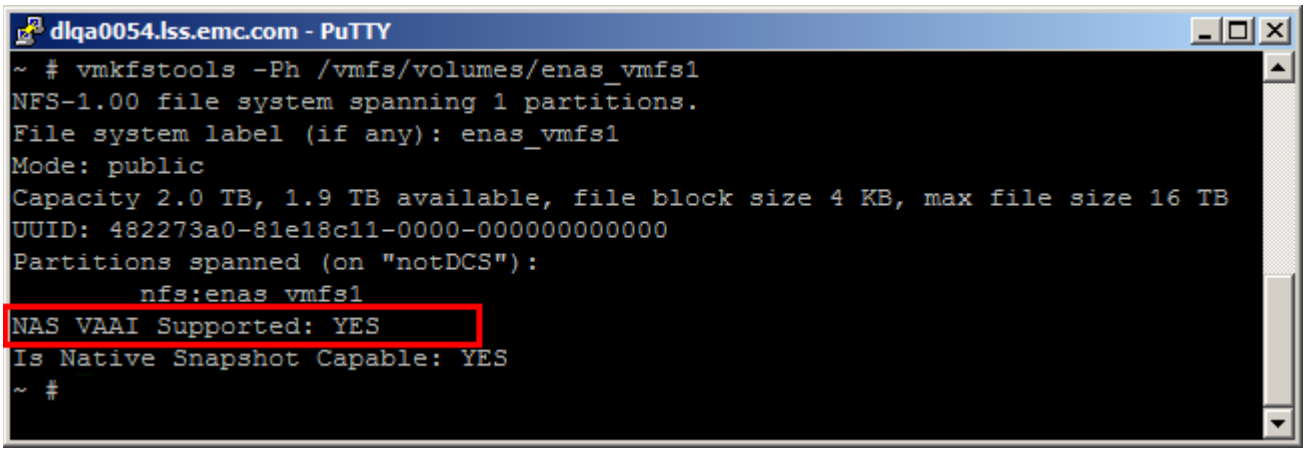


Figure 12. Viewing VAAI primitive support for NFS

In the vSphere to view VAAI support, first navigate to the chosen datastore in the left-hand panel. Then select **Configure -> Hardware Acceleration**. The **Hardware Acceleration** column indicates the support status of the primitives for the device or datastore. There are three values for this column: "Supported," "Not supported," or "Unknown"[3]. An example is in Figure 13.

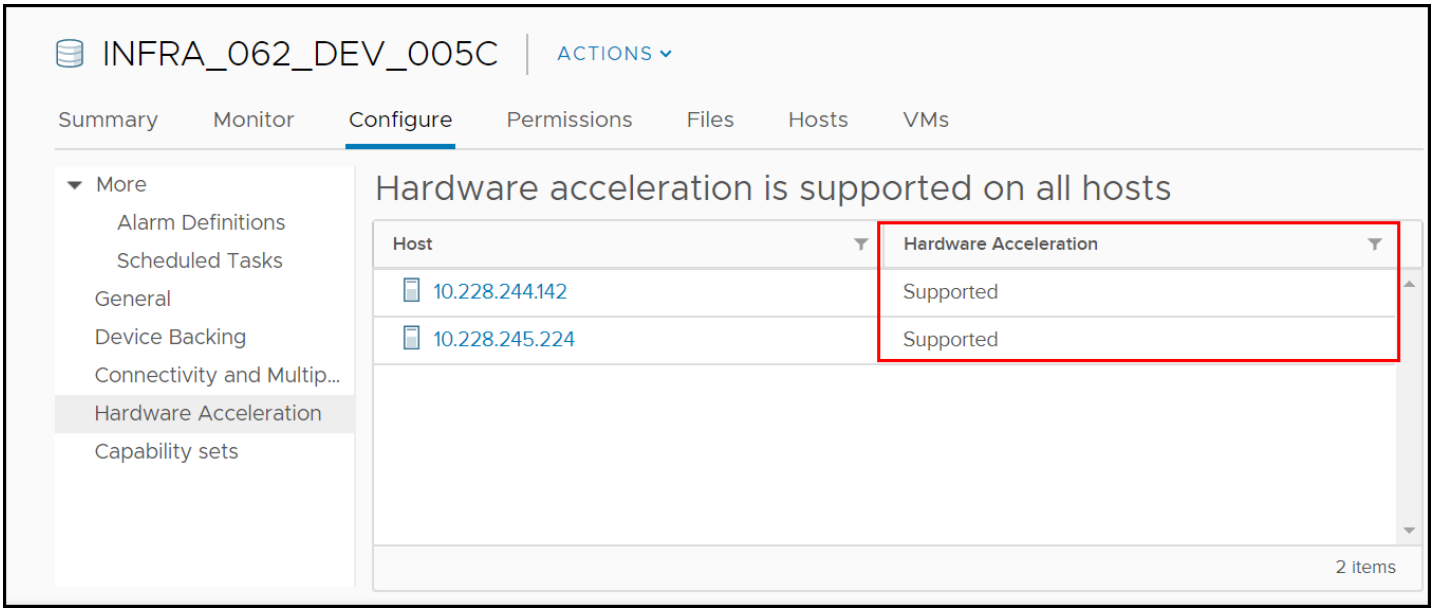


Figure 13. Viewing VAAI support in the vSphere Client

Note that this column, unlike the CLI, does not provide any detail about the individual primitives. For example, if the host supports hardware-accelerated locking but not UNMAP, this column will still show "Supported."

Note: The Hardware Acceleration column only shows support, not whether the primitives are enabled or disabled.

[3] A particular device or datastore is "Unknown" until successfully queried.

1.9 VAAI Statistics

Prior to PowerMaxOS 5978.444.444 there is no user interface to view VAAI primitive activity on the array. For Full Copy, however, if you run the Solutions Enabler command `symdev show` against the source or target device involved in a hardware-accelerated operation, the parameter “Extent Based Clone” will either display “Source” or “Target.”

Beginning with PowerMaxOS 5978.444.444, Unisphere for PowerMax includes the following VAAI statistics at the thin device[4] and storage group level:

- VAAI Total Command Count
- VAAI Total Time
- VAAI Unmap Command Count
- VAAI Unmap KB
- VAAI Unmap MB
- VAAI Write Same Command Count
- VAAI Write Same KB
- VAAI Write Same MB
- VAAI XCopy Command Count
- VAAI XCopy KB
- VAAI XCopy MB

In the Performance tab of Unisphere, you can find the VAAI metrics under **All** in Figure 14. The only VAAI metric that is a KPI is **VAAI Total Command Count**.

[4] These metrics are not available for vVols.

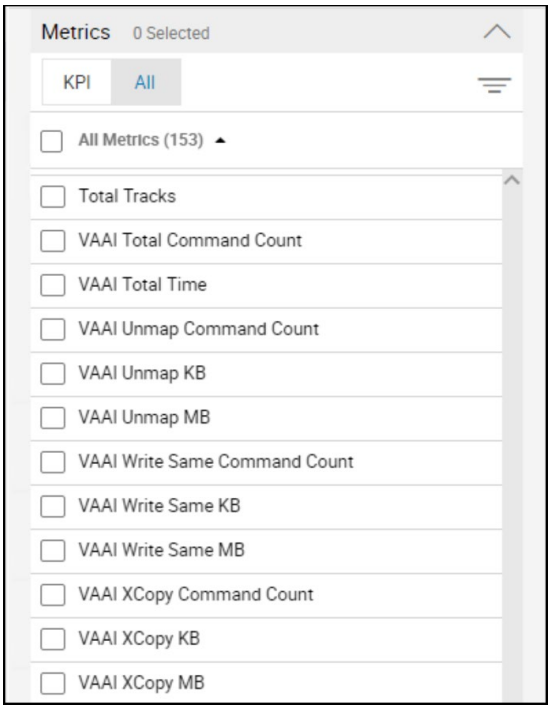


Figure 14. VAAI metrics in Unisphere

Note: More current versions of Unisphere reduced the number of VAAI metrics to eight, removing the MB measurements and renaming the KB measurements to “Throughput.”

The statistics available in Unisphere do not directly correlate with what VMware records for commands in *esxtop* so it is not beneficial to compare them; however, the Unisphere stats provide information not available in *esxtop*. For example, Unisphere can show the total reclaimed storage from the last auto UNMAP process.

1.9.1 UNMAP Example

In this example, we removed two VMs from a datastore with auto UNMAP enabled. VMware issued 3024 UNMAP commands, however the array only required ten commands to unmap the space on the array as seen in Figure 15. Those ten commands reclaimed about 215 MB of space.

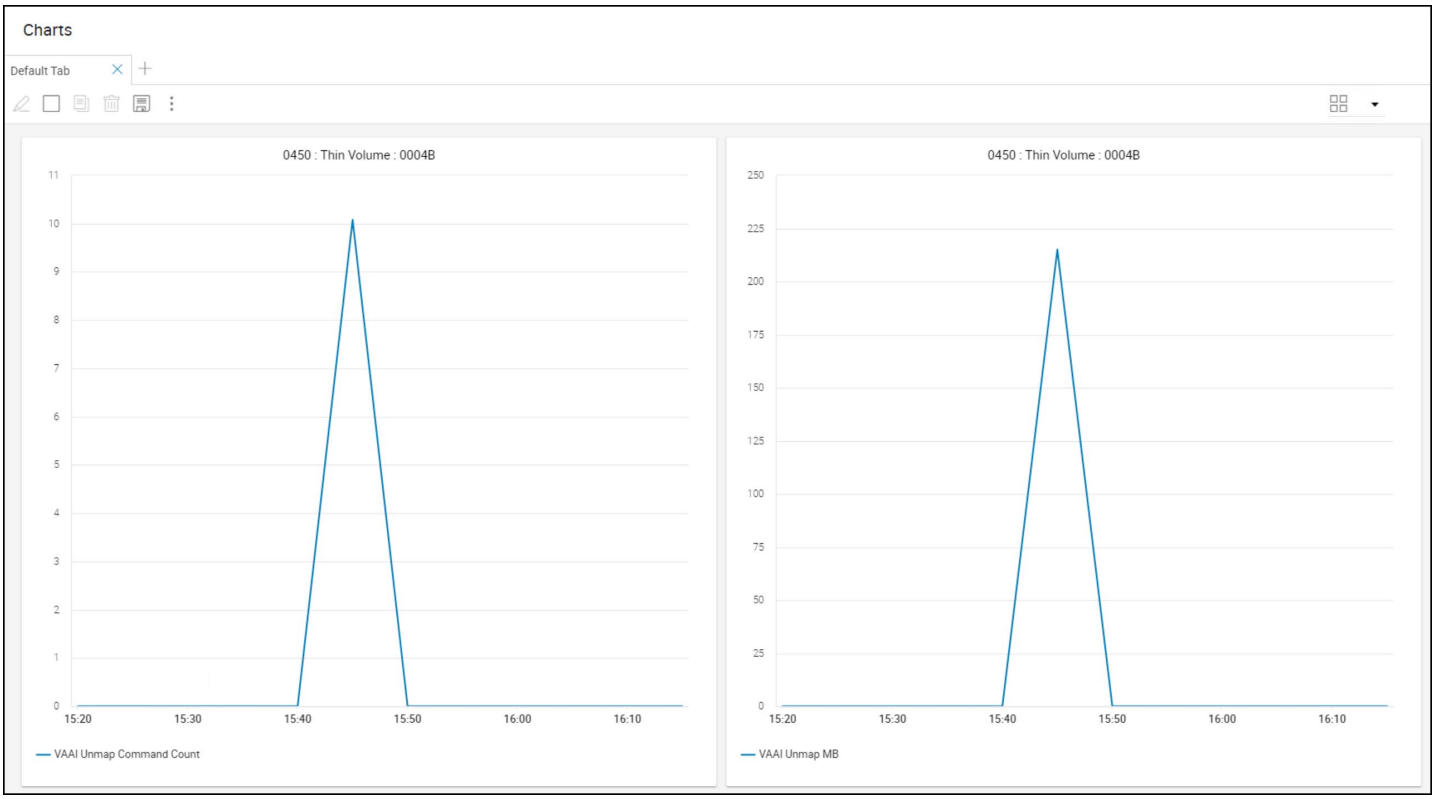


Figure 15. UNMAP statistics example

1.9.2 XCOPY Example

In the following example, we cloned a virtual machine within the same datastore which will engage XCOPY. We then executed two clones, the first using the default transfer size of 4 MB, and the second using the maximum transfer size (without a claim rule) of 16 MB. Note the values in Figure 16.

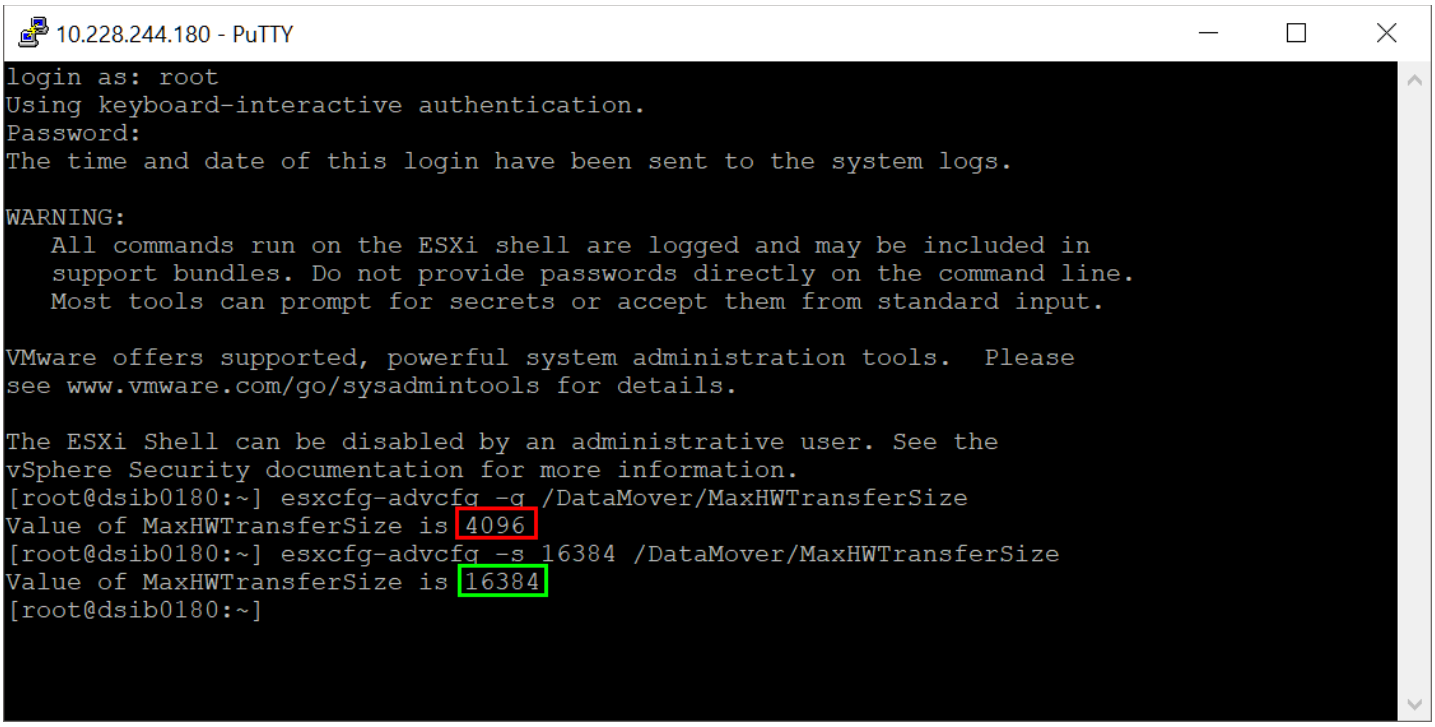


Figure 16. XCOPY transfer size

Figure 17 is a screenshot of the results of the two clones. Note the first run in the left of each graph where there were 12 XCOPY commands issued for a total of ~48 MB. When we change the transfer size to 16 MB and re-run the clone, notice that the command count reduces to three because of the copy size quadrupling. The size of the actual copied data, however, does not change. Remember, though, that the max transfer size is just that, a max, and that depending on the data, there may not always be perfect ratio between the original copy count and the one done after changing the transfer size.

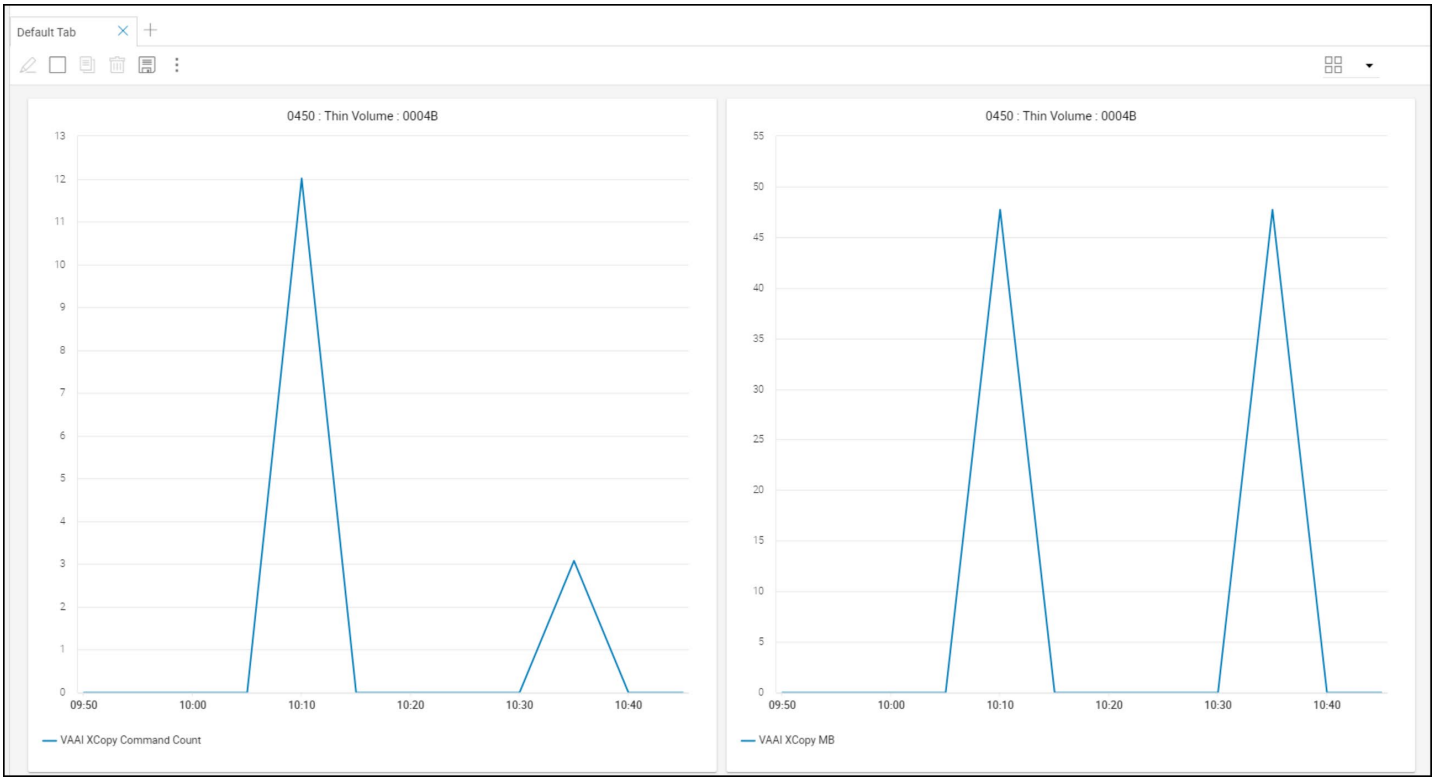


Figure 17. XCOPY statistics example

The new statistics are obviously most useful to tell when a device or devices in a storage group participate in VAAI activity. Knowing this information can help diagnose performance issues or in determining if VAAI commands are being issued.

1.10 Enabling the Storage APIs for Array Integration

This section on enabling and disabling VAAI does not apply to VAAI with NFS. Once you install the NAS plug-in, VAAI for NFS is enabled. You cannot disable it using any of the methods described in this paper.

1.10.1 Full Copy, Block Zero, and Hardware-assisted locking

The VAAI primitives are enabled by default on both the PowerMax using ESXi servers (properly licensed[5]) and should not require any user intervention. You can disable three of the primitives through the ESXi server if desired, either through the CLI or UI. Using the vSphere Client, you can disable or enable Full Copy and Block Zero independently by altering the respective settings, ***DataMover.HardwareAcceleratedMove*** and ***DataMover.HardwareAcceleratedInit***, in the ESXi server advanced settings under DataMover as shown in Figure 18. Under normal circumstances, do not alter these settings.

[5] Refer to VMware documentation for required licensing to enable VAAI features.

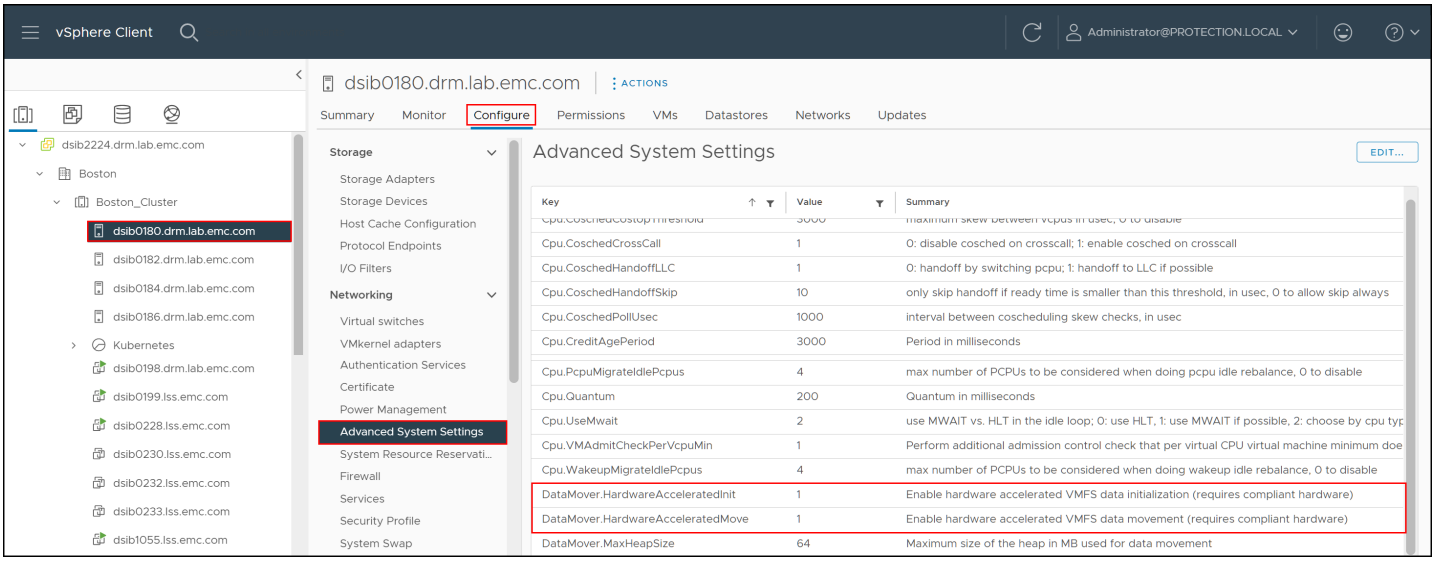


Figure 18. Enabling VAAI in the vSphere Client

Disabling or enabling the primitives is a dynamic process on the ESXi server and does not require the ESXi server to be in maintenance mode, nor does it necessitate a reboot of the server.

2 NVMe over Fabrics (NVMeoF)

NVMe or non-volatile memory express is a set of standards which define a PCI Express (PCIe) interface used to efficiently access data storage volumes on NVM. A consortium of companies, including Dell, developed the standards. NVMe enables high concurrency, parallelism, and scalability. It replaces the other SCSI protocols which do not work at the speed of non-volatile media memory. The PowerMax is an all-NVMe box so it benefits from this increase in speed, however, to take full advantage of NVMe you must extend the access over the storage network.

NVMe over fabrics, or NVMeoF, defines a common storage architecture for accessing the NVMe block storage protocol over a storage network. This means going from server to SAN, including a front-end interface to NVMe storage. There are any number of fabrics that make up NVMeoF including FC, RoCE, and IP. As FC is the predominant fabric for PowerMax customers, Dell introduced support for FC-NVMe with PowerMaxOS release 5978.444.444. Dell supports TCP with PowerMaxOS 10.

2.1 VAAI

The future of NVMeoF is the replacement of all SCSI commands, but for now, VMware's implementation of FC-NVMe and NVMe/TCP is not a complete conversion to the new command set. There is still some command-set translation required between the new protocol and traditional SCSI. Unlike traditional SCSI, the VAAI Plug-in cannot claim NVMeoF devices (see Figure 19 where the name is blank). Instead, ESXi translates the command directly, though the command number itself is different. For example, ATS Compare and Write in SCSI is the same in NVMeoF. UNMAP in SCSI becomes deallocate in NVMeoF and WRITE SAME or Block Zero becomes Write Zeroes. VMware will not attempt to send XCOPY commands as they have not ported it over. Instead ESXi will use host-based copy.

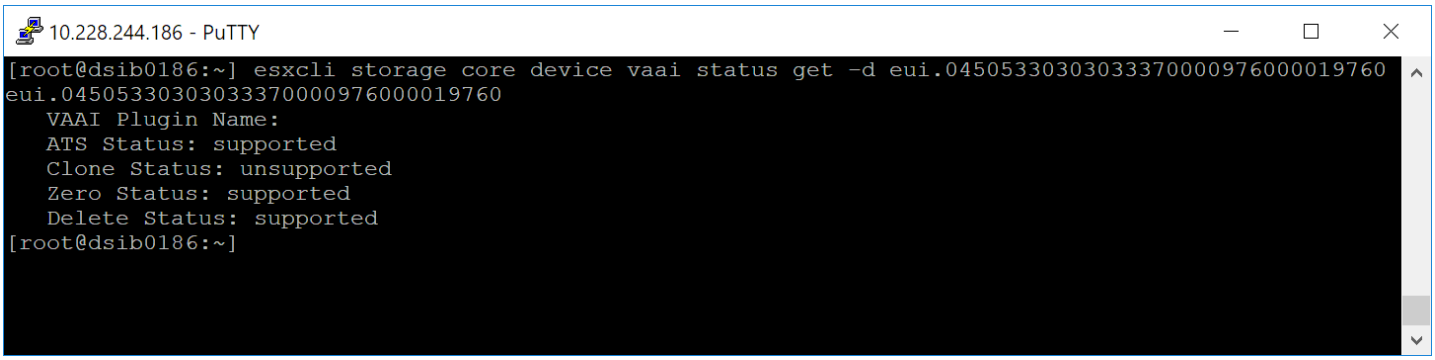


Figure 19. VAAI support with NVMeoF

3 eNAS and PowerMax File

Dell offers Embedded NAS (eNAS) guest OS (GOS) on the PowerMax 2000/8000 and Embedded File GOS on the PowerMax 2500/8500. NAS on the PowerMax enables customers to leverage vital Tier 1 features including SLO-based provisioning, Host I/O Limits, and data reduction technologies. eNAS is comprised of virtual instances of VNX Software Data Movers and Control Stations that run on the PowerMax, while File is a new implementation present across different arrays in the portfolio.

Unlike block storage, file storage does not support VAAI as part of the array code. All NFS file systems, regardless of vendor, require a NAS plug-in. Fortunately, the plug-in code is generic so that you can use the same plug-in for the Dell Unity platform and other arrays like PowerMax and PowerStore. Note that the plug-in works with either NFS version 3 or 4.1.

The VAAI features for NAS are not exactly equivalent to those available on block, though the idea is the same: to offload tasks to the array. Features on NAS include NFS Clone Offload, extended stats, space reservations, and snap of a snap. The NFS clone offload works much the same way as XCOPY as it offloads ESXi clone operations to the array.

To install the plug-in, download the NAS plug-in from Dell support. Dell delivers the plug-in as a VMware Installation Bundle (vib). You install the plug-in can through VMware vCenter Update Manager or CLI as demonstrated in Figure 20. Be sure to check for any existing NAS plug-in as you must remove it before installing the new one. Stop and start the vaai-nasd service if installing version 4.0.1 as below.

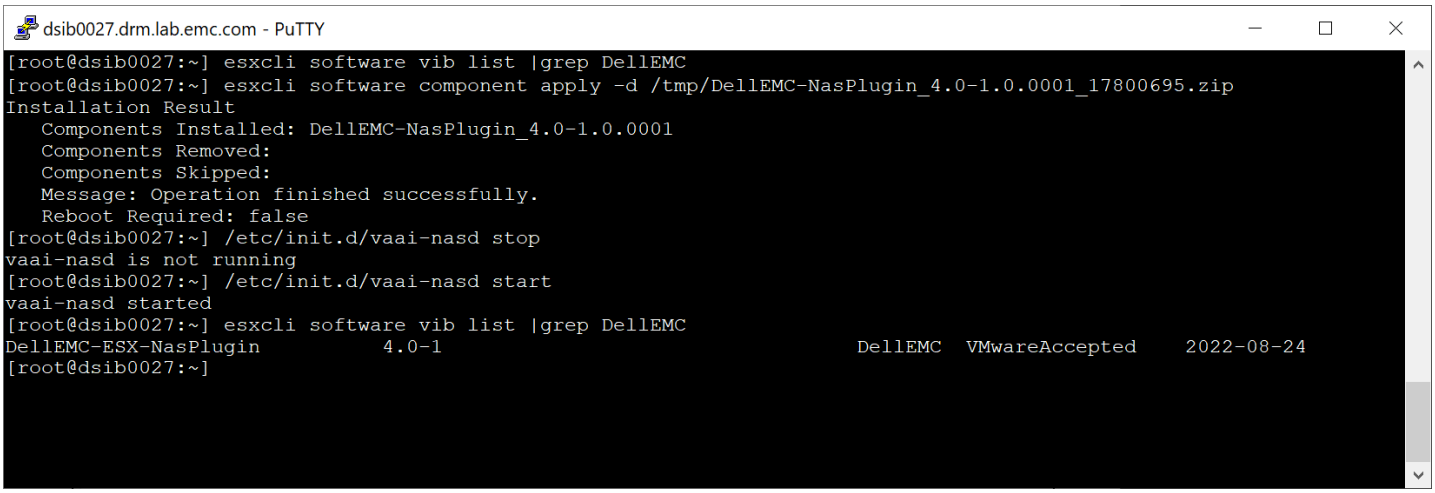


Figure 20. Installation of the NAS plug-in for VAAI

Once installed, Hardware Acceleration will show as “Supported” in the vSphere Client for NFS datastores as in Figure 21; otherwise the column shows as “Unknown.”

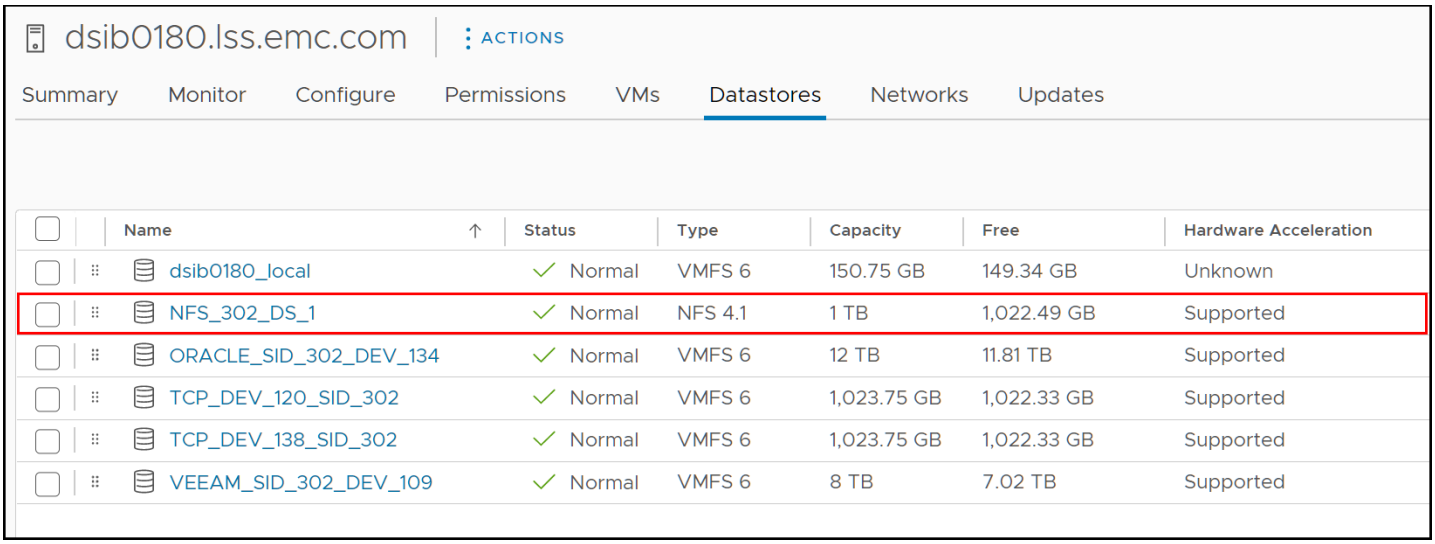


Figure 21. NFS datastore with VAAI support

3.1 Extended stats

As mentioned, one of the other VAAI integrations for NFS is extended stats. Using vmkfstools the user can display the disk utilization for virtual machine disks configured on NFS datastores. The ***extendedstat*** argument provides disk details for the virtual disks. The command reports, in bytes, virtual disk size, used space, and unshared space as in Figure 22.

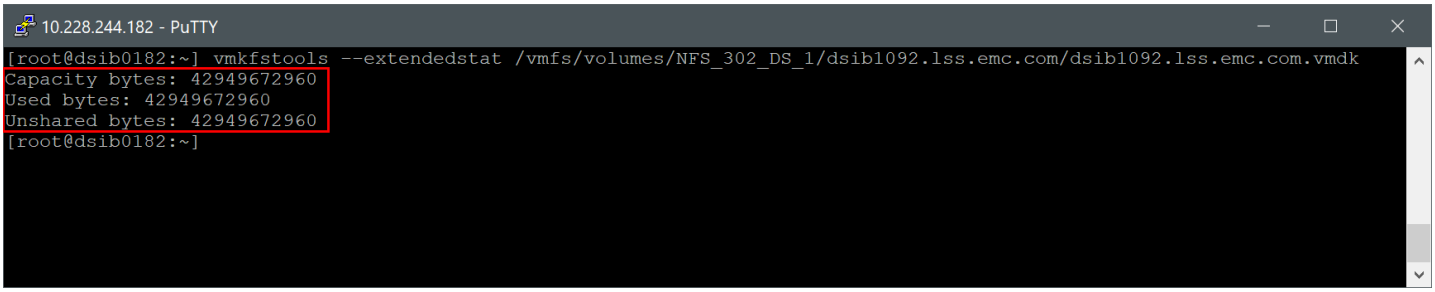


Figure 22. VAAI NFS extended stats feature

3.2 Nested clones

Another VAAI capability on NFS is the ability to create a thin-clone virtual machine from an existing thin-clone virtual machine. This functionality is nested clones. The functionality uses the snapshot architecture to instantaneously create lightweight virtual machine clones. ESXi offloads the clone operations to the array. To take advantage of this feature with eNAS, enable the nested clone support property when you create the file system as highlighted in Figure 23. You cannot change this setting after creating the file system. File enables the capability automatically upon file system creation.

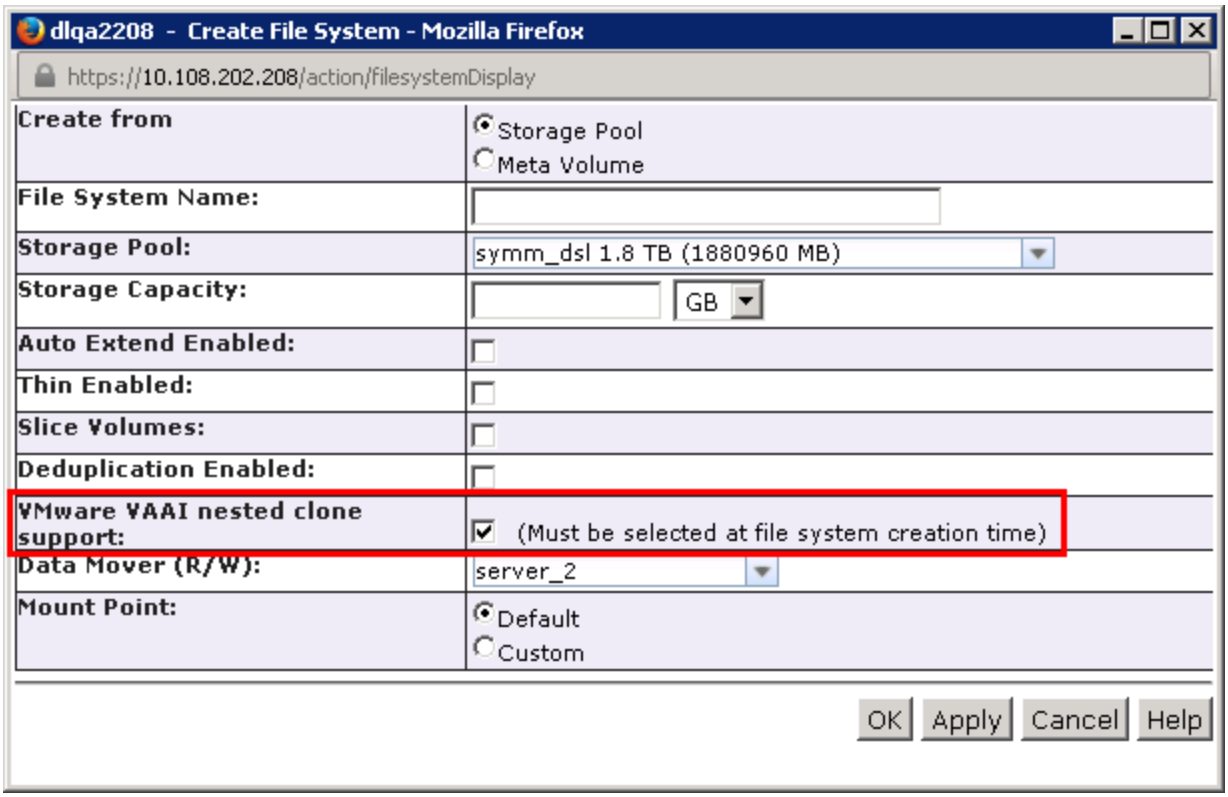


Figure 23. Setting nested clone support on a file system

4 Full Copy Detail

By default, when the array supports Full Copy, VMware informs the storage array to copy data in 4 MB chunks. For ESXi hosts that utilize more than PowerMax arrays, do not change the default value. However, in PowerMax only, or a combination of PowerMax and VNX environments, customers should take advantage of VMware's ability to send larger chunks to the array for copy. Some arrays offer claim rules to increase the copy size for Full Copy. The following two sections outline how to adjust the default copy size and how to add claim rules.

4.1 Default copy size

You can increase the default copy size to a maximum value of 16 MB. You should set this value in all PowerMax environments to take advantage of the array's ability to move larger chunks of storage at once.

The following commands show how to query the current copy (or transfer) size and then how to alter it.

```
esxcfg-advcfg -g /DataMover/MaxHWTransferSize

     Value of MaxHWTransferSize is 4096

esxcfg-advcfg -s 16384 /DataMover/MaxHWTransferSize

     Value of MaxHWTransferSize is 16384
```

Note that this change is per ESXi server and you can only execute it through the CLI (no UI interface) as in Figure 24:

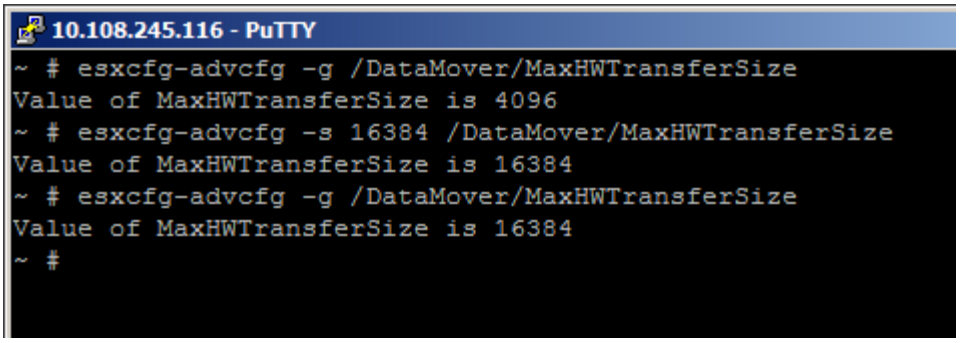


Figure 24. Changing the default copy size for Full Copy on the ESXi host

Alternatively, you can use vSphere PowerCLI at the vCenter level to change the parameter on all hosts as in Figure 25.

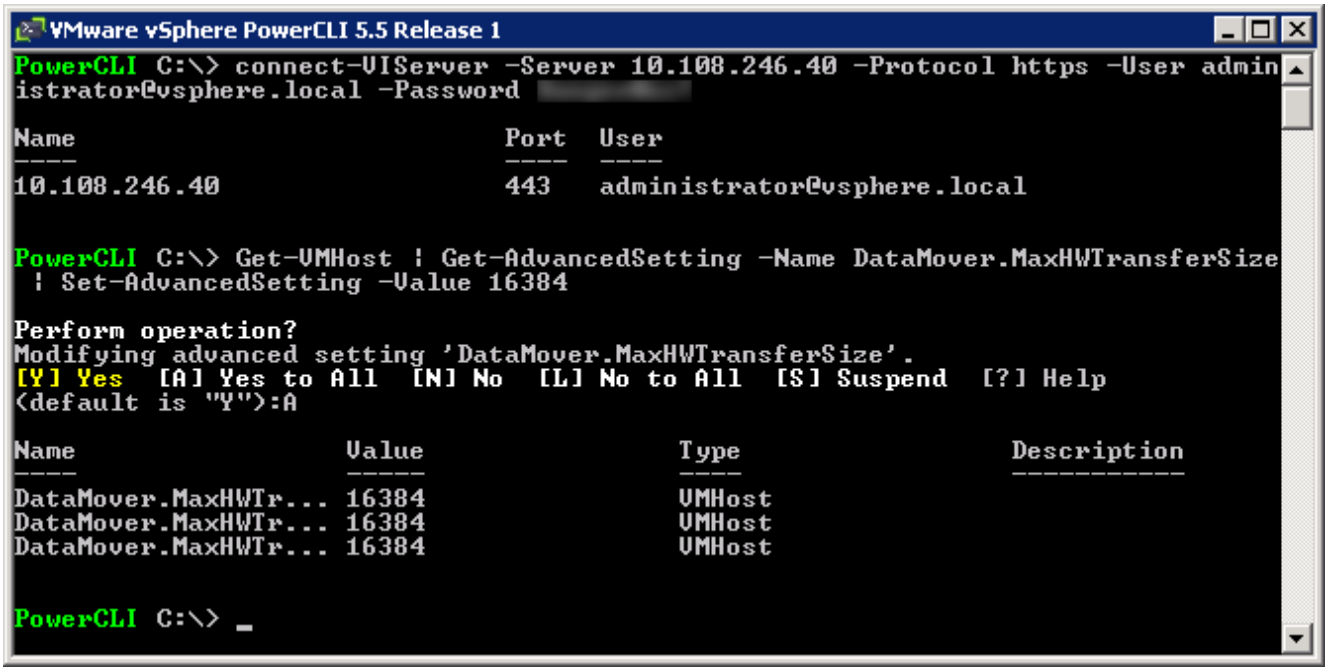


Figure 25. Changing the default copy size for Full Copy using vSphere PowerCLI

It is important to adjust the default transfer size in case claim rules are not in use or a condition occurs which renders them non-functional.

4.2 XCOPY claim rules

In addition to increasing the default transfer size, VMware offers claim rules which are a different methodology to increase extent size. These claim rules take precedence over the MaxHWTransferSize if both exist.

VMware uses claim rules, in general, to implement VAAI for PowerMax. VAAI is a plug-in claim rule that says if a device is from a particular vendor, in this case a Dell PowerMax device, it should go through the VAAI filter. In other words, VMware will use VAAI if it can. Each block storage device managed by a VAAI plug-in needs two claim rules, one that specifies the hardware acceleration filter and another that specifies the hardware acceleration plug-in for the device. These claim rules are where VMware made changes. VMware added three new columns to the storage core claim rules to control the new behavior. They are:

- XCOPY Use Array Reported Values
- XCOPY Use Multiple Segments
- XCOPY Max Transfer Size

Figure 26 shows the output with the new columns and their default values.

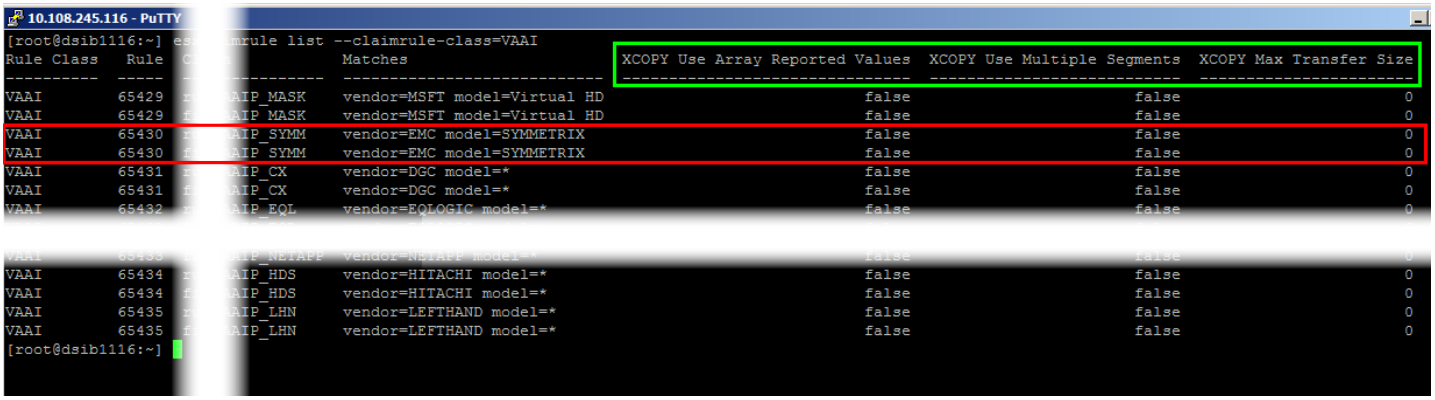


Figure 26. VAAI claim rule class

By altering the rules and changing these three new settings, you can adjust the max transfer size up to a maximum of 240 MB, rather than 16 MB. Though the columns exist in both the Filter and VAAI claim rule classes, you only need to change the VAAI class. The required settings are:

- XCOPY Use Array Reported Values = **true**
- XCOPY Use Multiple Segments = **true**
- XCOPY Max Transfer Size = **240**

It is not sufficient to simply change the transfer size because VMware requires that the **Use Array Reported Values** is set to true if the transfer size is set. When **Use Array Reported Values** is set it means that VMware will query the array to get the largest transfer size if the max transfer size is not set or if there is a problem with the value. Unfortunately, the PowerMax advertises a value higher than 240 MB so in the case of an issue with the transfer size VMware will default to the previous value set for MaxHWTransferSize; therefore, it is important to set that to 16 MB. In addition, for the PowerMax to accept up to 240 MB, you must enable multiple segments since Dell limits a single segment to 30 MB (there are eight segments available). Therefore, when one modifies the claim rules, set the size to 240 MB and set both other columns to true (even though you do not use the **Use Array Reported Values** column).

4.2.1 Custom vendor rule

The process to change the claim rules differs slightly between what the **custom** rule versus the **generic** rule. Both, however, involve removing the existing VAAI rule class, loading that change, then adding it back with the new columns set. Following this you must reboot the server.[6] The custom claim rule plugin on PowerMax is **VMW_VAAIP_SYMM**. Note that each storage vendor has their own custom plug-in. To change the new column values run the following commands. They will not return a response. Note that failure to load the claimrule before adding back the new one may result in a duplicate rule error.

Note: Refer to the VMware documentation to ensure you use the proper syntax for the claim rule commands.

```
esxcli storage core claimrule remove --rule 65430 --claimrule-class=VAAI
```

[6] There is a process by which you can accomplish this on a running server. It involves unmounting datastores and detaching devices; however, Dell does not recommend using this process as it is inconsistent.

```
esxcli storage core claimrule load --claimrule-class=VAAI

esxcli storage core claimrule add -r 65430 -t vendor -V EMC -M SYMMETRIX -P
VMW_VAAIP_SYMM -c VAAI -a -s -m 240
```

To verify the changes are set before reboot, list the claim rules as in Figure 27 below:

```
esxcli storage core claimrule list --claimrule-class=VAAI
```

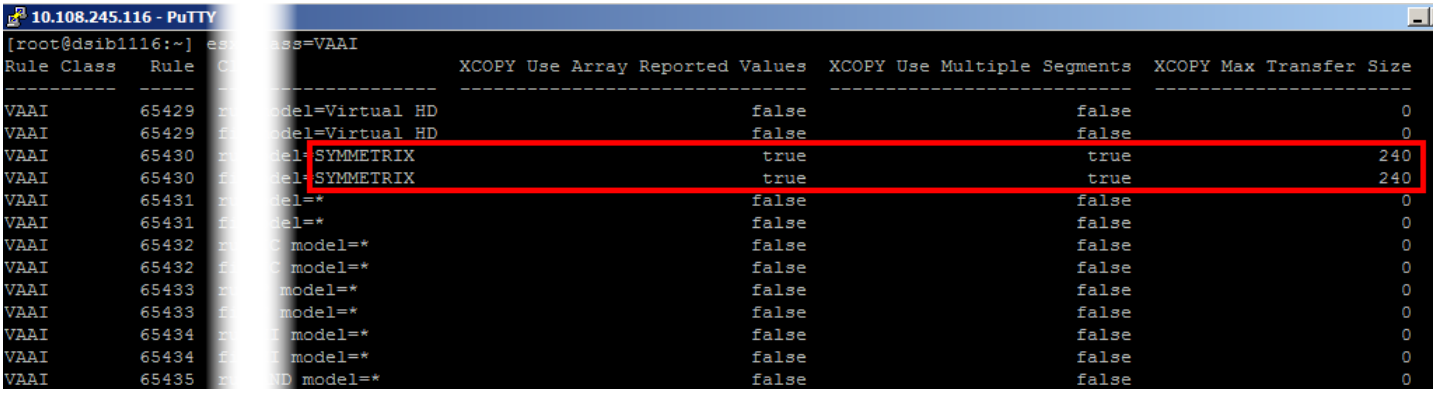


Figure 27. Updated VAAI claim rule for XCOPY

The changes will not be permanent until a reboot of the server. Once the reboot is complete, re-run the command to ensure the changes appear. Dell always recommends using the maximum value of 240 MB. Using the maximum ensures that, if possible (the extent must be contiguous), VMware will issue a 240 MB extent.

The new claim rule does not impact how VAAI functionality works. For instance, the following still hold true:

- It will revert to software copy when it cannot use XCOPY.
- You can still enable or disable it through the UI or CLI.
- VMware will use the **DataMover/MaxHWTransferSize** setting if the claim rule does not exist.
- The maximum value is just that, a maximum. That is the largest extent VMware can send but does not guarantee all extents will be that size. It is common for extent copy sizes to vary from under 1 MB to 240 MB and everything in between.

To return to the former functionality, simply remove the rule, and add back the claimrule without turning on any settings:

```
esxcli storage core claimrule remove --rule 65430 --claimrule-class=VAAI

esxcli storage core claimrule load --claimrule-class=VAAI

esxcli storage core claimrule add -r 65430 -t vendor -V EMC -M SYMMETRIX -P
VMW_VAAIP_SYMM -c VAAI
```

Reboot again.

4.2.1.1 Generic rule

VMware began the process of moving away from the custom vendor VAAI plug-ins (VMW_VAAIP_SYMM) enumerated in the previous section. To that end, VMware offers a generic

plug-in for VAAI named VMW_VAAIP_T10. This generic plug-in works the same way the custom plug-in does. The idea is to eventually deprecate all vendor custom plug-ins. Although you can use the custom plug-in, for new servers which do not already have the claim rules setup for PowerMax, Dell recommends using the new generic plug-in. The steps follow and are similar to the custom plug-in.

4.2.1.2 VMW_VAAIP_T10

First, you must remove the existing custom vendor rule. Because VMware suggests using claim rule number 914, remove both the Filter and VAAI classes. This is different than the custom plug-in process as you did not need to remove the Filter class. Note that if you attempt to add claim rule 914 before removing 65430, VMware will error and indicate 914 is a duplicate of 65430.

Note: The number 914 for the claim rule is arbitrary. If it is currently in use, use a different number, for example 950.

```
esxcli storage core claimrule remove --rule 65430 --claimrule-class=Filter
esxcli storage core claimrule remove --rule 65430 --claimrule-class=VAAI
```

Reload the filter and rule before adding the new ones to avoid duplication.

```
esxcli storage core claimrule load --claimrule-class=Filter
esxcli storage core claimrule load --claimrule-class=VAAI
```

Now add the new filter and rule.

```
esxcli storage core claimrule add -r 914 -t vendor -V EMC -M SYMMETRIX -P
VAAI_FILTER -c Filter
esxcli storage core claimrule add -r 914 -t vendor -V EMC -M SYMMETRIX -P
VMW_VAAIP_T10 -c VAAI -a -s -m 240
```

Reboot the ESXi host for the changes to take effect. Once the host comes back up, verify the changes. Be aware that because rule 65430 is still part of the ESXi build, VMware will add it back after the reboot; however, the rule shows as a “file” and not also “runtime” as is the new generic claim rule. Therefore, it cannot claim any devices. You can see the new rule in the red box, while the old rule is in blue in Figure 28.

```
esxcli storage core claimrule list --claimrule-class=VAAI
```

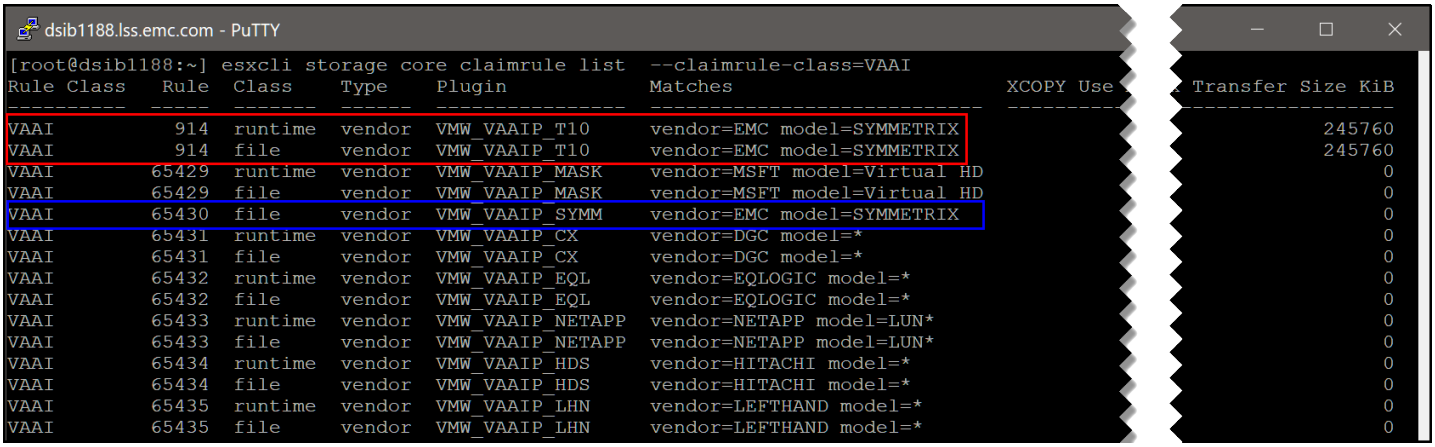


Figure 28. Generic claim rule listing

Note: For Storage vMotion operations, VMware will use a maximum of 64 MB for the extent size. This is a VMware restriction and has nothing to do with the array. Remember if VMware can use an extent size up to 240 MB it will, if not it will use something smaller. In the case of Storage vMotion it will not exceed 64 MB per VMware design.

4.2.1.3 Multiple storage arrays and claim rules

Because claim rules for VAAI are unique based on a combination of the vendor and model of array, it is possible to have multiple rules on a single ESXi host which do not conflict with one another. VMware applies each claim rule only to devices from that claim rule's associated vendor and model of array. Furthermore, a claim rule must be unique to vendor and model. As already explained, if a rule already exists for a storage array, attempting to add another will result in an error, forcing the user to first remove the existing rule. Rule numbers, therefore, are arbitrary and do not represent any hierarchy. While VMware uses the rule number '914' in their documentation, you can use any available number. In fact, by using the switch '-u,' instead of specifying a rule number, VMware will generate one for the user.

For example, if you have both PowerMax and PowerStore arrays in use by the same ESXi host, you might create the following rules. Note how PowerStore uses the 'u' switch so that VMware auto assigns the rule id.

```
esxcli storage core claimrule add -r 914 -t vendor -V EMC -M SYMMETRIX -P
VAAI_FILTER -c Filter
esxcli storage core claimrule add -r 914 -t vendor -V EMC -M SYMMETRIX -P
VMW_VAAIP_T10 -c VAAI -a -s -m 240

esxcli storage core claimrule add -u -t vendor -V DellEMC -M PowerStore -P
VAAI_FILTER -c Filter
esxcli storage core claimrule add -u -t vendor -V DellEMC -M PowerStore -P
VMW_VAAIP_T10 -c VAAI -a -s -m 240
```

Not all storage arrays support XCOPY claim rules so review vendor documentation prior to creation.

4.2.2 Virtual Storage Integrator

As an alternative to using CLI, the Virtual Storage Integrator (VSI) plug-in to vCenter can automatically create the claim rule for an ESXi host. If installed, navigate to **Cluster (right-click) -> Dell VSI -> Apply Host Best Practices…** in Figure 29.

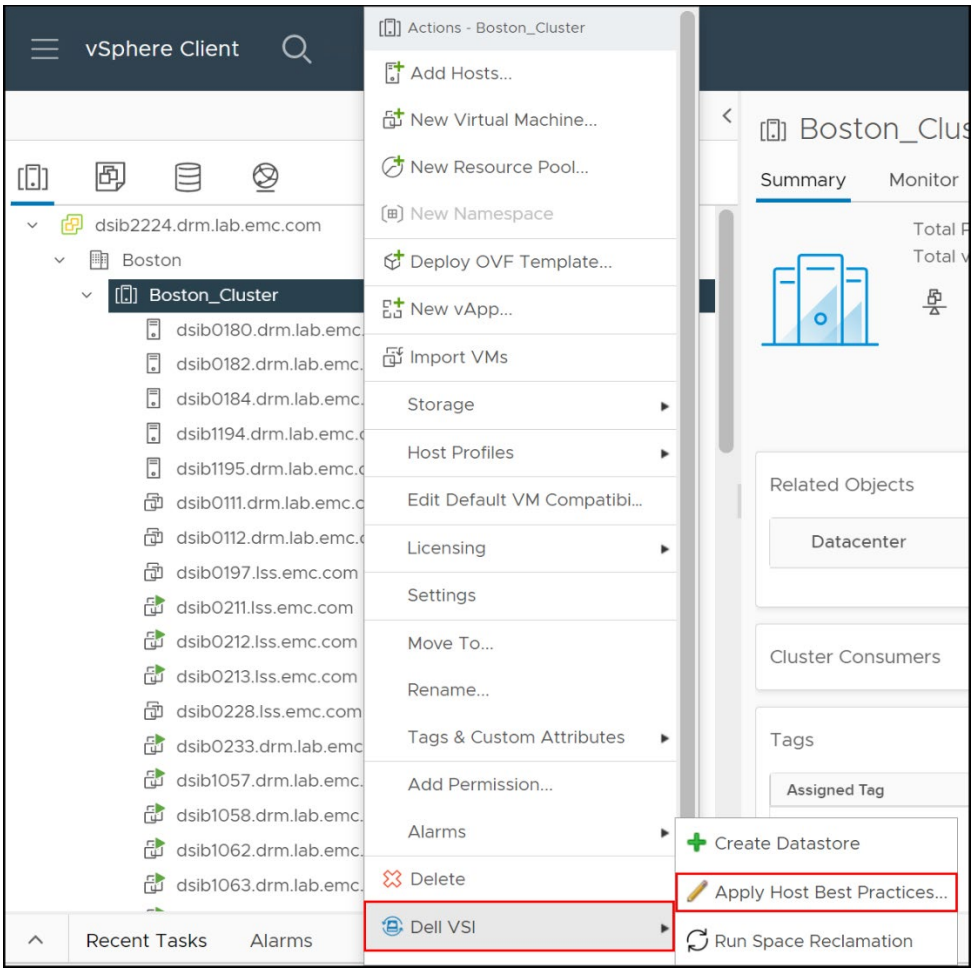


Figure 29. VSI Host Best Practices

In the next dialog in Figure 30, select **PowerMax** as the storage platform from the drop-down box. Check the box next to the **Host Setting** for the **Create VAAI rule…** Click the **SAVE** button to complete. Reboot the hosts, when possible, for the change to take effect.

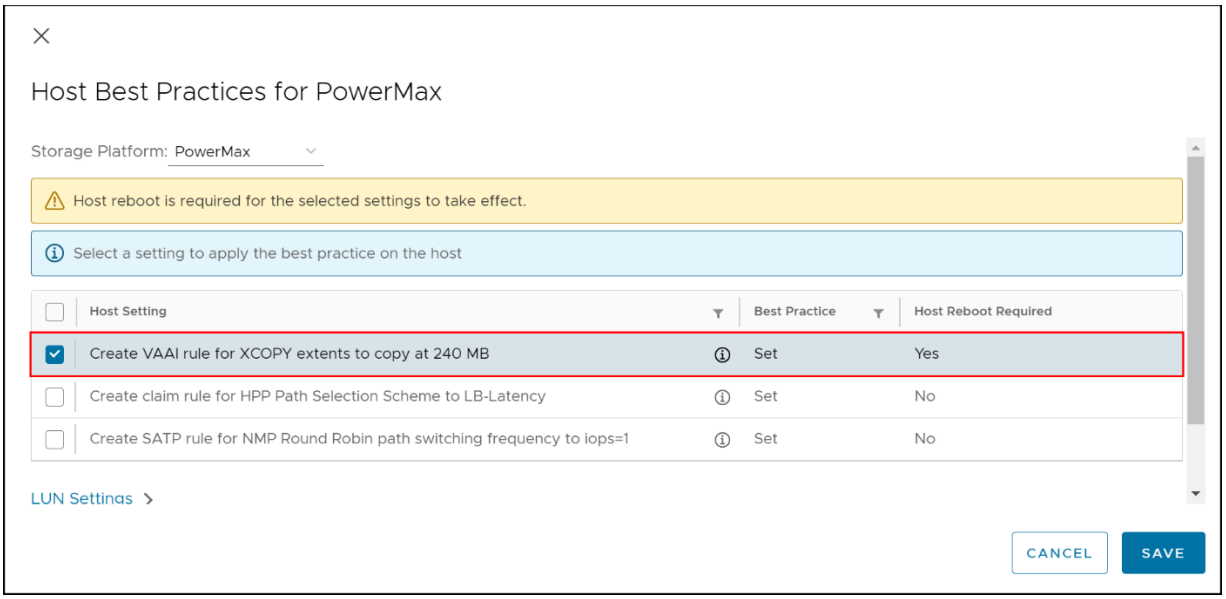


Figure 30. Creating XCOPY rule

Using VSI is the preferred method for adding the claim rule as it removes the complexity of CLI and is cluster wide. Note, however, that VSI hard-codes the rule number. If there are multiple arrays in use or ESXi is using the default rule of 914, do not use VSI to create the rule.

4.2.3 Claim rule results

The following graphs, Figure 31 and Figure 32, demonstrate the effectiveness of using claim rules for XCOPY when cloning 100 GB of data. Note that source and target refer to devices. XCOPY only works within an array, not between them.

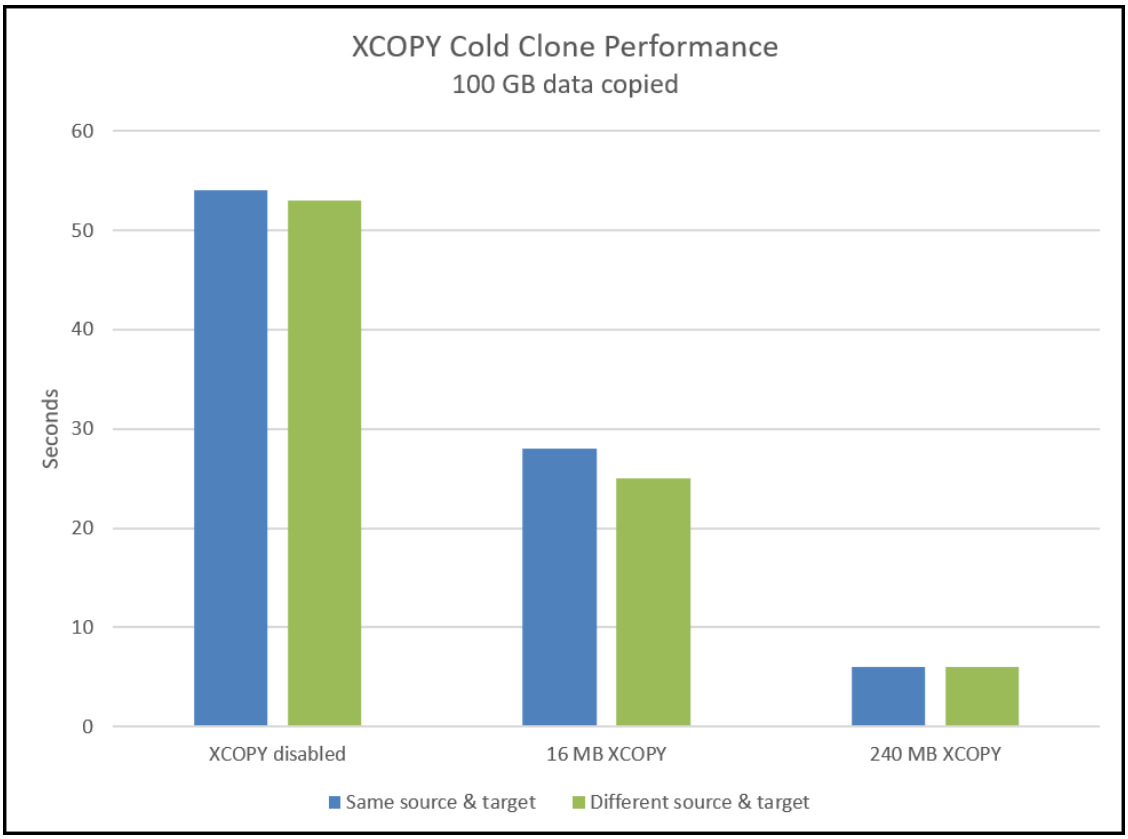


Figure 31. XCOPY cold clone performance on the PowerMax

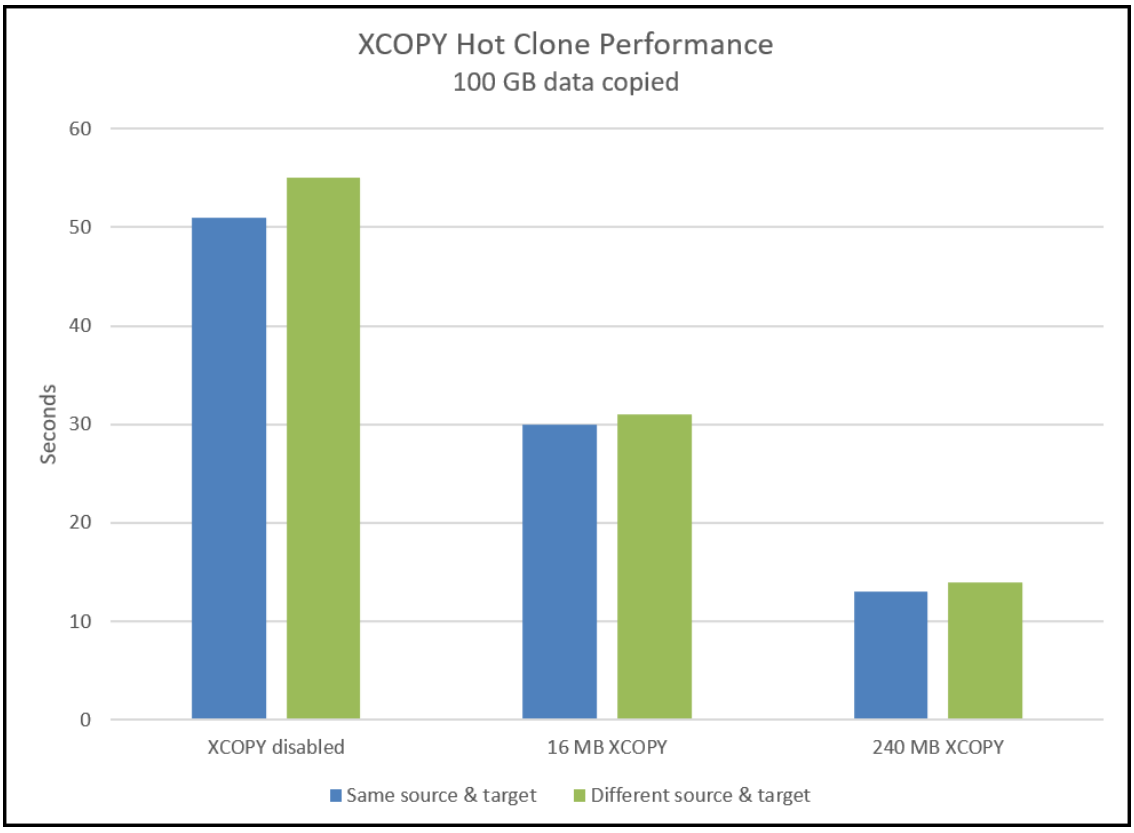


Figure 32. XCOPY hot clone performance on the PowerMax

Similarly, in Figure 33, you can see the benefits of XCOPY on a Storage vMotion. Here, VMware moves 250 GB of data between datastores. The benefit of the larger 64 MB extent size is apparent in the thin vmdk results.

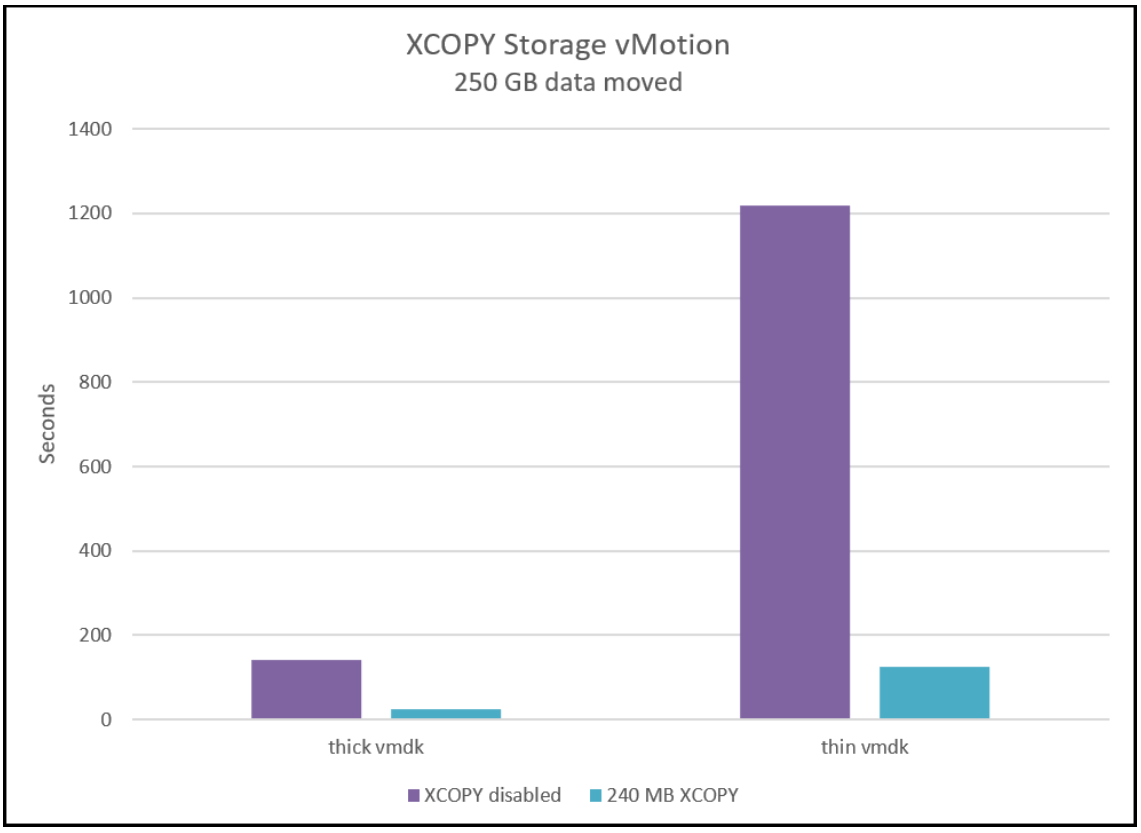


Figure 33. XCOPY SvMotion performance using claim rules with PowerMax

4.3 XCOPY and vmkfstools

On PowerMax arrays, Dell blocks the use of XCOPY for any datastore extents that contain a VMFS signature to prevent the occurrence of a VMware bug. The block is within the code, so the user is unaware this occurs, and it has no impact on performance since the number of extents is tiny. A side effect of this implementation, however, is that if you try to clone a vmdk using vmkfstools, the clone fails with “Connection timed out” as below.

```
[root@dsib1010:/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test] vmkfstools -i mint_test_1.vmdk -d
rdmp:/vmfs/devices/disks/naa.60000970000120001473533030313545 /vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce6
2/mint_test/test_rdm.vmdk -v 1
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/mint_test_1-flat.vmdk" : open suc
cessful (269) size = 4294967296, hd = 0. Type 3
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/mint_test_1-flat.vmdk" : closed.
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/mint_test_1-flat.vmdk" : open suc
cessful (14) size = 4294967296, hd = 232117994. Type 3
Destination disk format: pass-through raw disk mapping to '/vmfs/devices/disks/naa.600009700001200014735330303
13545'
Cloning disk 'mint_test_1.vmdk'...
DISKLIB-LIB_CLONE   : DiskLibCreateNativeClone: Incompatible object type 'file' specified.
DISKLIB-LIB_CLONE   : DiskLibCloneGrowInt: Failed to clone disk using Object Cloning.
DISKLIB-LIB_CREATE   : DiskLibCreateVMFS: No allocation type given; assuming zeroedthick.
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/test_rdm-rdmp.vmdk" : open succes
sful (33554433) size = 4295884800, hd = 0. Type 11
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/test_rdm-rdmp.vmdk" : closed.
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/test_rdm-rdmp.vmdk" : open succes
sful (33288) size = 4295884800, hd = 201512683. Type 11
DDB: "longContentID" = "e22d42e27666bff9357cf33843310729" (was "5b4d3bd5ce55abd3e982d6fdffffffe")
DISKLIB-LINK  : DiskLinkNativeVmfsClone: Incompatible disk types.
OBJLIB-FILEBE : FileBEIoctl: ioctl operation IOCTLCMD_VMFS_MOVEDATA(3030) failed on '/vmfs/volumes/67eed37b-7f
d81a92-25d3-e4434be8ce62/mint_test/mint_test_1-flat.vmdk' : Connection timed out (7208962)
DISKLIB-VMFS :Vmfs_MoveData : failed to move data (Connection timed out:0x6e0009).
DISKLIB-LIB_CLONE   : DiskLibCloneWholeDisk: Failed to clone whole disk: Connection timed out
DISKLIB-LIB_CLONE   : DiskLibCloneGrowInt: Failed to clone whole disk: Connection timed out
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/test_rdm-rdmp.vmdk" : closed.
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/test_rdm-rdmp.vmdk" : open succes
sful (1115137) size = 0, hd = 0. Type 11
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/test_rdm-rdmp.vmdk" : closed.
DISKLIB-LIB   : DiskLibUnlinkInt: Disk delete successfully completed { result:0, Msg: 'The operation completed
 successfully', fileName:'/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/test_rdm.vmdk'}
Failed to clone disk: Connection timed out (7208969).
[root@dsib1010:/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test]
```

Figure 34. vmkfstools clone failure when XCOPY enabled

To prevent the failure, disable XCOPY. VMware will use software copy which succeeds.

```
[root@dsib1010:/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test]  esxcfg-advcfg -s 0 /DataMover/Har
dwareAcceleratedMove
Value of HardwareAcceleratedMove is 0
[root@dsib1010:/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test]  esxcfg-advcfg -g /DataMover/Hardw
areAcceleratedMove
Value of HardwareAcceleratedMove is 0
[root@dsib1010:/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test] vmkfstools -i mint_test_1.vmdk -d
rdmp:/vmfs/devices/disks/naa.600009700001200014735330303135545 /vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce6
2/mint_test/test_rdm.vmdk -v 1
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/mint_test_1-flat.vmdk" : open suc
cessful (269) size = 4294967296, hd = 0. Type 3
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/mint_test_1-flat.vmdk" : closed.
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/mint_test_1-flat.vmdk" : open suc
cessful (14) size = 4294967296, hd = 182573155. Type 3
Destination disk format: pass-through raw disk mapping to '/vmfs/devices/disks/naa.600009700001200014735330303
13545'
Cloning disk 'mint_test_1.vmdk'...
DISKLIB-LIB_CLONE   : DiskLibCreateNativeClone: Incompatible object type 'file' specified.
DISKLIB-LIB_CLONE   : DiskLibCloneGrowInt: Failed to clone disk using Object Cloning.
DISKLIB-LIB_CREATE   : DiskLibCreateVMFS: No allocation type given; assuming zeroedthick.
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/test_rdm-rdmp.vmdk" : open succes
sful (33554433) size = 4295884800, hd = 0. Type 11
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/test_rdm-rdmp.vmdk" : closed.
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/test_rdm-rdmp.vmdk" : open succes
sful (33288) size = 4295884800, hd = 205379684. Type 11
DDB: "longContentID" = "e22d42e27666bff9357cf33843310729" (was "78de15de957c534f2927208affffffe")
DISKLIB-LINK  : DiskLinkNativeVmfsClone: Incompatible disk types.
Clone: 100% done.DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/test_rdm-rdmp.vm
dk" : closed.
DISKLIB-VMFS  : "/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test/mint_test_1-flat.vmdk" : closed.

AIOMGR-S : stat o=6 r=12 w=0 i=134 br=196608 bw=0
OBJLIB-LIB: ObjLib cleanup done.
WORKER: asyncOps=0 maxActiveOps=0 maxPending=0 maxCompleted=0
[root@dsib1010:/vmfs/volumes/67eed37b-7fd81a92-25d3-e4434be8ce62/mint_test]
```

Figure 35. vmkfstools clone failure when XCOPY enabled

In PowerMaxOS 10.4.0.0, Dell removed the block so you can use XCOPY. There are E Packs for two previous releases which you can find in the Table 1. HYPERMAX OS/PowerMaxOS and VAAI support.

4.4 SRDF and XCOPY

As mentioned previously, Dell implements XCOPY asynchronously on the PowerMax. Unfortunately, this implementation can cause a potential issue for customers when they use XCOPY in conjunction with SRDF/Metro or SRDF/Synchronous target devices. Therefore, Dell uses synchronous XCOPY for those target SRDF devices. The synchronous process ensures that a consistency group will not enter a “sync in progress” state. The two main reasons Dell implemented the synchronous copy instead of defaulting to software copy are:

1. Avoid any consistency issues with SRDF devices.
2. Offload the copy process from the host to the array, thus saving CPU, memory, and bandwidth.

The performance of the synchronous implementation is, by its nature, more akin to VMware’s software copy than Dell’s asynchronous implementation. As such, while Dell synchronous copy will outpace VMware’s host copy (though not always), the improvement will not mirror those for asynchronous copy. It is, however, more desirable to offload the copy to the array to free the host and network resources for more important activities. If the time to complete an XCOPY task is more critical than the resources saved on the host, you can disable XCOPY for SRDF devices on the array by Dell. For more detail on XCOPY performance considerations, please see Dell KB 533648. An example of performance is in Figure 36.

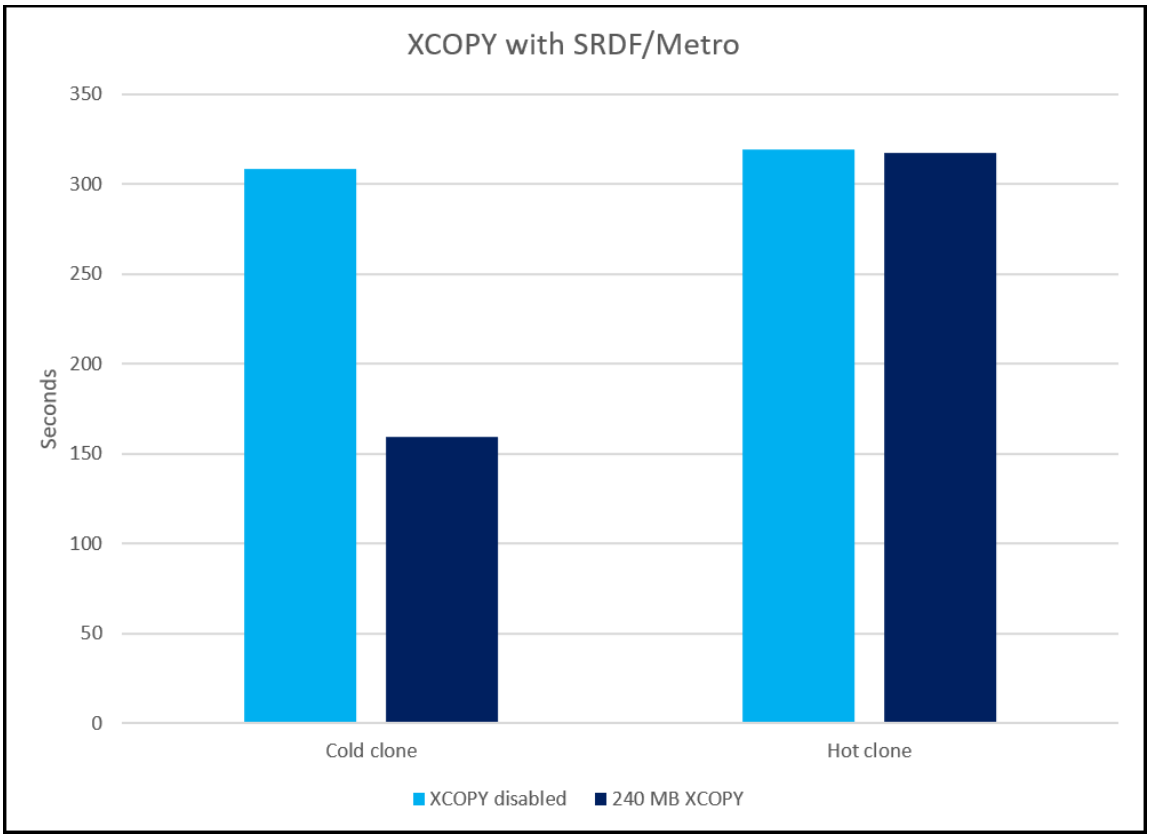


Figure 36. XCOPY with SRDF/Metro

4.5 XCOPY with multiple arrays on an ESXi host

One final scenario where you need to modify XCOPY best practices for PowerMax is if you have an ESXi host that uses both a PowerMax and another Dell array, e.g., Unity. Fortunately, as explained above, claim rules are array-specific, and as such their existence on an ESXi host has no bearing on devices presented from other arrays; however, on PowerMax it is always the best practice to also set the default hardware acceleration to its maximum of 16 MB in case there is an issue with the claim rules. When other arrays are present, though, do not modify this parameter from the default of 4 MB as other Dell arrays have different best practices. For more detail, including other vSphere settings in multi-array scenarios, please refer to Dell KB 000020863.

4.6 Caveats for using hardware-accelerated Full Copy[7]

The following are general caveats that Dell provides when using this feature:

- VMware publishes a KB article (1021976) which details when Full Copy will not be issued. Such cases include VMs that have snapshots or cloning or moving VMs between datastores with different block sizes, among others.

[7] You can disable Full Copy (XCOPY) at the system level if the caveats are particularly problematic for a customer.

  - One case not included is when the configuration parameter disk.EnableUUID is set to TRUE on Windows VMs. VMware Tools will set this parameter so be aware this will prevent XCOPY.

- Always set MaxTransferSize to 16 MB and use claim rules.
- Limit the number of simultaneous clones using Full Copy to three or four. This is not a strict limitation, but Dell believes this will ensure the best performance when offloading the copy process.
- A PowerMax device that has SAN Copy, TimeFinder and certain RecoverPoint sessions will not support Full Copy. VMware will divert to software copy any cloning or Storage vMotion operation run on datastores backed by these. Note that the vSphere Client has no knowledge of these sessions and as such the “Hardware Accelerated” column in the vSphere Client will still indicate “Supported” for these devices or datastores.
- If an XCOPY session exists on a device, any TimeFinder operation will fail with the following error: “*Cannot use the device for this function because it is a Copy Session target.”* Once the XCOPY operation completes, the TimeFinder action will succeed.
- Open Replicator devices do not work with XCOPY. VMware will revert to software copy in these cases.
- Federated Live Migration (FLM) target devices cannot use Full Copy.
- Dell will block certain RDF operations, such as an RDF failover, until the PowerMax has completed copying the data from a clone or Storage vMotion.
- When the available capacity of a Storage Resource Pool (SRP) reaches the Reserved Capacity value, active or new Full Copy requests can take a long time to complete and may cause VMware tasks to timeout. For workarounds and resolutions please see Dell KB article 503348.
- Dell cannot use Full Copy between devices presented to a host using different initiators. For example, take the following scenario:
  - A host has four initiators.
  - The user creates two initiator groups, each with two initiators.
  - The user creates two storage groups and places the source device in one, the target device in the other.
  - The user creates a single port group.
  - The user creates two masking views, one with the source device storage group and one of the initiator groups, and the other masking view with the other initiator group and the storage group with the target device.
  - The user attempts a Storage vMotion of a VM on the source device to the target device. Dell rejects Full Copy, and VMware reverts to software copy.

4.6.1 SRDF/Metro specific

The following are caveats for SRDF/Metro when using Full Copy. In SRDF/Metro configurations the use of Full Copy does depend on whether the site is the one supplying the external WWN identity.

- Dell does not support Full Copy between a non-SRDF/Metro device and an SRDF/Metro device when the device WWN is different from the external device WWN. Typically, but not always, this means the non-biased site (recall even when using witness, there is a bias site). In such cases Dell supports Full Copy only when operations are between SRDF/Metro devices or within a single SRDF/Metro device; otherwise, VMware uses software copy.

- As Full Copy is a synchronous process on SRDF/Metro and SRDF/S devices, which ensures consistency, performance will be closer to software copy than asynchronous copy.
- While an SRDF/Metro pair is in a suspended state, Full Copy reverts to asynchronous copy for the target R1. It is important to remember, however, that Full Copy is still bound by the restriction that the copy must take place on the same array. For example, assume SRDF/Metro pair AA, BB is on array 001, 002. In this configuration device AA is the R1 on 001 and its WWN is the external identity for device BB, the R2 on 002. If you suspend the pair and the array switches the bias such that BB becomes the R1, the external identity remains that of AA from array 001 (i.e., it appears as a device from 001). The device is on array 002 and therefore Dell can only use Full Copy within the device itself or between devices on array 002.

4.6.2 Compression/Deduplication (Data Reduction) specific

The following are caveats when using Full Copy with devices in storage groups with compression or compression/deduplication enabled. Note that deduplication is only available on the PowerMax platform and only in conjunction with compression. It is not possible to enable one without the other. As such, references to compression below also include the PowerMax.

- When Dell uses Full Copy on a device in a storage group with compression enabled, or between storage groups with compression enabled, Full Copy will uncompress the data during the copy and then re-compress on the target.
- When Dell uses Full Copy on a device in a storage group with compression enabled, to a storage group without compression enabled, Full Copy will uncompress the data during the copy and leave the data uncompressed on the target.
- When Dell uses Full Copy on a device in a storage group without compression enabled to a storage group with compression enabled, the data will remain uncompressed on the target and be subject to the normal compression algorithms.
- As Full Copy is a background process, Dell may delay deduplication of data until after it copies all tracks. Full clone copies will deduplicate at close to 100% in enabled storage groups.

5 Block Zero Detail

Hardware-accelerated Block Zero provides benefits and performance increases in a variety of use cases. This paper will cover the two most encountered scenarios. The first scenario discussed will be concerned with deploying fully allocated virtual machines (virtual machines using the “eagerzeroedthick” virtual disk allocation mechanism) on the PowerMax. The second use case discusses the performance gains achieved using Block Zero when performing I/O in a virtual machine with virtual disks using the zeroedthick allocation format on the PowerMax.

5.1.1 Eagerzeroedthick disks

In certain situations, when you deploy virtual disks using the “eagerzeroedthick” allocation mechanism to fully zero out the disk at creation. A VMware feature that requires EZT disks is VMware Fault Tolerance. Fault Tolerance requires that all virtual disks in use by protected virtual machines use this allocation mechanism. In other cases, you create virtual disks using the “thin” or “zeroedthick” allocations mechanisms where space is not fully allocated on creation of the virtual disk. Both these virtual disks will not zero out the space until the guest OS writes data to a previously unallocated block. This behavior inherently will cause a performance drop when allocating new space as the kernel must first write zeros to initialize the new space before the virtual machine can write actual data. Therefore, you can use the “eagerzeroedthick” allocation mechanism to avoid performance degradations caused by the on-the-fly zeroing of “thin” or “zeroedthick” virtual disks.

Since VMware zeroes “eagerzeroedthick” virtual machines at the initial creation, it can take a great deal of time to write these zeros, especially in virtual machines with large virtual disks. Furthermore, this creation operation consumes SAN bandwidth during execution. With the introduction of Block Zero, VMware offloads the zeroing to the array, which saves not only on SAN bandwidth but also takes advantage of the increased processing power inherent on the PowerMax. In the case of the PowerMax, the array does not write the zeros.

Note: As the PowerMax does not actually reserve space, it seems logical to conclude that there is slight difference between using an eagerzeroedthick and a zeroedthick virtual disk on that platform. While this is true from a storage performance perspective, it is important to remember that if VMware or an application requires the use of an eagerzeroedthick virtual disk (e.g., Fault Tolerant VM or Oracle RAC), you must select that disk type when creating the virtual machine.

In this use case, we deployed new eagerzeroedthick virtual disks of three sizes 100 GB, 500 GB, and 1000 GB with hardware-accelerated Block Zero (WRITE SAME) disabled and enabled. As shown by the graph in Figure 37, the noted improvements are significant.

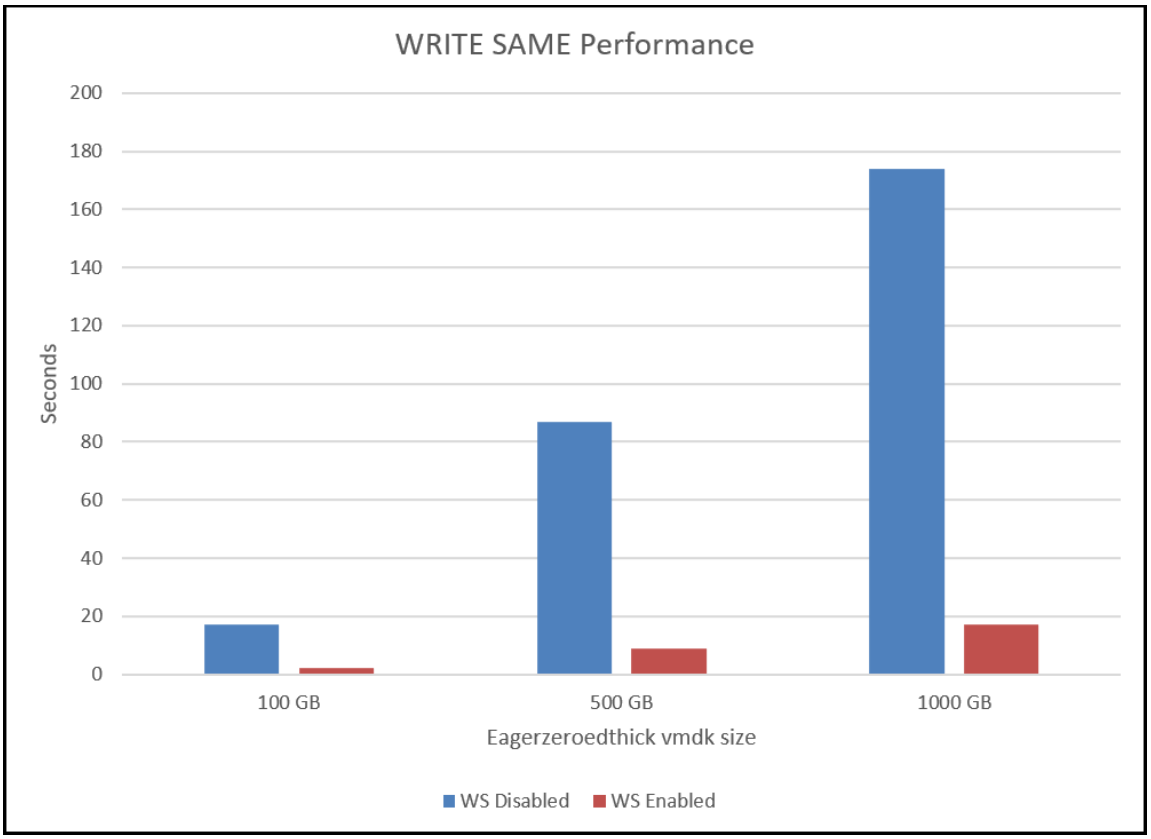


Figure 37. Fully allocated virtual disk deployment times using Block Zero

In addition to time savings, there is a substantial difference in throughput during the creation of the virtual disks when Block Zero is off and on. Figure 38 shows the ESXi performance graph of the write rate in KB per second (throughput) of the target PowerMax device from the ESXi server creating the virtual disk. You can see a dramatic difference between when Block Zero is off and on. Creating a virtual disk with hardware acceleration off takes about three minutes and has a write rate exceeding 6,000,000 KB/s. After the virtual disk creation completes, hardware-accelerated Block Zero is enabled and we create a second identical virtual disk on the same target PowerMax device. The throughput required to deploy this is so low that it is unseen on the same scale as the first virtual disk creation. The throughput spikes at around 4,000 KB/s for around one second — a 1500x decrease in required throughput to create the virtual disk.

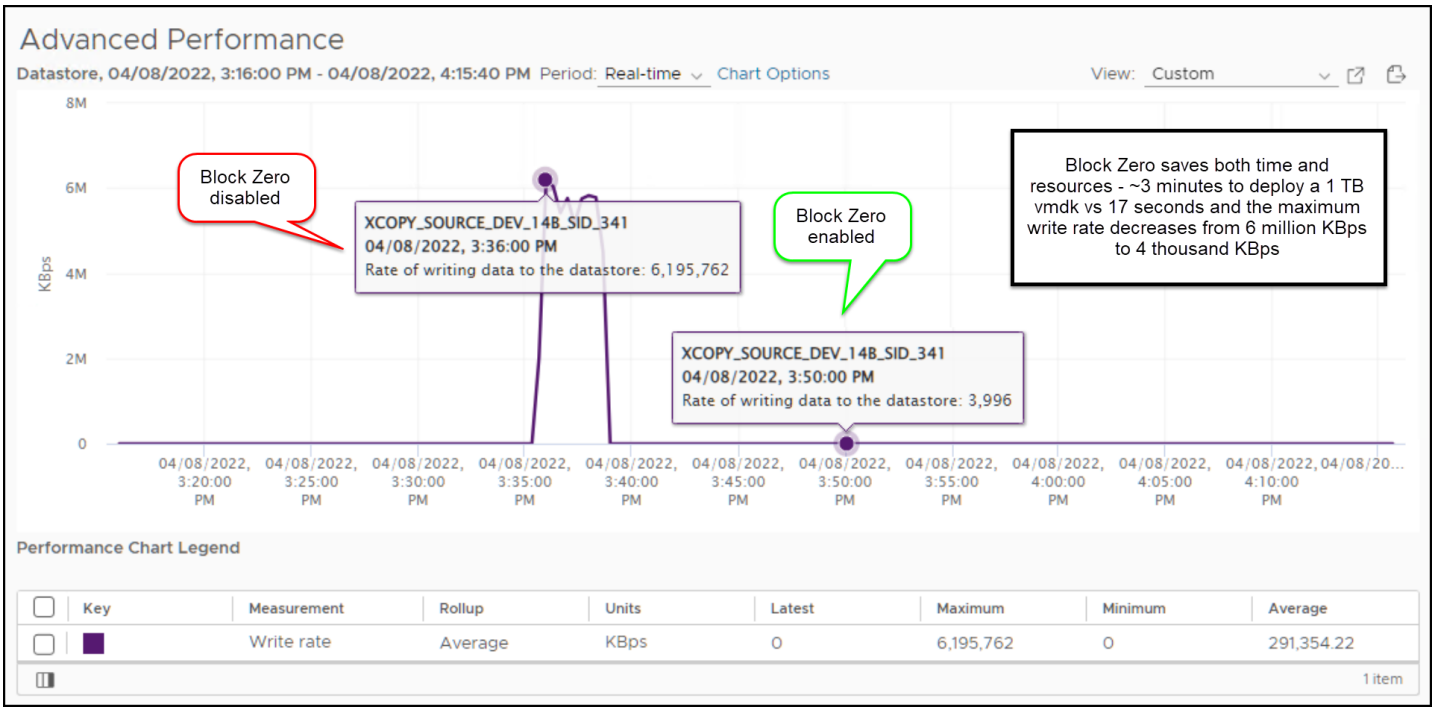


Figure 38. Throughput difference deploying an EZT virtual disk with Block Zero disabled and enabled

5.1.2 Zeroedthick virtual disks

By default, VMware creates new virtual disks using the “zeroedthick” allocation mechanism. This is also the recommended disk format for use with PowerMax to improve storage efficiencies by not writing zeros on creation. In this allocation scheme, VMware reserves the storage required for the virtual disks in the datastore, but the VMware kernel does not initialize all the blocks with zeros. The ESXi kernel, in response to I/O activities, initializes blocks with zeros to previously uninitialized blocks by the guest operating system. Consequently, as discussed in the previous use case, there is overhead when writing to unallocated regions of the virtual disk as the ESXi kernel first must write zeros to the previously unallocated space before the guest can complete any new write operations. Prior to the VAAI primitives, to avoid this overhead, customers used “eagerzeroedthick” virtual disks to ensure optimal performance. Unfortunately, especially in the case of Virtual Provisioning, “eagerzeroedthick” virtual disks wasted inordinate amounts of space in the storage pool due to the zeros written at creation. VMware forced the user to decide between consuming more capacity with “eagerzeroedthick” virtual disks and accepting the inherent performance overhead associated with “zeroedthick” virtual disks. With the advent of Block Zero, however, VMware and Dell removed that performance overhead.

5.2 Caveats for using hardware-accelerated Block Zero

The following are general caveats that Dell provides when using this feature:

- Block Zero does not work with Open Replicator. VMware will revert to traditional writes in these cases.
- Block Zero commands will take longer to execute on devices in SRDF relationships.

6 UNMAP Detail

The VMware use cases for UNMAP[8] revolve around the time it takes to reclaim storage space after the deletion of virtual disks, virtual machines, or the execution of a Storage vMotion. All of these tasks create empty space on the datastore but not on the array device. While Dell recommends the use of automated UNMAP for VMFS 6 datastores, there are use cases where customers wish to use manual UNMAP. Below is a discussion of both options, automated and manual.

6.1 Automated UNMAP

VMware offers automated UNMAP capability at the VMFS 6 datastore level. The automation is enabled by a background process (UNMAP crawler) that keeps track of which LBAs can be unmapped and then issues those UNMAPs at various intervals to maintain the reclaim rate. It is an asynchronous process. By default, the reclaim rate for UNMAP is 26 MB/s and is the recommended setting on PowerMax.

VMware uses a low priority scheduling queue for UNMAP. The queue functions as follows: If an array is not active, the set rate will be honored; however, on a heavily active array, the queue might result in a reduced rate. By throttling the rate, VMware can respond to those arrays under heavy load which cannot process the UNMAP commands quickly enough. Note that the queue is under the control of VMware and it is not adjustable.

Note: Any ESXi host can send UNMAP commands concurrently with any other ESXi host. If, for example, you delete two VMs on the same datastore, each residing on a different ESXi host, each of those hosts can send UNMAP commands for their respective VMs. VMware enforces the rate of reclaim, by default 26 MB/second, at the host level, not the datastore level. Thus, in this example the reclaim rate could run as high as 52 MB/s for that datastore. Therefore, increasing the reclaim rate above the default value can have a multiplying effect.

UNMAP is enabled by default when you create a VMFS 6 datastore through the vSphere Client wizard. There are two parameters available: Space Reclamation Granularity and Space Reclamation Priority. The first parameter, Space Reclamation Granularity, designates at what size VMware will reclaim storage. You cannot modify it from the 1 MB value. The second parameter, Space Reclamation Priority, dictates how aggressive the UNMAP background process will be when reclaiming storage. This parameter only has two values: None (off), and Low. It is set to Low by default. You can view the parameters in Figure 39.

[8] The UNMAP process is also known as Dead Space Reclamation and for the purposes of this paper the terms reclaim or unmapping are synonymous with the UNMAP process.

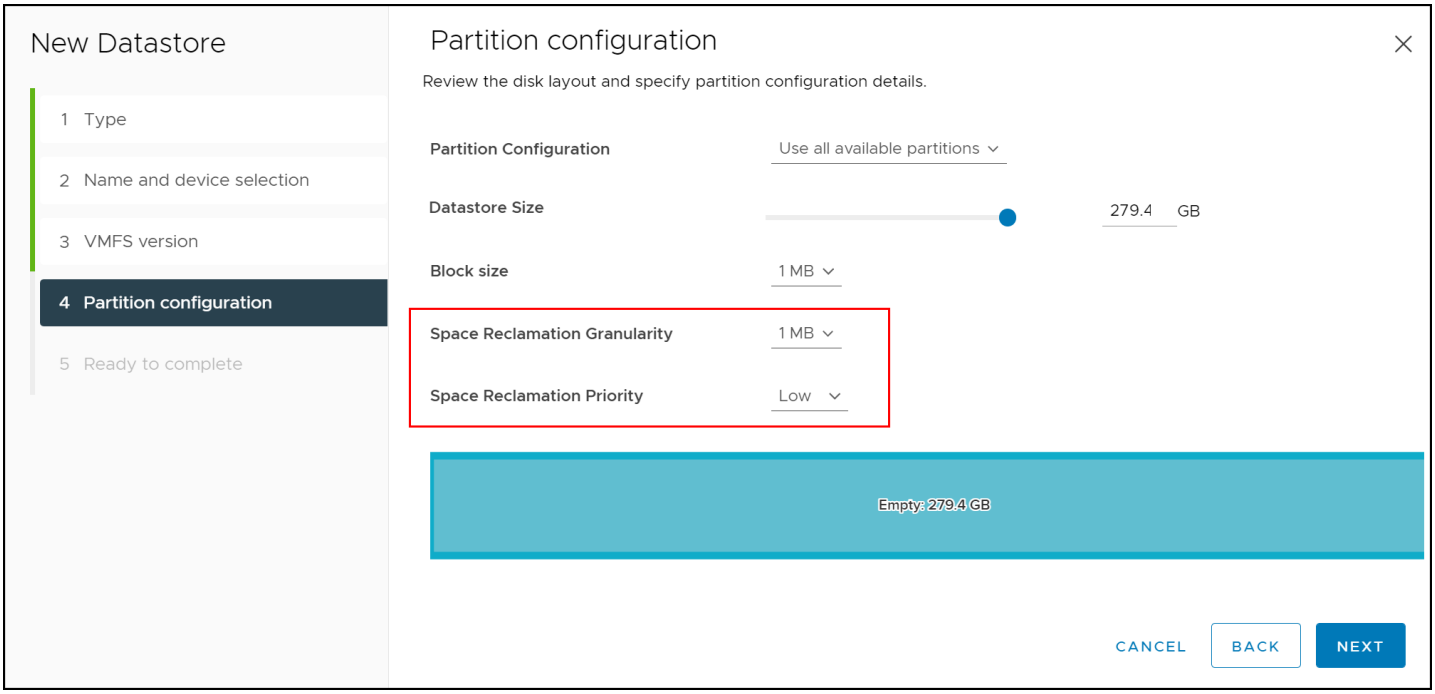


Figure 39. Enabling automated UNMAP on a VMFS 6 datastore

If it becomes necessary to either enable or disable UNMAP on a VMFS 6 datastore after creation, you can modify the setting through the vSphere Client or CLI. Within the datastore view in the Client, navigate to the Configure tab and the General sub-tab. On this page, find a section on Space Reclamation where you can use the Edit button to make the desired change as in Figure 40.

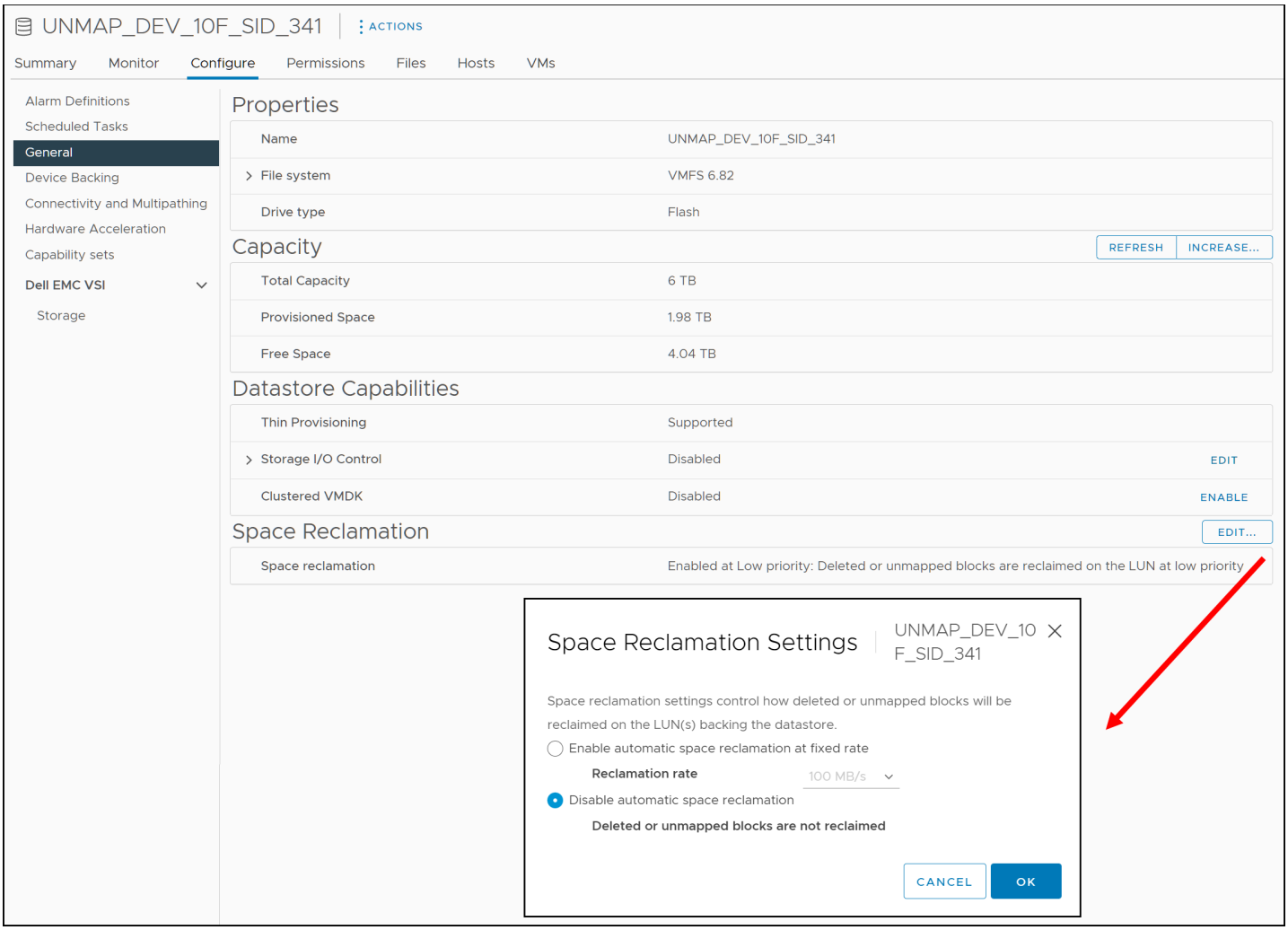


Figure 40. Modifying automated UNMAP on a VMFS 6 datastore

Changing the parameter is also possible through CLI, demonstrated in Figure 41.

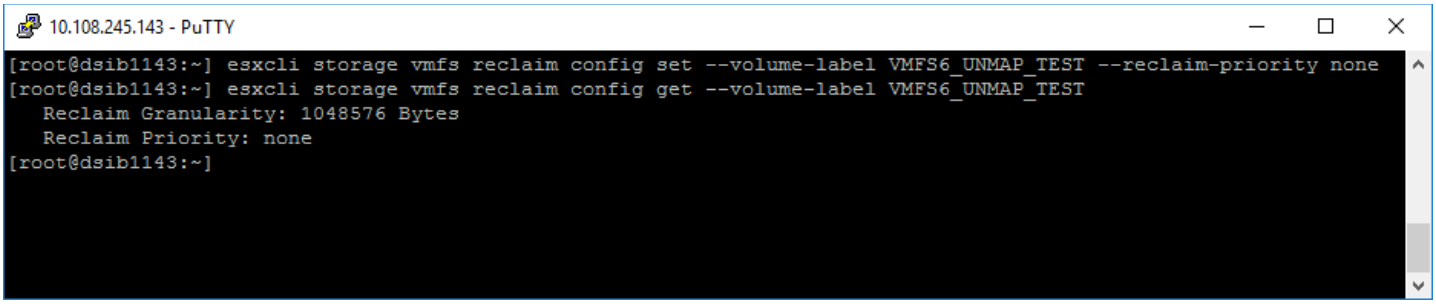


Figure 41. Modifying automated UNMAP through CLI

6.1.1 Reclamation priority

VMware offers the ability for the user to not only change the reclamation priority for UNMAP, but also to change the method from priority to a fixed of bandwidth if desired. You can change the priority from the default of low to medium or high, each having their own bandwidth setting. Medium will translate into 77 MB/s while high will be 256 MB/s. You can only change the priority, however, through CLI.

By default, the “Low” priority setting shown in Figure 39 translates into reclaiming at a rate of 26 MB/s. To change it to medium priority, issue the following command seen in Figure 42:

```
esxcli storage vmfs reclaim config set –volume-label RECLAIM_TEST_6A –
reclaim-priority medium
```

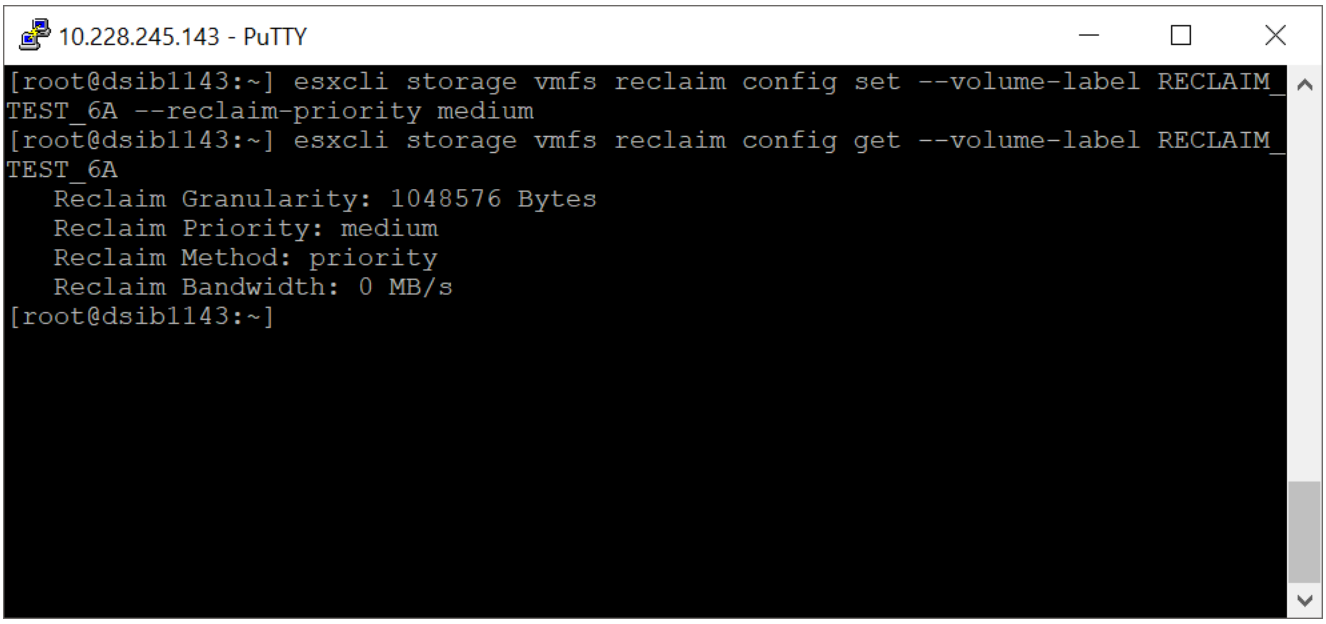


Figure 42. Modifying reclaim priority

To take advantage of the power of all flash storage arrays, VMware also offers the ability to set a fixed rate of bandwidth. The user can set the value anywhere from 10 MB/s to a maximum of 2000 MB/s. VMware offers this capability, unlike priority, in the UI. In the following example in Figure 43, we change the reclaim method to “fixed” at a 1000 MB/s bandwidth.

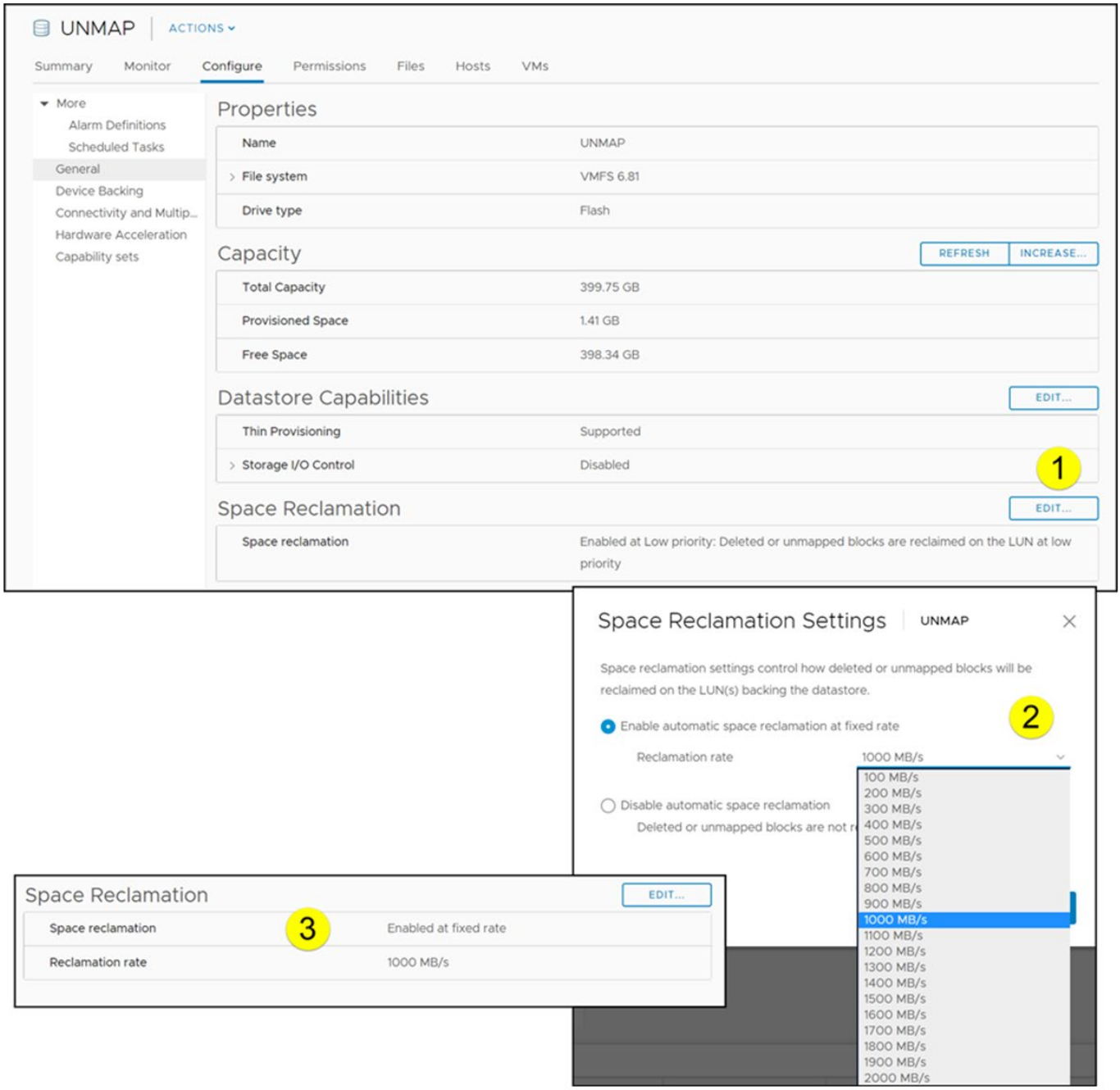


Figure 43. Modifying reclaim method to fixed

You can use the CLI also for the fixed method. For example, to change the reclaim rate to 200 MB/s on the datastore “RECLAIM_TEST_6A” issue the following:

```
esxcli storage vmfs reclaim config set –volume-label RECLAIM_TEST_6A –reclaim-method fixed –b 200
```

Below is the actual output in Figure 44, along with the `config get` command which retrieves the current value.

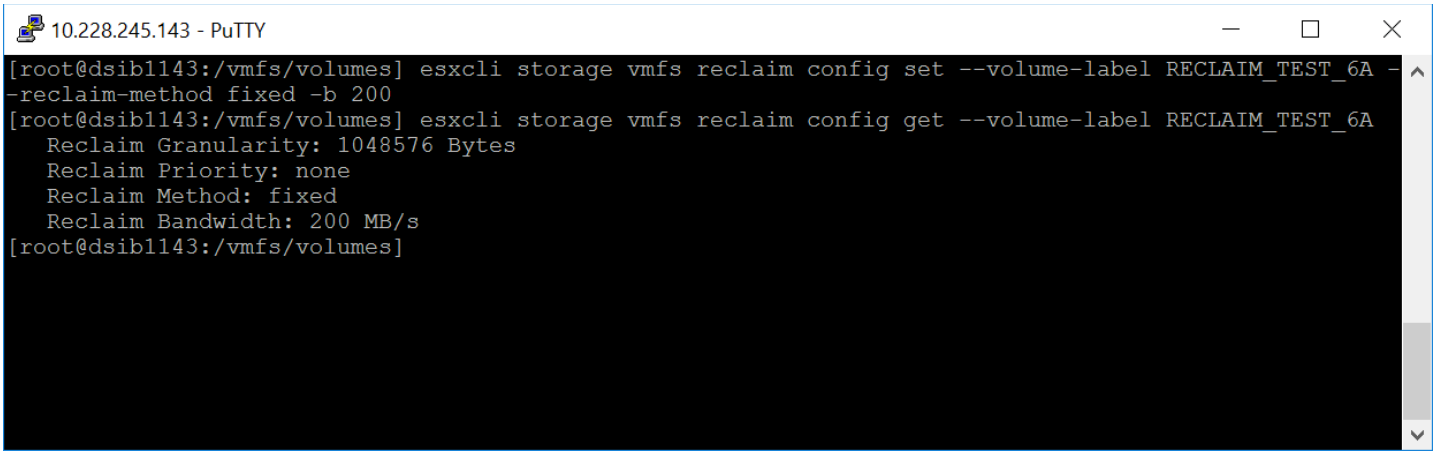


Figure 44. Changing reclaim priority

VMware has indicated that they do not advise changing the reclaim granularity.

Note: While an active UNMAP operation is underway, changing the bandwidth value will take longer than normal to complete.

One may notice that in Figure 42, the reclaim bandwidth shows as 0 MB/s, despite knowing the actual value is 75 MB/s. VMware will not show the correct value, however, unless the reclaim method is first set to fixed, then changed back to priority as in Figure 45.

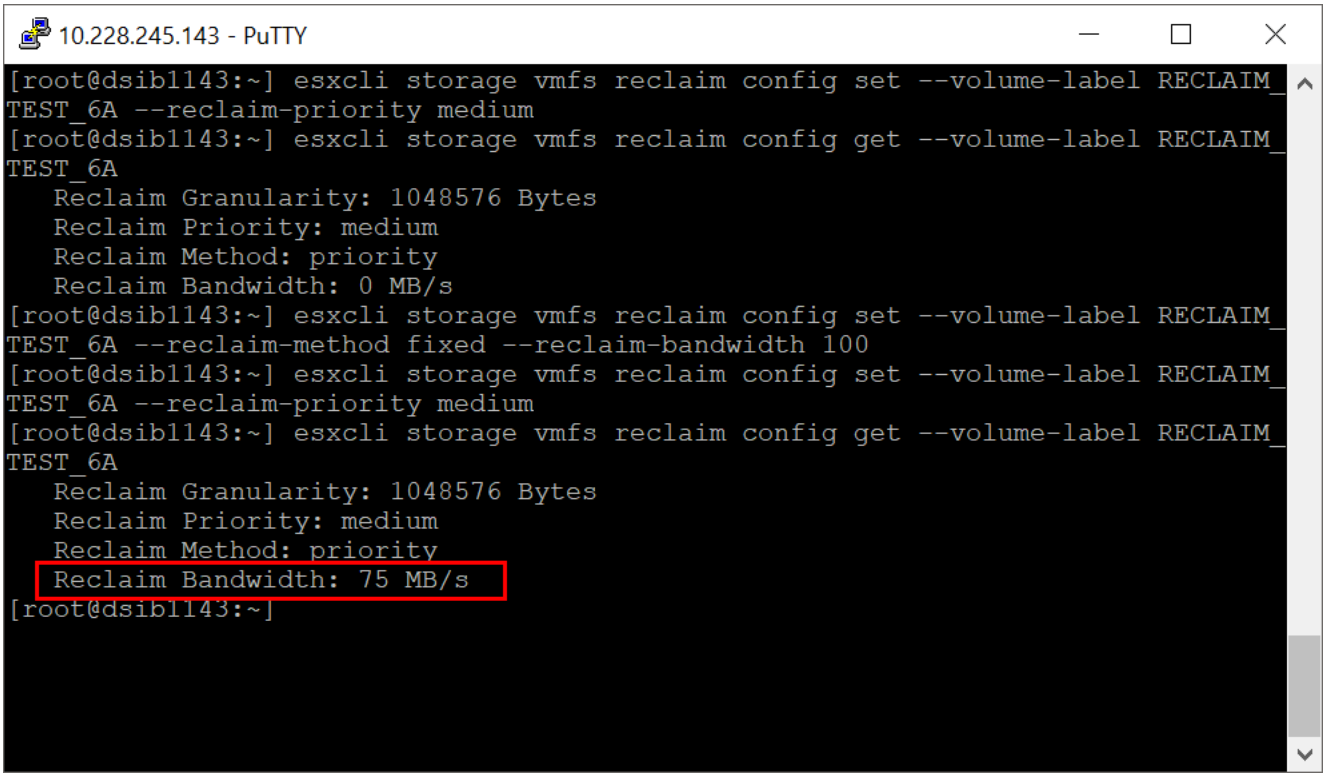


Figure 45. Showing the actual reclaim bandwidth of medium priority

6.1.2 Changing reclamation values in vSphere

There is an additional requirement when changing the UNMAP method and bandwidth or simply enabling/disabling UNMAP in the vSphere Client. After you make the change to the datastore on an ESXi host, the datastore must be unmounted and remounted on every other ESXi host that accesses the datastore for the change to propagate. If using CLI to make the change, the new value will take effect immediately on the initial ESXi host, but not on the other hosts until the unmount/remount process is complete. Although the unmount/remount process is the only one supported by VMware, changing the value on each ESXi host that accesses the datastore through CLI may also accomplish the same goal. This is not the case for the vSphere Client, however.

6.1.3 Reclamation hosts in 8.0U3

Beginning in 8.0U3 you can limit the number of hosts in a cluster that issue UNMAP. As explained in the section Automated UNMAP, VMware does not restrict the number of ESXi hosts that can issue UNMAP commands against the same datastore, regardless of the priority or fixed max. In 8.0U3, the user can now set a max value for reclaim hosts, thus ensuring the fixed or priority setting is honored. In Figure 46, we adjust the max value down to a single host from the default of 128.

```
[root@dsib1001:~] esxcli storage vmfs reclaim config set -l 1473_DS_1 -m priority -p low -n 1
[root@dsib1001:~] esxcli storage vmfs reclaim config get -l 1473_DS_1
   Reclaim Granularity: 1048576 Bytes
   Reclaim Priority: low
   Reclaim Method: priority
   Reclaim Bandwidth: 26 MB/s
   Reclaim Max Hosts: 1
[root@dsib1001:~]
```

Figure 46. Changing reclaim max hosts

6.1.4 Automatic UNMAP support for SESparse

For VM's with a SESparse snapshot (SESparse is the default snapshot format on VMFS 6), the space reclamation is also automatic. Note that VMware will not trigger automatic UNMAP for SESparse until it clears 2 GB of space.

6.2 Recommendations

Dell conducted numerous tests on UNMAP capabilities and recommends accepting the default setting of low priority for datastores. This minimizes the impact on VM IO, without regard to the type of storage backing the datastores. If the array is under heavy utilization, however, VMware offers the fixed value of 10 MB/s which may be more suitable to avoid contention.

When you delete a large number of VMs from a datastore that represent a significant amount of storage space (e.g., TBs), it is preferable to use one of the following options:

- If the goal is to empty and reuse the datastore completely (or even delete it), do not delete the VMs. Instead, power them down and remove them from inventory. Then delete the datastore. Remove the device from the storage group on the array and delete the device. This will avoid ESXi issuing any UNMAPs and the array will reclaim storage far more quickly. Create and mask a new device if desired.
  - If you are keeping a small number of VMs which are not too large, it is still better to disable automatic UNMAP and Storage vMotion the VMs to an existing or new datastore and proceed with the datastore deletion as above.
- If the goal is to only remove some of the large VMs from the datastore, disable automatic UNMAP before deleting the VMs. Then, during a maintenance window, run a manual UNMAP

to reclaim which will be faster and a more complete method for reclaiming space. Once the manual UNMAP completes, re-enable automatic UNMAP.

Note that Dell supports any setting for UNMAP and that these are best practices.

6.3 Manual UNMAP after VM deletion

This example will demonstrate the manual reclamation of a datastore's storage using ESXi CLI. The environment for the test consists of:

- A single device (4FD), in a storage group (data reduction enabled), presented to the ESXi cluster.
- 1 TB datastore, NO_AUTO_UNMAP, created on that device, with auto unmap disabled.
- 1 Windows VM, NO_AUTO_UNMAP_VM_1, with 600 GB of storage. Most of that storage is comprised of files on NTFS. There is minimal free space.

In Figure 47, VMware reports that the VM is occupying 601.42 GB. Unisphere for PowerMax has a similar allocation in Figure 48.

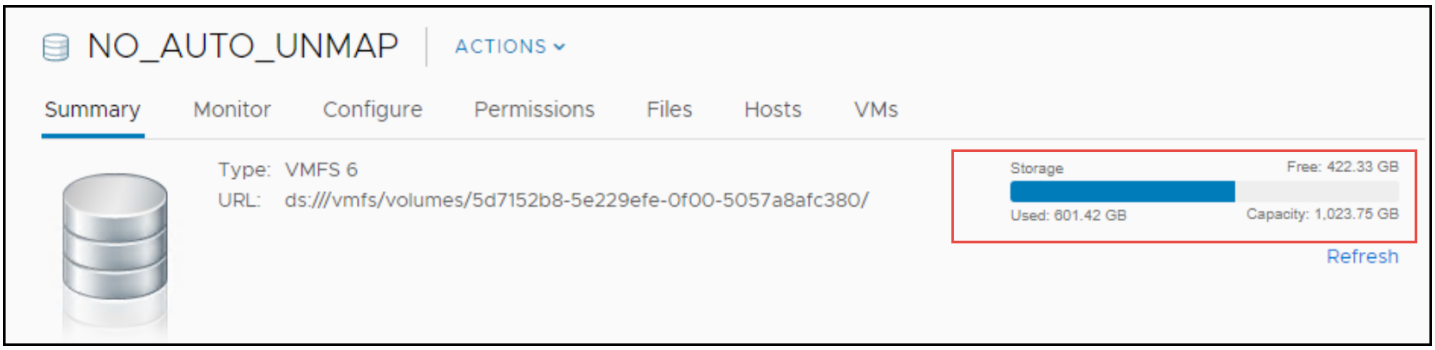


Figure 47. Datastore usage in vCenter

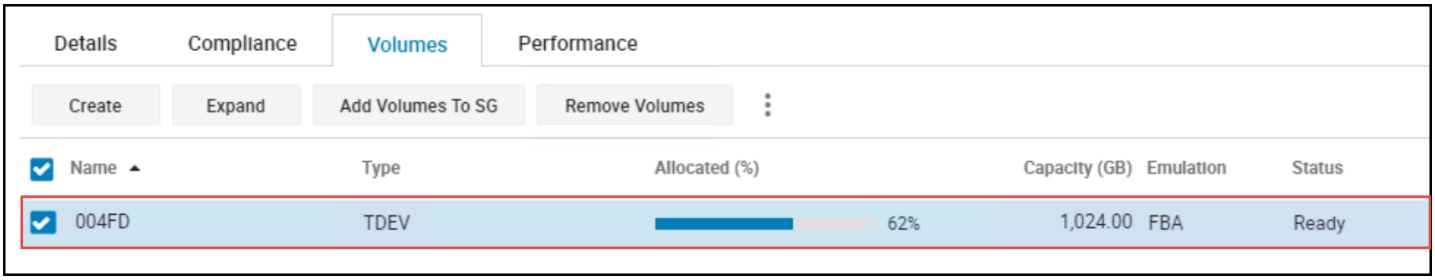


Figure 48. Obtaining device allocation in Dell Unisphere for PowerMax

More detail about how PowerMax has allocated the storage is available through SYMCLI. Note in Figure 49 how tracks are in different pools, depending on the compression. In addition, the allocated tracks are different from exclusive tracks since some data in this VM is already available on the array and thus has been deduplicated. So, while the VM would normally consume 600 GB of storage, it only needs 421 GB[9] on the PowerMax.

[9] You calculate the total by multiplying the number of exclusive tracks by the track size of 128 KB.

```
10.228.246.116 - PuTTY
dsib2116:~ # symcfg -sid 357 list -tdev -dev 4FD -detail

Symmetrix ID: 000197600357

Enabled Capacity (Tracks) :  430489080
Bound   Capacity (Tracks) :    8388615

                S Y M M E T R I X   T H I N   D E V I C E S
-------------------------------------------------------------------------------
                                    Pool          Pool    Exclusive   Comp
        Bound       Flags      Total  Subs     Allocated  Allocated  Ratio
Sym    Pool Name    ESPT       Tracks   (%)     Tracks (%)    Tracks
------ ------------ ----- ---------- ----- ---------- --- ---------- ------
004FD -             FS.B     8388615     -          0   0    3452190  1.0:1
      DG1_F_F       -.--           -     -    4696662  56          -      -
      DG1_F_1       -.--           -     -      19429   0          -      -
      DG1_F_6       -.--           -     -      99304   1          -      -
      DG1_F_2       -.--           -     -      11082   0          -      -
      DG1_F_8       -.--           -     -      47154   1          -      -
      DG1_F_9       -.--           -     -      19797   0          -      -

Total                     ---------- ----- ---------- --- ---------- ------
Tracks                       8388615     -    4893428  58    3452190

Legend:
 Flags:  (E)mulation : A = AS400, F = FBA, 8 = CKD3380, 9 = CKD3390
         (S)hared Tracks : S = Shared Tracks Present, . = No Shared Tracks
         (P)ersistent Allocs : A = All, S = Some, . = None
         S(T)atus    : B = Bound, I = Binding, U = Unbinding, A = Allocating,
                       D = Deallocating, R = Reclaiming, C = Compressing,
                       N = Uncompressing, F = FreeingAll, . = Unbound
dsib2116:~ #
```

Figure 49. Allocation detail with compression pools

Once you delete the VM or relocate it with Storage vMotion, VMware will recognize that the datastore is empty in Figure 50, but as automatic UNMAP is not enabled, the allocation on the array remains as it did in Figure 49.

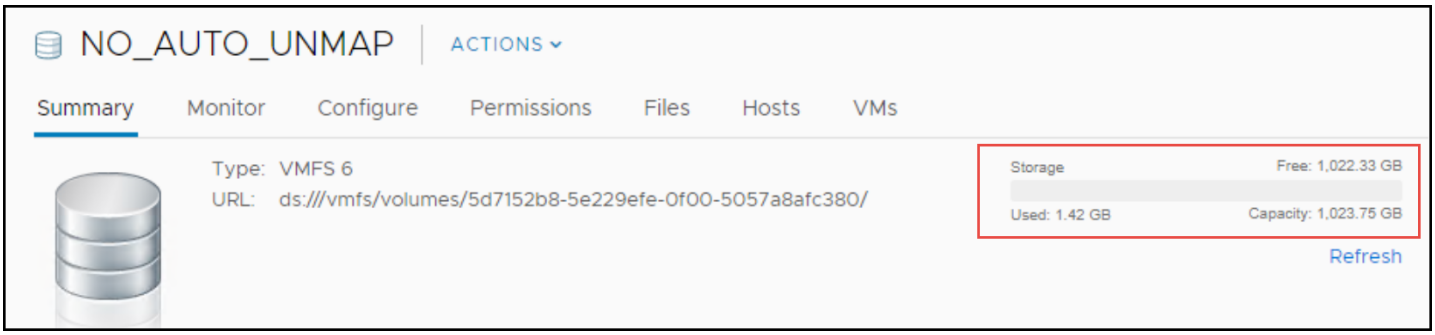


Figure 50. Datastore usage after VM deletion

At this point you can reclaim space using a manual UNMAP. One thing to understand about manual reclaim is that VMware cannot do a “precision” UNMAP like automatic reclamation. If auto unmap was enabled on this datastore, VMware would only send commands for those LBAs which used to represent the VM. With manual reclamation, VMware must iterate all the segments of the entire datastore, reclaiming what it can. Therefore, the time it takes to reclaim will be dependent both on

the size of the device and the amount of storage to reclaim, with the device size being the key factor in timing; however it is a more complete method for reclamation.

In the above example, we ran the following script in Figure 51 which records the time before and after the UNMAP command. Note we do not recommend changing the reclaim blocks from the default of two hundred, which is 200 MB reclaimed for each iteration. To reclaim the 421 GB on a 1 TB datastore took 58 seconds.

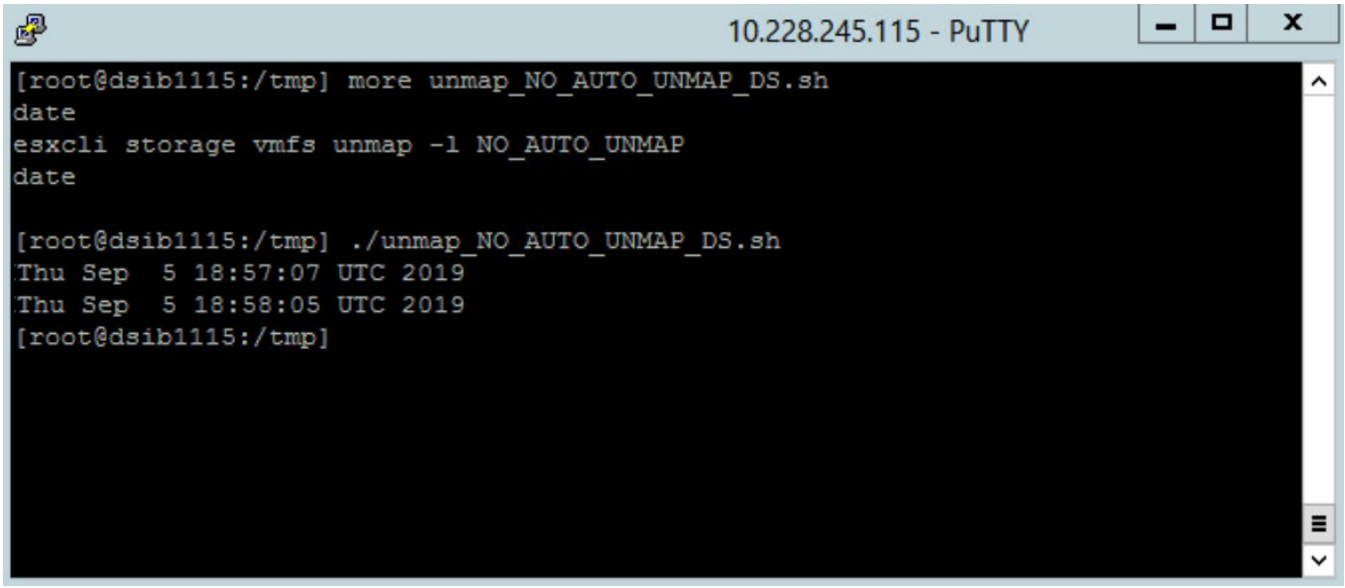


Figure 51. Manual UNMAP command

Here is the device allocation after the run in Figure 52.

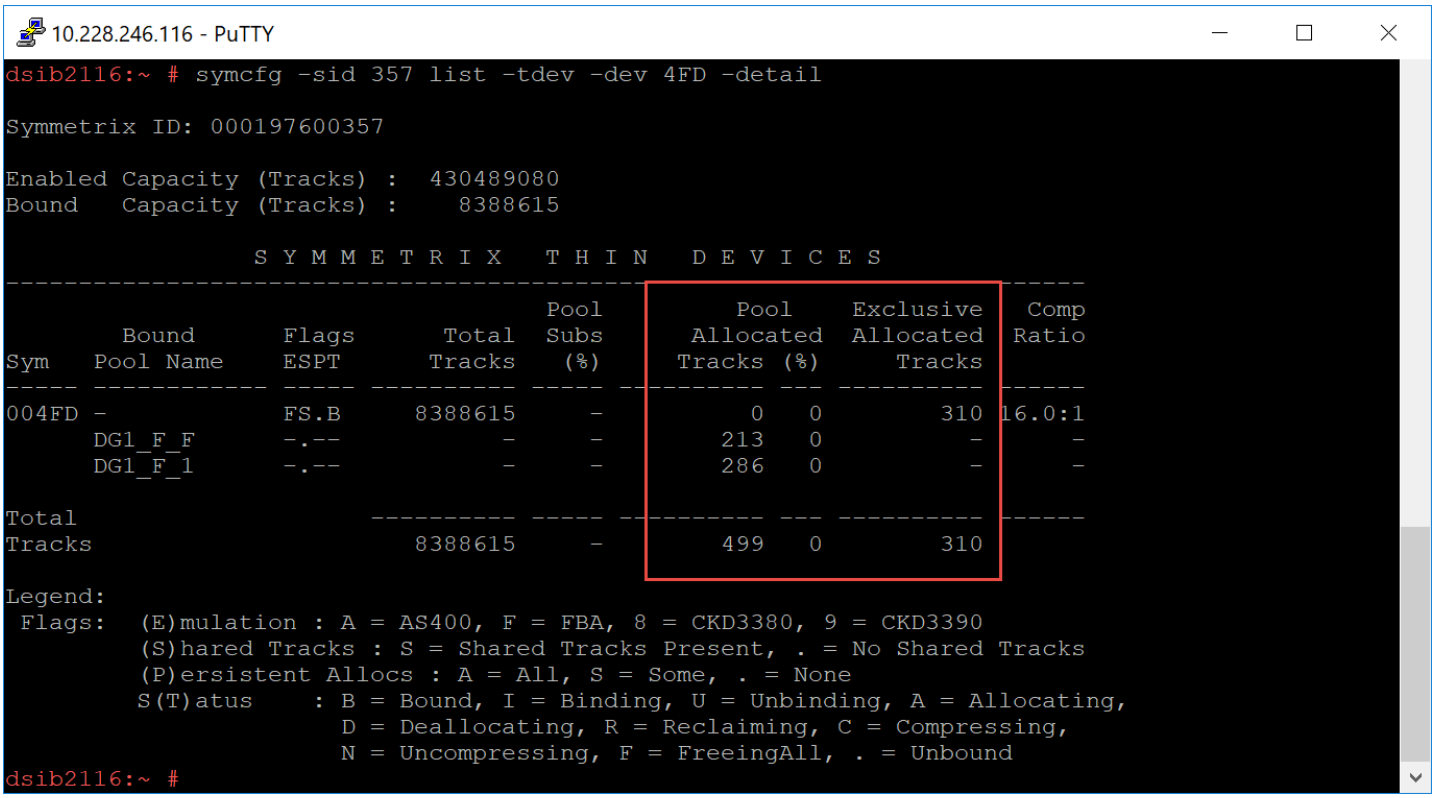


Figure 52. Device allocation after a manual reclaim

VMware reclaims all storage (possible) from the device. Note what happens in Figure 53 if you rerun the script, even though there is nothing left to reclaim. The run still takes 46 seconds. This is because VMware must examine all the segments even though there is nothing left to reclaim. Interestingly, if compared to the first run, it shows that the actual UNMAP process took about 12 seconds. Hence, device size is particularly important to consider when running a manual UNMAP. Extremely large datastores will take time and impact ongoing IO on the datastore, and therefore it is advisable to execute any manual reclamations during maintenance windows.

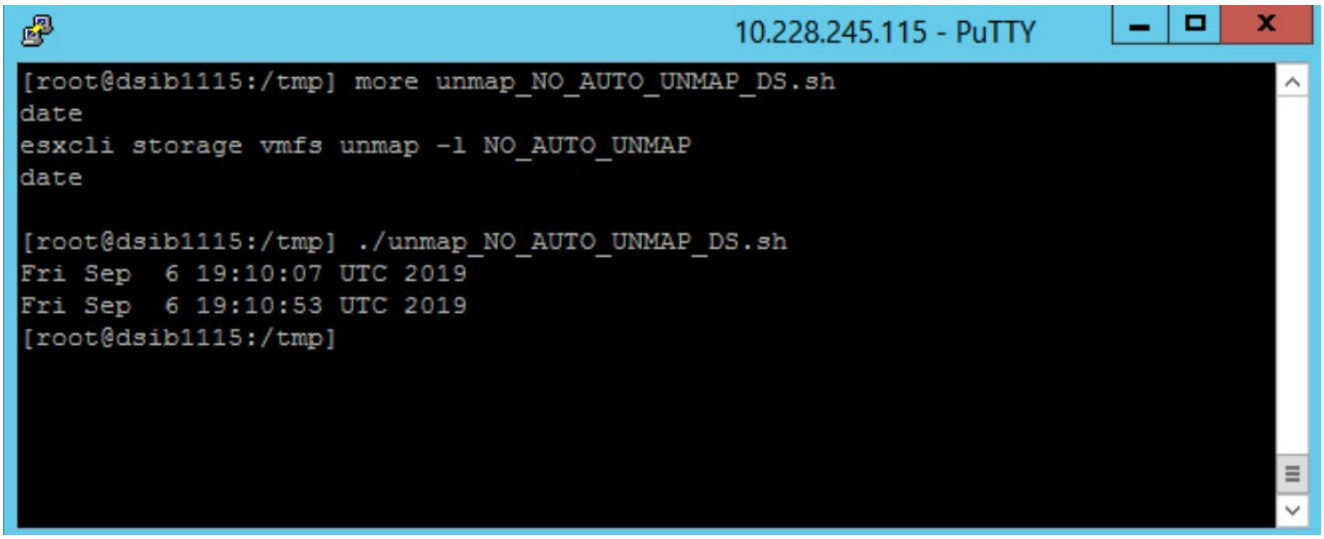


Figure 53. Running an additional reclamation

6.4 UNMAP internals

VMware reclaims dead space in multiple iterations. You can provide the number of blocks (the default is two hundred and recommended) to reclaim for each iteration. This specified block count controls the size of each temp file that VMware creates as part of the process. Since vSphere defaults to 1 MB block datastores, this would mean that if you do not pass a number, VMware will unmap 200 MB per iteration. VMware still issues UNMAP to all free blocks, even if empty. VMware will iterate serially through the creation of the temp file as many times as needed to issue UNMAP to all free blocks in the datastore.

6.4.1 Recommendation

When employing manual UNMAP, Dell recommends using VMware's default block size number (two hundred) based on internal testing, though fully supports any value VMware does. Below is an example of running manual UNMAP on a PowerMax, using the default block size. We conducted the test using a single 2 TB datastore, with auto unmap disabled. We cloned a completely filled 500 GB vmdk once, then twice, and finally three times. We then deleted the VMs and ran a manual UNMAP against each datastore. You can see the results in Figure 54.

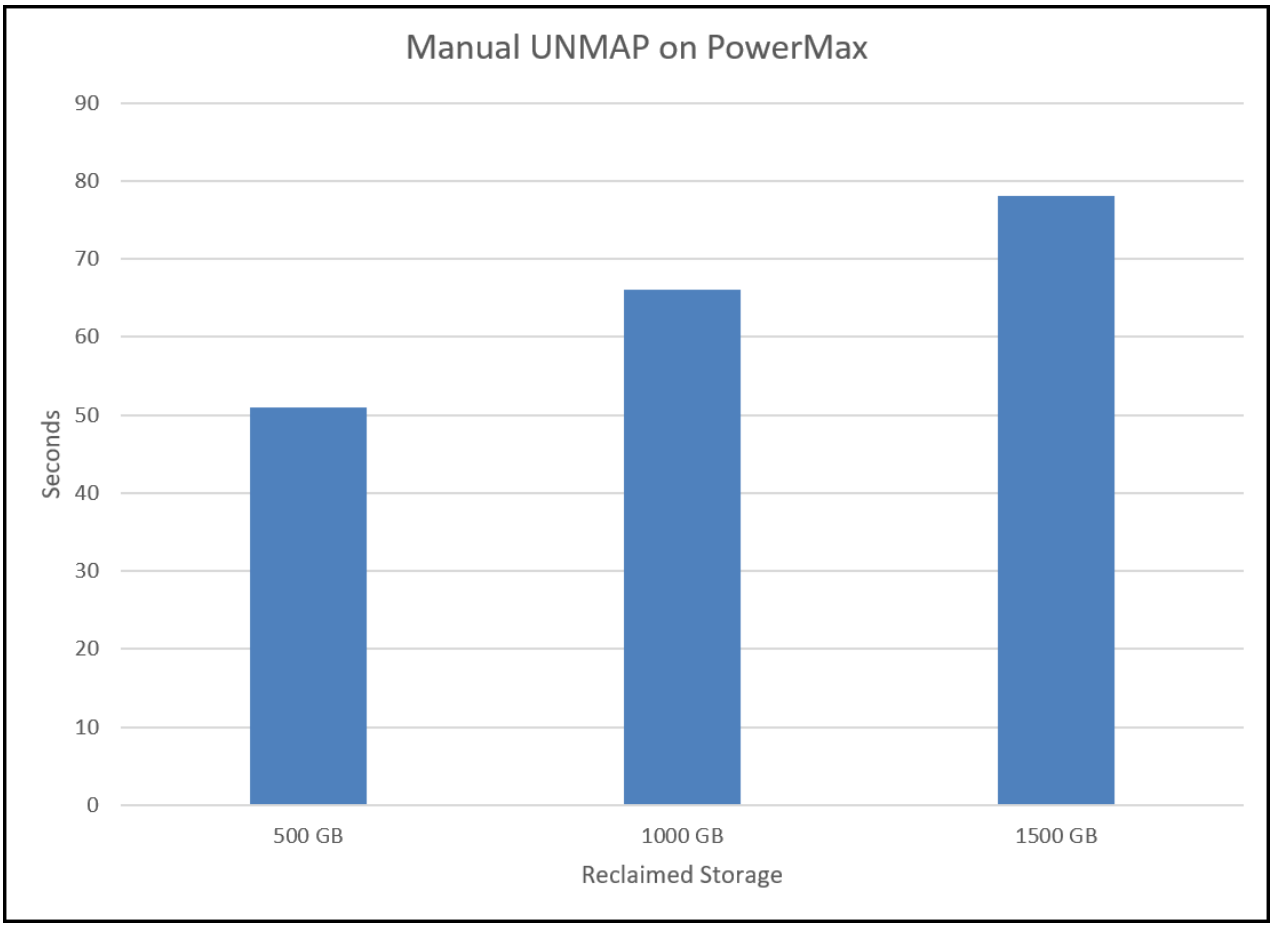


Figure 54. Manual UNMAP – PowerMax

6.5 Caveats for using UNMAP

The following are some general caveats that Dell provides when using this feature:

- Dell supports UNMAP on SRDF devices, however, though the R1 receives bulk commands from VMware, Dell breaks up those commands on the array before sending over to the R2 device. This means UNMAP, in general, increases SRDF traffic.
- UNMAP is an online activity which you can run on datastores with active VMs, however manually reclaiming substantial amounts of storage at one time can impact device and thus VM performance. Therefore, Dell recommends running any manual reclaim either during maintenance windows or during a time of lower business activity.
- When you run an UNMAP process on a device (datastore or GuestOS), the PowerMax prioritizes releasing the storage on the device over the thin pool. This means it is possible that even if the device reports the space as reclaimed, the array may still be freeing space from the thin pool and SRP. It is a background process which avoids impacting host IO. This is as Dell designed since it ensures that control of the device in VMware returns to the user as soon as possible so they can continue to work. Normally the asynchronous nature of UNMAP is not an issue, however if the thin pool or SRP as a whole is approaching capacity, it may be necessary to monitor the release of storage in the SRP after a manual UNMAP before re-using the space the device now shows as free. To avoid this problem in an environment where the SRP is approaching capacity, Dell does not advise performing many large, manual UNMAP procedures of datastores at one time. In addition, if using VMFS 6, Dell recommends automated UNMAP as the freeing of extents

occurs in smaller amounts, and only as needed. Freeing space from the SRP can take time, particularly if host IO is significant.

- Beginning with vSphere 8.0U2, VMware supports manual UNMAP on config vVols. VMware formats config vVols with VMFS 6 and therefore are a datastore. There is no automated UNMAP.
- Dell does not support UNMAP on devices in use by Open Replicator, RecoverPoint, or FTS.
- Dell blocks UNMAP on devices that have persistent allocations. The PowerMax accepts the UNMAP command but respects persistent allocations. If the specified UNMAP range has persistent and non-persistent allocations, the PowerMax will de-allocate the non-persistent allocations and leave the persistent allocations.

7 Monitoring VAAI operations with ESXTOP

The VMware ESXTOP tool provides a real-time view (updated every five seconds, by default) of ESXi Server performance details. ESXTOP lists CPU utilization for each physical processor, memory utilization, and disk and network bandwidth/information for each network and disk device available to the ESXi server. These statistics let you monitor the resource utilization for each of your virtual machines and the ESXi kernel. ESXTOP also provides VAAI statistics for each device present to the given ESXi host and allows users to monitor the number of commands being issued (if any) and if there are failures.

To use ESXTOP, login to the console of the ESXi host directly or via SSH.

You can run ESXTOP either in live mode to monitor in real time or batch mode to collect statistics over a period. These examples use live mode. For information on batch mode refer to VMware documentation.

You launch ESXTOP by executing the command `esxtop`. ESXTOP defaults to showing CPU information of current running processes. In order to see VAAI information, you must select the "disk device" screen. To change to this screen simply hit the "u" key (no quotation marks) on your keyboard and the screen will automatically change to the "disk device" screen (as seen in Figure 55) which will list the storage devices and their live statistics.

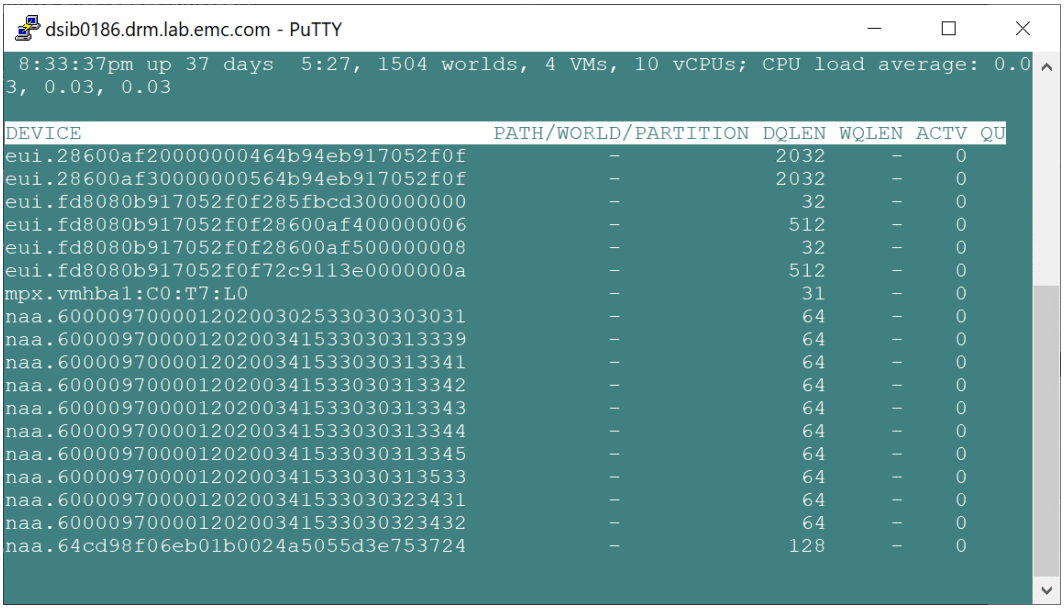


Figure 55. Disk device ESXTOP screen

For the purposes of VAAI monitoring, most of the statistics shown by default are unnecessary so you can remove them to make the display more manageable. To edit the view, press the "f" key on the keyboard and this will show the "Current Field Order" screen. In this screen the user can re-order or add/remove statistics. For VAAI monitoring you only need the columns "Device Name," "VAAI Stats" and the "VAAI Latency Stats." You can add or remove statistics by locating their assigned letter to the left of the field name and toggling the asterisk next to it by typing the associated letter. The presence of an asterisk means the column/field is enabled and will appear when the user navigates back to the "Disk Device" screen. In this example neither of the VAAI fields are enabled so they were enabled by pressing their respective assigned letters, "o" and "p." We disabled four other default fields which were unnecessary for VAAI purposes by toggling their asterisk by typing their respective lower-case letters. You can view these selections in Figure 56.

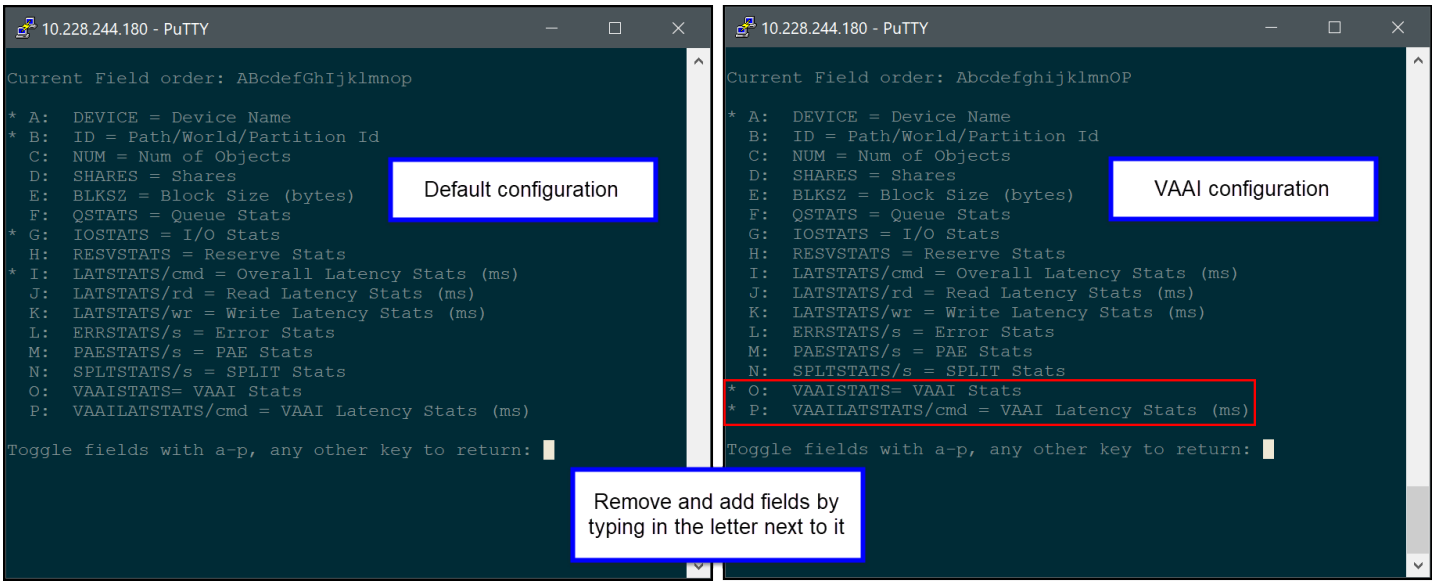


Figure 56. Adding VAAI statistics

Once you add or remove the desired statistics, return to the original screen by pressing return or enter on the keyboard. Figure 57 displays the VAAI statistic columns while Table 2 defines the most critical ones.

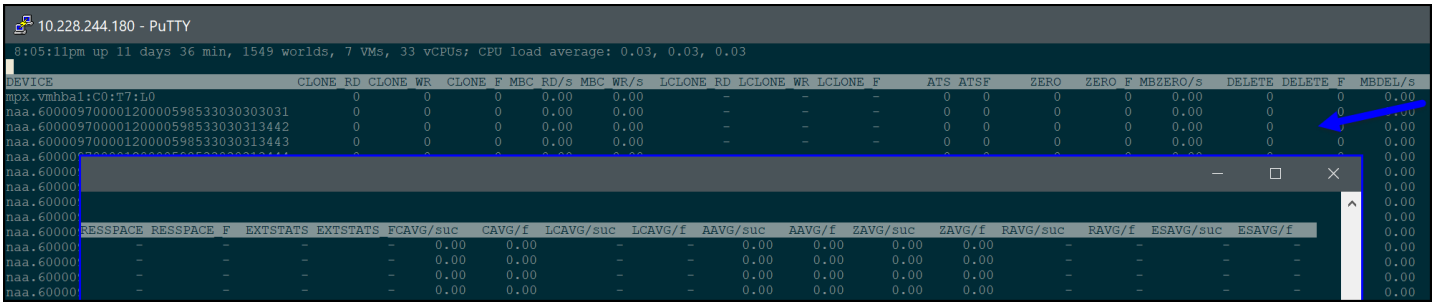


Figure 57. ESXTOP VAAI Statistic counters

Table 2. ESXTOP VAAI Statistics

Statistic	Description
CLONE_RD	The number of XCOPY commands successfully completed where this device was a source.
CLONE_WR	The number of XCOPY commands successfully completed where this device was a target.
CLONE_F	The number of failed XCOPY commands.
MBC_RD/s	XCOPY source reads throughput in MB per second.
MBC_WR/s	XCOPY target reads throughput in MB per second.
ATS	Successful hardware assisted locking/Atomic Test & Set commands
ATSF	Failed hardware assisted locking/Atomic Test & Set commands
ZERO	Successful WRITE SAME commands

ZERO_F	Failed WRITE SAME commands
MBZERO/s	WRITE SAME throughput in MB per second
DELETE	Successful UNMAP commands
DELETE_F	Failed UNMAP commands
MBDEL/s	UNMAP throughput in MB per second

Note that some VAAISTATS" counters are running totals and some are averages. The throughputs are averages for the last interval (default is five seconds but you can be change it by typing "s," inputting another time, and pressing enter) so if there has been no VAAI command in the last interval the number will be zero. All of the other commands are running totals since the last reboot.

A common use case for the ESXTOP VAAI statistics is to find out if the array is rejecting XCOPY because you notice a task that uses it, e.g., clone, is running slow. By watching the CLONE_F column for the given device, a user can watch for failed XCOPY commands to appear[10]. If the CLONE_F number increments and the CLONE_RD/CLONE_WR statistics are not incrementing for that device, VMware is using software copy. Review the device configuration to see if one of the known caveats is to blame. Be aware, however, that if VMware knows XCOPY does not apply, it will not even attempt the commands. An example of this is if the UUID is enabled on the VM. Figure 58 shows an ESXTOP example of an XCOPY failure that occurred after some successful commands. It reports four failed XCOPY operations before aborting XCOPY and reverting it to traditional software copy.

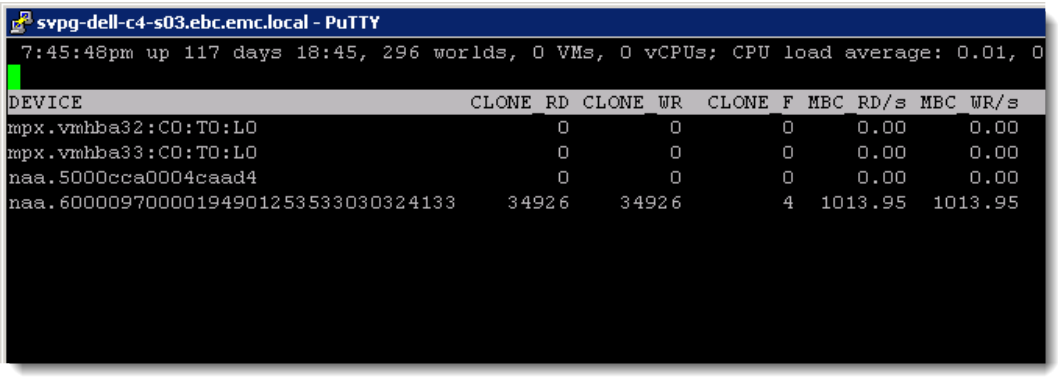


Figure 58. ESXTOP XCOPY Statistics

ESXTOP will only show the number of commands sent for XCOPY. It cannot show the user how large the extent size of each command request is. When using claim rules, it is important to get a baseline of a VM clone with the transfer size of 16 MB before adding the claim rule so that it will be easy to determine if the new 240 MB extent size is in use.

In addition to the "VAAISTATS" are the "VAAILATSTATS/cmd" which shows the average latency for the different VAAI commands in milliseconds. Table 3 shows the most important available statistics.

[10] The exact number of expected failures varies depending on the number of logical paths to the device(s), the Path Selection Policy (PSP), and the Multi-Pathing Plug-in (MPP). In general, however, this number will be low (single digits or low double digits).

Table 3. ESXTOP VAAI Latency Statistics

Statistic	Description
CAVG/suc	The average latency of successful XCOPY commands in milliseconds.
CAVG/f	The average latency of failed XCOPY commands in milliseconds.
AAVG/suc	The average latency of successful hardware-assisted locking/ATS commands in milliseconds.
AAVG/f	The average latency of failed hardware-assisted locking/ATS commands in milliseconds.
ZAVG/suc	The average latency of successful WRITE SAME commands in milliseconds.
ZAVG/f	The average latency of failed WRITE SAME commands in milliseconds.

If VAAI operations are taking a long time or you see failures, the VAAI latency statistics (Figure 59) are a valuable mechanism to help troubleshoot the problem. If for instance XCOPY sessions are running slower than normal but there are no rejections reported in the CLONE_F column, abnormally high latency could be a factor. If the latencies are much higher than the baseline this could indicate resource contention on the SAN fabric or the frontend of the PowerMax array. At that point, the System Administrator would need to troubleshoot the problem through Unisphere for PowerMax or other SAN fabric tools.

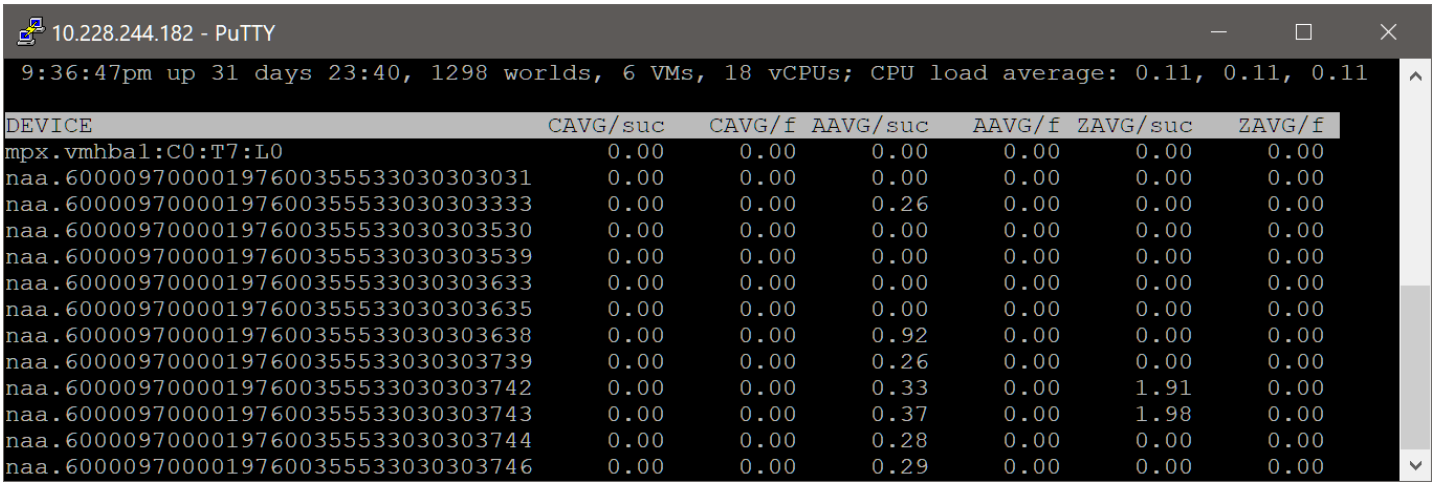


Figure 59. ESXTOP VAAI latency statistics

7.1 Monitoring VAAI with NFS

Unfortunately, ESXTOP will not show the user VAAI NFS activity. It is necessary to tail the vpxa log file, searching for “Vstorage” entries. Figure 60 has a sample output of an environment during a clone.

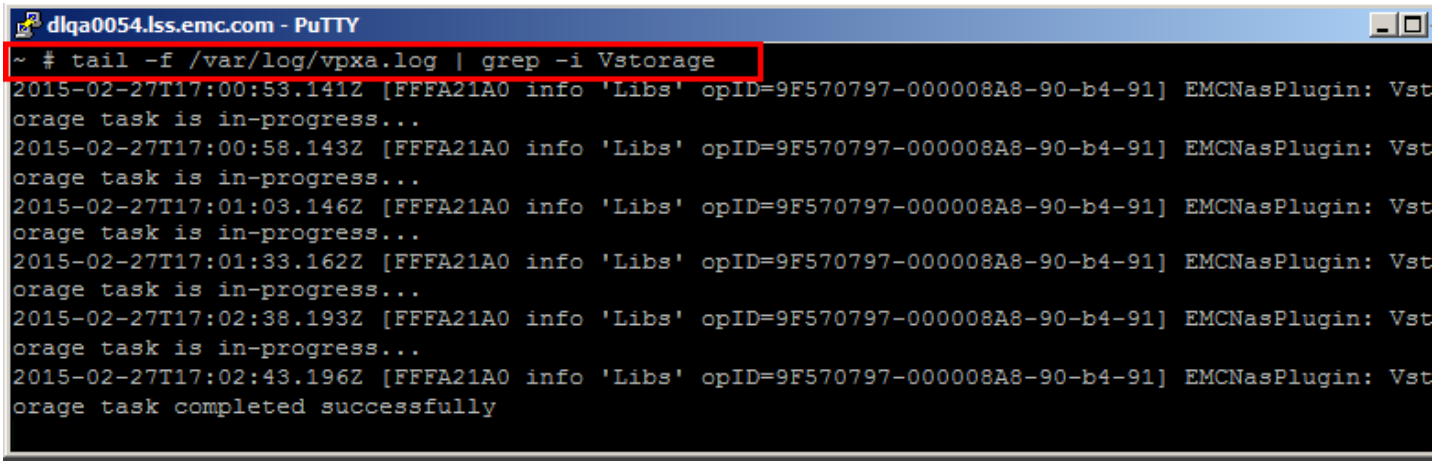


Figure 60. Monitoring VAAI commands on NFS

8 Performance

In the vast majority of customer environments, the VAAI primitives provide the benefit they intend by offloading tasks to the array with little performance impact, completing the tasks far quicker than VMware could. As noted above in the discussion around XCOPY and SRDF, however, there are scenarios where they can have an unintended impact to the environment. This is most common in arrays which are already under heavy stress and the extra commands from VAAI prove too much to process in an acceptable time for the customer. In these rare cases, it is likely that more than one VAAI primitive is being issued, for example WRITE SAME and UNMAP or XCOPY and UNMAP. When SRDF is part of the configuration, there are additional commands generated by SRDF as previously explained. When the issue does occur, some customers choose to disable one or more primitives, reverting to VMware's software solution. The UNMAP primitive is the most logical to disable both because you can do it at the datastore level, and because you can issue a manual UNMAP during low activity which will minimize the impact. You must disable XCOPY and WRITE SAME at the host level, which makes the option less appealing.

8.1 VDI

VDI environments which are highly dynamic, built up and torn down frequently, if not daily, can be susceptible to VAAI performance impacts. This is most common when deploying to a single large datastore with automated UNMAP enabled. Deploying VMs to the datastore issues significant XCOPY commands for the copy and then WRITE SAME commands to allocate new extents. When those VMs are torn down all at once, VMware sends the UNMAP commands to reclaim the storage which will include more WRITE SAME commands. For some customers, the impact will not be a problem given the array's resources, but for others it can be significant. For those latter customers you can take steps to mitigate the issue. By sizing the datastore closer to the actual usage of the desktops and disabling automated UNMAP, this ensures that both WRITE SAME and UNMAP will be issued only sparingly since VMware will reuse existing extents and over time the array will not need to allocate new ones for the device. The datastore can be resized on-the-fly if storage needs increase. Although you could also disable XCOPY, it is incredibly beneficial in VDI deployments and unlikely to cause issues with the WRITE SAME primitive neutered.

9 Conclusion

By leveraging VMware's Storage API integrations, VMware offloads specific operations directly to the Dell PowerMax, enhancing overall system performance and efficiency. The VMware Storage APIs for Array Integration include Full Copy, Block Zero, hardware-assisted locking, and Dead Space Reclamation. In supported VMware and PowerMax environments, these features are enabled by default and operate automatically on the PowerMax array without manual intervention.

A Technical support and resources

Dell.com/support the site focused on meeting customer needs with proven services and support.

A.1 Dell

- The Dell PowerMax and VMware vSphere Configuration Guide

  https://infohub.delltechnologies.com/t/dell-powermax-and-vmware-vsphere-configuration-guide-1/

- Using VMware vSphere Virtual Volumes and VASA with Dell PowerMax

  https://www.delltechnologies.com/asset/en-us/solutions/infrastructure-solutions/industry-market/h19812-vmware-vvol-vasa-powermax-whitepaper.pdf

- Unisphere for PowerMax

  https://www.dell.com/support/product-details/en-us/product/unisphere-powermax/overview

A.2 VMware

- VMware vSphere

  https://techdocs.broadcom.com/